

Eötvös Loránd Tudományegyetem

Informatikai Kar

Informatikatudományi Intézet

Programozási Nyelvek és Fordítóprogramok Tanszék

Háromdimenziós játékfejlesztési rendszer

Szerző:

Tamás Péter

Programtervező informatikus BSc.

Témavezető:

Pataki Norbert

Docens, PhD

Budapest, 2025

EÖTVÖS LORÁND TUDOMÁNYEGYETEM
INFORMATIKAI KAR

SZAKDOLGOZAT TÉMABEJELENTŐ

Hallgató adatai:
Név: Tamás Péter
Neptun kód: GR1ZH1

Képzési adatok:
Szak: programtervező informatikus, alapképzés (BA/BSc/BProf)
Tagozat : Nappali

Belső témavezetővel rendelkezem

Témavezető neve: *Pataki Norbert*
munkahelyének neve, tanszéke: ***ELTE IK, Programozási nyelvek és Fordítóprogramok Tanszék***
munkahelyének címe: ***1117, Budapest, Pázmány Péter sétány 1/C.***
beosztás és iskolai végzettsége: ***Docens, PhD***

A szakdolgozat címe: Háromdimenziós játékfejlesztési rendszer

A szakdolgozat témája:
(A témavezetővel konzultálva adja meg 1/2 - 1 oldal terjedelemben szakdolgozat témájának leírását)

A szakdolgozat célja egy háromdimenziós játékfejlesztési rendszer megtervezése és megvalósítása, amely kiemelten adat-orientált entity-component-system architektúrát alkalmaz. A rendszer fő struktúrája tehát ezen tervezési minta köré épülne, ahol az entitások alapvetően csak egy azonosítóból állnak, amikhez különböző komponenseket rendelhetünk hozzá. A komponensek feladata főleg az adatok tárolása, amiket lehetőleg strukturáltan, egymás után szorosan tárolok a memóriában, így biztosítva a hatékony CPU cache működését. A tényleges funkcionalitásért a systemek lesznek felelősek, amik a hozzájuk rendelt komponenseket frissítik, illetve további viselkedéseket is definiálnak. A rendszer fontos feladata a színtér grafikus megjelenítése, melyet OpenGL grafikus API segítségével biztosítok. A játékfejlesztési rendszer egy modern grafikus felhasználói interfésszel rendelkezik, amely elősegíti a fejlesztéseket. A szakdolgozatomhoz tartozik az objektumok ütközésvizsgálata és fizikai szimulációja. A program elkészítéséhez a C++ programozási nyelvet használom.

Budapest, 2023. 11. 10.

Tartalomjegyzék

1. Bevezetés 8

2. Felhasználói dokumentáció 9

2.1. Rendszerkövetelmények 9

2.2. A program telepítése 9

2.3. Projektek, játékjelenetek és felhasználói scriptek kezelése 11

2.3.1. Indítási paraméterek 11

2.3.2. Projekt generálása 11

2.3.3. Projekt betöltése 12

2.3.4. Játékjelenet elmentése 12

2.3.5. Játékjelenet betöltése 12

2.3.6. Felhasználói scriptek 12

2.3.7. Játék buildelése 13

2.4. Editor felépítése 14

2.5. Editor paneljei 15

2.5.1.1. A Viewport Panel 15

2.5.1.2. Komponens panel 16

2.5.1.2.1 Tag Panel 17

2.5.1.2.2 Transform Panel 17

2.5.1.2.3 Shape Panel 17

2.5.1.2.4 Model Panel 18

2.5.1.2.5 Animation Panel 18

2.5.1.2.6 Material Panel 19

2.5.1.2.7 Direction Light Panel 20

2.5.1.2.8 Point Light Panel 20

2.5.1.2.9 Spot Light Panel 21

2.5.1.2.10 Camera Panel 21
2.5.1.2.11 Box Collider Panel 22
2.5.1.2.12 Sphere Collider Panel 22
2.5.1.2.13 Convex Collider Panel 23
2.5.1.2.14 Mesh Collider Panel 23
2.5.1.2.15 Rigidbody Static Panel 23
2.5.1.2.16 Rigidbody Dynamic Panel 23
2.5.1.2.17 Water Panel 24
2.5.1.2.18 Script Panel 24
2.5.1.3. Entity Panel 25
2.5.1.4. A FileSystem Panel 26
2.5.1.5. Console Panel 27
2.5.1.6. Settings Panel 27
3. Fejlesztői dokumentáció 29
3.1. Felhasznált külső könyvtárak 29
3.1.1. GLEW (OpenGL Extension Wrangler Library) 29
3.1.2. GLFW (Graphics Library Framework) 29
3.1.3. GLM (OpenGL Mathematics) 29
3.1.4. IMGUI (Immediate Mode Gui): 29
3.1.5. STB_IMAGE 30
3.1.6. ASSIMP (Open Asset Import Library): 30
3.1.7. IRRKLANG 30
3.1.8. PHYSX 30
3.1.9. NLOHMANN JSON 30
3.1.10. TINYXML2 30
3.2. Játékmotor felépítése 31

3.3. Adatorientált Entity-Component-System elmélete és megvalósítása 32
3.3.1. Objektumorientáltság problémái és adatorientáltság elmélete 32
3.3.2. Component Pool megvalósítása 33
3.3.3. Registry megvalósítása 37
3.4. Játékmotor komponensei és rendszerei 42
3.4.1. Komponensek és rendszerek általános felépítése 42
3.4.2. Alapvető komponensek és rendszerek 43
3.4.2.1. Név azonosító 43
3.4.2.2. Transzformáció 43
3.4.2.3. Kamera 44
3.4.3. Geometriák 45
3.4.3.1. Egyszerű alakzatok (Shape) 45
3.4.3.2. Komplex alakzatok (Model) 46
3.4.3.3. Animáció 47
3.4.3.4. Anyagtulajdonságok 48
3.4.4. Fényforrások 49
3.4.4.1. Irány fényforrás 49
3.4.4.2. Pont fényforrás 50
3.4.4.3. Spot fényforrás 51
3.4.5. Optimalizációs eljárások 52
3.4.5.1. Frustum Culling 52
3.4.5.2. Árnyék Culling 53
3.4.5.3. Instancing előkészítése 54
3.4.6. Ütközési alakzatok és algoritmusok 54
3.4.6.1. AABB ütközés vizsgálat 54
3.4.6.2. GJK ütközés vizsgálat 55

3.4.6.3. Default ütközési alakzat 58
3.4.6.4. Téglatest ütközési alakzat 59
3.4.6.5. Gömb ütközési alakzat 60
3.4.6.6. Convex ütközési alakzat 60
3.4.6.7. Háromszög ütközési alakzat 61
3.4.7. Fizikai szimuláció 61
3.4.7.1. Statikus szilárdtest 62
3.4.7.2. Dinamikus szilárdtest 62
3.4.7.3. Fizikai tér szimulálása 63
3.4.8. További komponensek és rendszerek 64
3.4.8.1. Víz 64
3.4.8.2. Hang/Zene 65
3.4.8.3. Scriptek 65
3.5. Renderelés 66
3.5.1. OpenGL absztrakciók 66
3.5.1.1. Shaderek 66
3.5.1.2. Programok 67
3.5.1.3. Bufferek 67
3.5.1.4. VertexArray és Vertex struktúra 68
3.5.1.5. Textúrák 69
3.5.1.6. Framebufferek 70
3.5.2. Forward Shading vs Deferred Shading 71
3.5.3. Instance renderelés 72
3.5.4. Játékmotor Renderelési Fázisai 73
3.5.4.1. GeometryRenderer 73
3.5.4.2. Deferred Renderer 75

3.5.4.2.1 Általános technikák a renderelés során. 75
3.5.4.2.2 Ambiens fény szimulálása 76
3.5.4.2.3 Irányfényforrás szimulálása 77
3.5.4.2.4 Pontfényforrás szimulálása 78
3.5.4.2.5 Spotfényforrás szimulálása 80
3.5.4.3. Shadow Renderer 83
3.5.4.4. Wireframe Renderer 84
3.5.4.5. Billboard Renderer 85
3.5.4.6. Skybox Renderer 86
3.5.4.7. Bloom Renderer 87
3.5.4.8. Water Renderer 89
3.5.4.9. Preview Renderer 90
3.6. Játékmotor tesztelése és mintajátékok 91
3.6.1. Pool és Registry osztályok unit tesztelése 91
3.6.2. Ügyességi tesztjáték (End-to-end) 91
3.6.3. Grafikai tesztelés 93
4. Összefoglalás és további fejlesztési lehetőségek 94
5. Köszönet nyilvánítás 95
6. Irodalomjegyzék 95

1. Bevezetés

Miért is választottam ezt a témát? A grafikus programozással az ELTE grafika tárgyán keresztül ismerkedtem meg és a nagybeadandó elkészítése után éreztem azt, hogy engem érdekel ez a téma, és szeretnék többet megtudni a grafikus programozásról és jelenségekről. Szabadidőmben elkezdtem grafikával foglalkozó videókat, cikkeket olvasni, amelyeknek köszönhetően egyre több jelenséget értettem meg és valósítottam meg OpenGL-ben. Szakdolgozatomban egy háromdimenziós játékmotort készítettem el, amelyben egyszerűbb játékokat lehet létrehozni. A játékmotorban szinte minden eddig összeszedett grafikai tudásomat és tapasztalataimat összegeztem. A szakdolgozatom célja bemutatni, milyen lépések, komponensek és grafikai módszerek szükségesek ahhoz, hogy egy működő, modern interfésszel rendelkező játékmotor elkészíthető legyen. Úgy gondolom az elkészített szakdolgozat forráskódja nagyszerű lehetőség lehet grafikával ismerkedő, vagy éppen játékmotort implementáló hallgatóknak, ezért úgy döntöttem, hogy teljesen nyílt forráskódúvá teszem a szakdolgozatomat, amelynek a forráskódja a https://github.com/TamasPetii/GameEngine oldalon érhető el bárki számára.

De mégis mit értünk játékmotor alatt? Játékmotor alatt egy olyan szoftver keretrendszert értünk, amely akár 2D, akár 3D játékok fejlesztését segíti elő különféle belső komponenseket és funkcionalitásokat is biztosítva a felhasználónak. Egy játékmotor rengeteg részből tevődik össze, amelyek közül a legfontosabb a grafikai megjelenítés és jelenségek szimulálása, a valósághű fizikai szimuláció, ütközésvizsgálat, felhasználói scriptkódok írása és felhasználása és jól kezelhető felhasználói interfész biztosítása. Szakdolgozatomban minden fontosabb részre kitértek, részletesen bemutatom a komponenseket és jelenségeket, amelyek együttes működése teszi lehetővé a felhasználónak a játékok elkészítését.

2. Felhasználói dokumentáció

2.1. Rendszerkövetelmények

A játékmotor és az elkészített játékok kizárólag **64 bites Windows 10/11** operációs rendszeren futnak el. Mivel a játékmotor grafikai része a legfrissebb OpenGL konstrukciókat használja, így olyan modern videókártyára van szükség, amely támogatja az **OpenGL 4.6** verzióját. A felhasználói kódok írására, illetve maga a játékmotor forráskódjának fordítására a **Visual Studio 2022** fejlesztői környezetre van még szükség, ami tartalmazza a C++ projektek írásához szükséges állományokat és az **MSBuild** fordítóprogramot.

2.2. A program telepítése

A program telepítéséhez a feltöltött **.zip** állományt szükséges letölteni, majd kicsomagolása után hat mappát fog találni a felhasználó.

A **Runnable** mappa tartalmazza a játékmotor futtatható állományát. A Build mappában belül található az Editor.exe futtatható fájl, amely letöltés után azonnal elindítható. Az elindítás kezdetén első alkalommal felugrik egy ablakban, amelyben szükséges megadni a *Runnable* mappájának elérési útvonalát, és az MSBuild.exe fordító útvonalát. Erre a 2.3.1 pontban még részletesebben kitérek.

A **GameEngine** mappa tartalmazza a teljes játékmotor forráskódját Visual Studio 2022 projectekben. Amennyiben a felhasználó ezen keresztül szeretné elindítani a játékmotort, vagy fejleszteni szeretné a báziskódot, akkor először szükséges az Engine és Editor projectekben az *include*, *lib* és *dll* elérési útvonalakat beállítani a *Runnable* mappa **External** mappájának elérési útvonalára, mivel ez tartalmazza az összes külső könyvtár állományait. A legegyszerűbb megoldás erre, ha a felhasználó az External mappához létrehoz egy **U:/** allemezt, amit a terminálba az alábbi paranccsal érhet el:

subst U: <External mappa elérési útvonala>

Amennyiben a felhasználó nem szeretne ezzel a lehetőséggel élni, akkor a projekt beállításoknál a Debugging opció alatt lévő Environment paraméternél szükséges megadnia az U:\bin elérési útvonal helyett a External mappa bin elérési útvonalát. Hasonló módon a VC++ Directories opciónál az Include és Libarary directiories paraméterek U:\include,

U:\include\physx és U:\lib elérési útvonalait átírni az External mappa adott elérési útvonalára. A forráskódok fordítását kizárólag **Release** módban lehet elvégezni, mivel a külső könyvtárak ebben a módban lettek lefordítva. A játékmotor futtatása előtt az x64/Release mappába át kell másolni manuálisan az External mappa bin mappájának tartalmát, mert így lesz biztosítható, hogy a futtatható állomány és a unit teszt projekt eléri a szükséges .dll állományokat.

A **GameProject** mappa tartalmazza a 3.6.2 pontban bemutatott ügyességi tesztjáték projekt mappáját (nem a futtatható játékot). A futtatható játékmotor elindítása után a felhasználó a GameProject projektmappát megnyitva, az Assets/Scene mappában található Game.json játékfájllal betöltheti a tesztjátékot (2.3.3 és 2.3.5 mutatja be ezeket a lépéseket). Fontos, a játékproject VS22-es script projektjében a felhasználónak manuálisan át kell írnia az include, lib elérési útvonalakat a fentebb már említett módon. Továbbá az include paraméterhez a futtatható játékmotor (Runnable mappa) Engine mappájának az elérési útvonalát, míg a library paraméterhez a Build mappájának elérési útvonalát kell megadni, hiszen így fogja a script elérni a játékmotor állományait.

A **GameRunnable** tartalmazza a tesztjáték futtatható változatát, amelyet bármilyen paraméter átírás nélkül azonnal ki lehet próbálni.

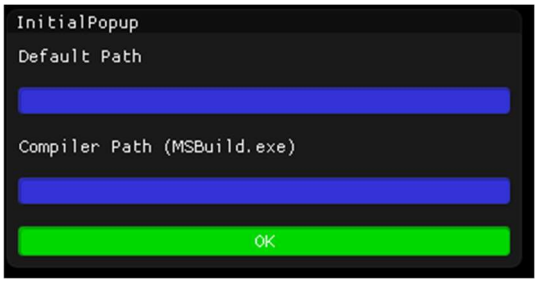
A **SponzaProject** mappa tartalmazza a 3.6.3 pontban bemutatott grafikai tesztprojektet. A projekt beállítása ugyanúgy történik, mint ahogy azt a GameProject esetén már bemutattam.

A **SponzaRunnable** mappa tartalmazza a grafikai tesztprojekt futtatható változatát, amelyet bármilyen paraméter átírás nélkül azonnal ki lehet próbálni.

Fontos: A futtatható játékmotorban létrehozott projektekhez legenerált Visual Studio script projektben nem szükséges a paramétereket beállítani, mert ezt a játékmotor automatikusan elvégzi, és beállítja a megfelelő elérési útvonalra. Onnantól kezdve, hogy a felhasználó elindította a játékmotort, minden megfelelően legenerálódik, minden útvonal megfelelően beállítódik. Akkor szükséges a paramétereket átírni manuálisan a fent említett módon, ha más számítógépen létrehozott projektmappát szeretne a felhasználó betölteni.

2.3. Projektek, játékjelenetek és felhasználói scriptek kezelése

2.3.1. Indítási paraméterek



1. ábra - Kezdetben felugró paraméter ablak

A játékmotor első elindításánál felugrik az 1. ábrán is látható inicializációs ablak, amelyben kötelezően meg kell adni a telepített játékmotor fájljait tartalmazó mappa elérési útvonalát, mivel ez a mappa tartalmazza az alapvető script projektet, ami a projectek generálásánál az új projektmappába fog átmásolódni. A script projektek játékmotorból történő fordításához meg kell adni második paraméterként a Visual Studio MSBuild fordító elérési útvonalát. Ez általában ezen az elérési útvonalon található meg:

C:/Program Files/Microsoft Visual Studio/2022/Community/MSBuild/Current/Bin/MSBuild.exe

A paraméterek megadása után többször nem ugrik fel az ablak indításkor, mivel az **%Appdata%/GameEngine/Init.json** fájlban elmentem a beállított útvonalakat, amiket innen olvas ki a játékmotor.

2.3.2. Projekt generálása

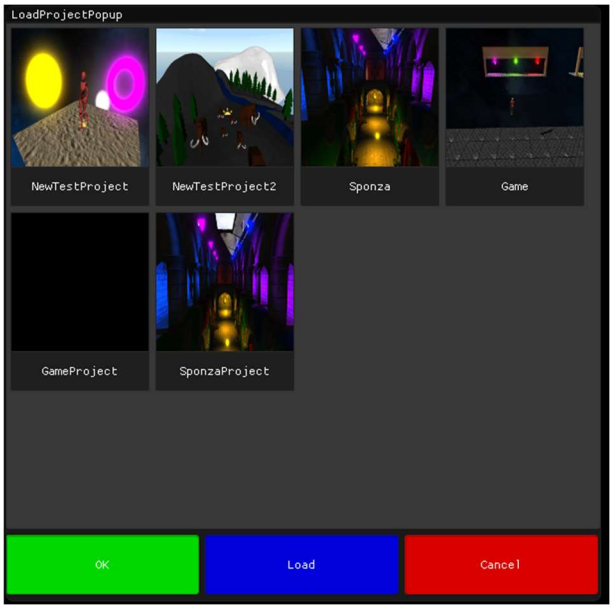


2. ábra - Új projekt létrehozása ablak

A játékprojektek generálását a fájl menü *New Project* menüpontjával érhetjük el, amely során felugrik egy ablak, amelyben a projekt nevét, illetve a projekt mappák és fájlok generálási útvonalát szükséges beállítani. A projekt generálása után a telepített játékmotor mappái közül az *Assets* és *Scripts* mappa tartalma átmásolódik a projekt mappába, így a felhasználó a játékprojektnek megfelelő asseteket és felhasználói scriptkódokat tud írni. A játékmotor által generált projectmappák érvényessége végett egy elrejtett **Project.json** fájl is generálódik, amelyet a felhasználónak tilos törölnie, vagy bármit benne megváltoztatnia.

2.3.3. Projekt betöltése

Már meglévő játékprojektet is betölthet a felhasználó a játékmotorba, ezt a *Load Project* menüpontra kattintva érheti el, amely hatására felugrik egy ablak, amelyben kártyás formában látható és kiválasztható az összes eddigi legenerált vagy betöltött játékprojekt, ezt mutatja be a 3. ábrán látható interfész ablak. Amennyiben a felhasználó által betölteni kívánt projekt nem jelenik meg az ablakban, akkor a *Load* gombra kattintva egy Windows fájlkezelő ablakon keresztül is lehetősége van a kiválasztott projektet betölteni.



3. ábra - Projekt betöltés ablak

2.3.4. Játékjelenet elmentése

A játékmotorban fejlesztett játékjelenetet a felhasználó tetszés szerint bármikor elmentheti. Az elmentett jelenet json formátumban tárolódik, és a teljes játékjelenethez tartozó entitásokat, komponenseket, globális beállításokat és más szimulációs paramétereket tartalmaz. A jeleneteket a felhasználó a fájlmenü *Save Scene* menüpontjára kattintva mentheti el, amely hatására felugrik egy Windows fájlkezelő ablak, amin belül kiválaszthatja az elmentendő játékjelenet nevét, illetve elérési útvonalát.

FONTOS: a fájl nevénél a *.json* kiterjesztést is oda kell írni a sikeres elmentés érdekében.

2.3.5. Játékjelenet betöltése

A játékmotor által elmentett játékjelenetet a felhasználó a *Load Scene* menüpontra kattintva betöltheti. Ekkor a felugró Windows fájlkezelő ablakban a betölteni kívánt játékjelenet json állományát kell kiválasztani, amit a játékmotor betölt, majd megjelenít.

2.3.6. Felhasználói scriptek

A legenerált játékmotor projektek tartalmaznak egy letisztított Visual Studio 2022 script project állományt, amelybe a felhasználó tetszőleges scriptkódokat írhat. Minden felhasználói script osztály a játékmotor által definiált, a 4. ábrán is látható Script ősosztályból származik,

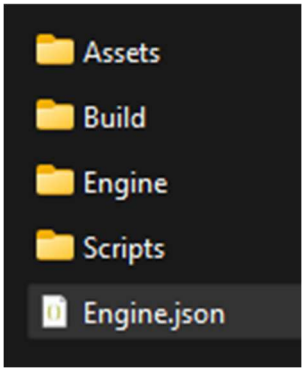
amely tartalmazza a hozzá csatolt entitás azonosítóját, és a teljes játéktér elemeit tartalmazó registry objektumot. Az osztály biztosít felüldefiniálható *OnStart*, *OnUpdate* metódusokat, amelyek a játék futása kezdetén, vagy folyamatosan minden frameben hívódnak meg. Tartalmaz továbbá *OnCollisionEnter*, *OnCollisionExit*, *OnTriggerEnter*, *OnTriggerExit* metódusokat is, amelyek a tényleges fizikai objektumok, vagy trigger alakzatokkal történő ütközés kezdetén, illetve végén hívódnak meg a játékmotor által. A felhasználónak lehetősége van tetszőleges ImGui interfészelem létrehozására is scriptből, amit az *OnGui* metódusban implementálhat, és ezt az Editor fogja meghívni, amely hatására tetszőleges UI ablakok jeleníthetőek meg a játékban. A felhasználói scripteket tartamazó VS22 projectet a játékmotor maga fordítja le, és tölti be a játék indításakor, illetve az Editor játéknézetbe való átváltásakor. Tehát a játékmotor támogatja a scriptek futás közbeni **hot reloading** megvalósítását.

```
class ENGINE_API Script
{
public:
    Script(std::shared_ptr<Registry> registry, Entity entity)
        : registry(registry), entity(entity) {}
    virtual ~Script() = default;
    virtual void OnStart() {}
    virtual void OnUpdate(float deltaTime) {}
    virtual void OnGui() {}
    virtual void OnCollisionEnter(Entity collidedEntity) {}
    virtual void OnCollisionExit(Entity collidedEntity) {}
    virtual void OnTriggerEnter(Entity collidedEntity) {}
    virtual void OnTriggerExit(Entity collidedEntity) {}
protected:
    Entity entity;
    std::shared_ptr<Registry> registry;
};
```

4. ábra - Script ősosztály

2.3.7. Játék buildelése

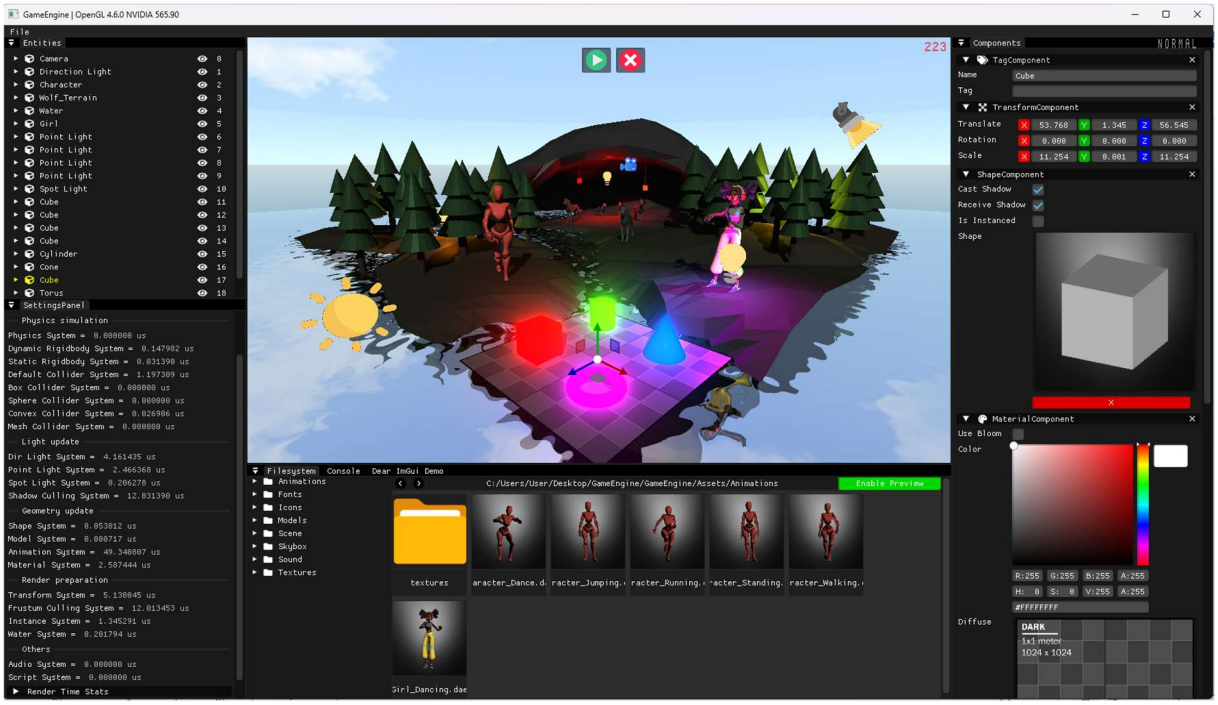
A játékmotorban elkészített játékok buildelése a fájlmenü *Build Project* menüpontjában érhető el, amely hatására egy önállóan futó játék generálódik le. A futtatható játék a telepített játékmotor futtatható állományát használja fel (tehát magát a játékmotort), annyi különbséggel, hogy a játék futása során az Editor felesleges paneljei nem jönnek létre. A buildeléshez szükséges elérési útvonal megadása után a játékmotor átmásolja a játékmotor futtatható állományát, és a szükséges könyvtárak dll fájljait. A rendereléshez szükséges shaderkódok, a felhasználó által megírt script project dll állománya is átmásolódik. Végül a játékprojekt teljes Assets mappája is átmásolódik, amely tartalmazza a játékhoz szükséges textúrákat, modelleket, animációkat és a betöltendő játéktereket leíró json fájlokat is. A játék elindításához a Build mappában található futtatható játékmotor állományt kell elindítani. A létrejött futtatható játék főmappái láthatóak az 5. ábrán.



5. ábra - Legenerált játék mappái

2.4. Editor felépítése

Az 6. ábrán látható az Editor, ami a játékmotor felhasználói felületét biztosítja, lehetővé téve többek között a színtérben lévő objektumok és azok komponenseinek testreszabását. Az interfészen keresztül a felhasználó könnyedén tölthet be modelleket, animációkat és textúrákat, valamint böngészhet a beépített fájlkezelő segítségével a számítógépén. Az editor felületén a felhasználó számos további hasznos információt is megtalálhat, ilyen például a belső motor rendszereinek benchmark idő adatai, illetve rengeteg további, a fejlesztést vizuálisan elősegítő opciót és beállítást.



6. ábra - Editor

Az Editor hat fő panelből épül fel:

- Viewport Panel (középen felül)
- Komponens Panel (a jobb oldalon)
- Entity Panel (bal oldal felső részén)
- FileSystem Panel (középen alul)
- Console Panel (középen alul, a FileSystem helyén)
- Settings Panel (bal oldal alsó részén)

2.5. Editor paneljei

Az editorhoz tartozó ablakokat az ImGui docking funkcionalitással hoztam létre. A felhasználónak lehetősége van a teljes editor megjelenésének átrendezésére. Az ablak nevek megfogásával az adott panel előtérbe kerül, majd annak mozgatásával újra pozícionálható, illetve a panelek egymásba/egymásra dokkolhatóak.

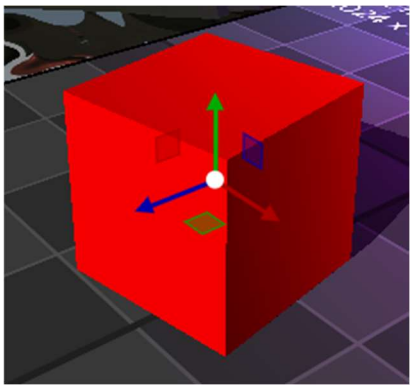
2.5.1.1. A Viewport Panel

A 7. ábrán látható Viewport Panel a színtér vizuális megjelenítéséért felelős, amelyen a fejlesztett játék 3D-s környezete valós időben megtekinthető és kezelhető. A panelre kattintva a jobb egér gombot nyomva tartva a felhasználó szabadon mozgathatja a színtér kameráját, a WASD gombok lenyomásával pedig megváltoztathatja a camera pozícióját a térben. A panel tetején két vezérlőgomb található: a zöld „Indítás” gomb lehetővé teszi az Editor nézetből a Game nézetbe való átváltást, ahol minden felesleges panel eltűnik, és a hangsúly a játék futtatásán van. Ilyenkor a szükséges scriptek betöltődnek, és a játék elindul. A piros „Leállítás” gombbal visszatérhetünk az Editor nézetbe. A játéktérben megjelenő objektumokra bal gombbal kattintva kiválaszthatjuk a szerkeszteni kívánt objektumot. Kiválasztást követően az objektumon a színtér aktív entitása lesz, és ezután megjelenik a transzformációkat elősegítő gizmo rajta, illetve azonnal frissül a komponens panel tartalma, ami az objektumhoz tartozó komponenseket fogja megjeleníteni.

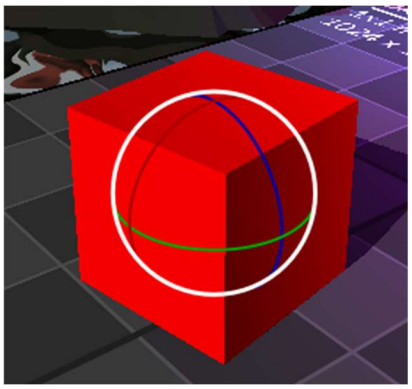


7. ábra - Viewport Panel

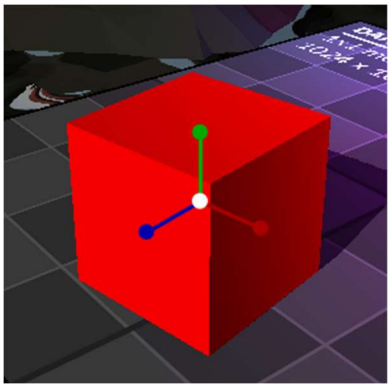
Az ImGuizmo könyvtár rengeteg ImGui-ra épülő összetettebb funkcionalitást biztosít, többek között a 8-10. ábrán látható gizmokat, animációs sávokat vagy akár gráfszerű paneleket is. A gizmo megjelenése és pozicionálása képernyőtérben történik, és a helyes 3D megjelenítés érdekében szüksége van a színtér kamerájának nézeti és vetítési transzformációjára, valamint az adott objektumhoz tartozó eltolás, forgatás és skálázás mátrixokra. Ezek a mátrixok biztosítják az ImGui panelen a gizmo pontos elhelyezését. Három különböző gizmo típus érhető el a felhasználó számára. Az „1” gomb megnyomásával az eltolást módosító gizmo jelenik meg, a nyilakra kattintva és azokat húzva az XYZ pozíció koordinátákat módosíthatjuk, a nyilak közötti síkok pedig az adott sík mentén történő mozgatást teszik lehetővé. A „2” gomb megnyomásával a forgatást, míg a „3” gomb megnyomásakor a skálázást módosító gizmo aktiválódik. A skálázás és a forgatás mechanizmusa hasonló elven működik, mint az eltolásé.



8. ábra - Eltolási gizmo



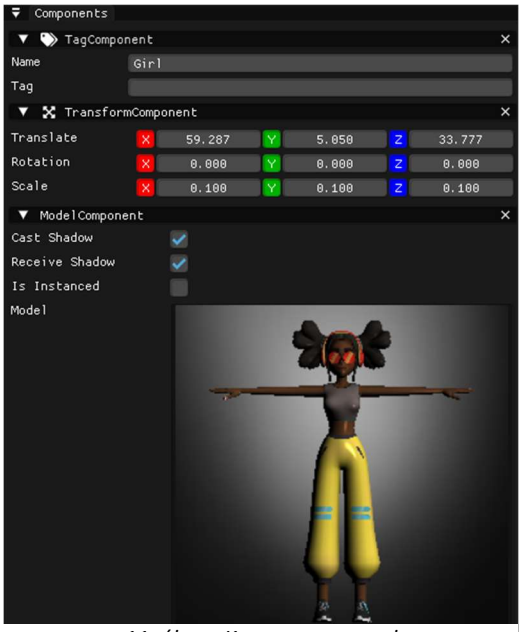
9. ábra - Forgatási gizmo



10. ábra - Skálázási gizmo

2.5.1.2. Komponens panel

A 11. ábrán látható komponens panel a kiválasztott entitáshoz tartozó komponenseket jeleníti meg. A komponensek paraméterei valós időben testreszabhatóak, a direkt és indirekt hatásuk a következő frameben fognak érvénybe lépni. A panelre jobb gombbal kattintva felugrik a komponens választó ablak, mely segítségével újabb, még nem felhasznált komponenseket tudunk az entitáshoz hozzárendelni. A komponensek jobb felső sarkában található egy X gomb, amire kattintva törölhetjük az adott komponenst.



11. ábra - Komponens panel

2.5.1.2.1 Tag Panel

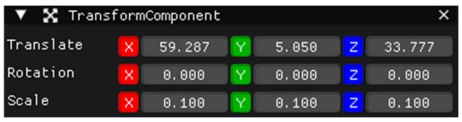
A 12. ábrán látható panel lehetővé teszi az adott entitások elnevezését, illetve tag megjelölését. A tag lényege, hogy bizonyos kategóriájú entitások (játékos, ellenfél, stb…) egy csoportként azonosíthatóak legyenek a felhasználói scriptekben.



12. ábra - Tag komponens panel

2.5.1.2.2 Transform Panel

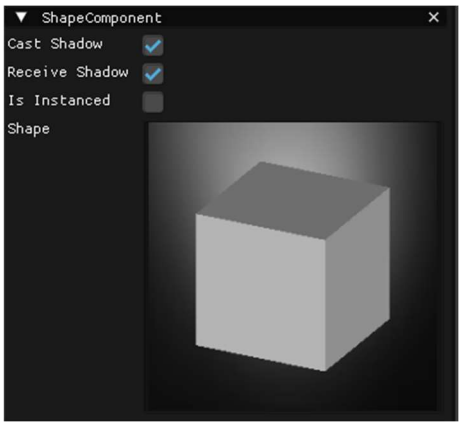
A 13. ábrán látható panel lehetővé teszi a kiválasztott entitáshoz tartozó eltolás, forgatás, skálázás paraméterek változtatását. Kifejtett hatása teljesen megegyezik a Viewport panelnél bemutatott gizmok hatásával. Az XYZ színes gombokra kattintva a hozzájuk tartozó értékeket visszaállíthatjuk alapértelmezettre.



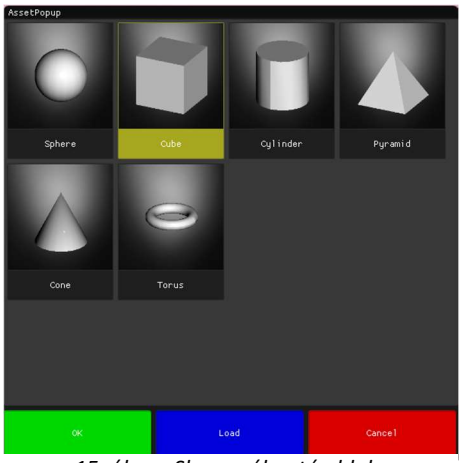
13. ábra - Transform komponens panel

2.5.1.2.3 Shape Panel

A 14. ábrán látható panel az egyszerű geometriákból álló entitásokhoz tartozó alakzatok paramétereinek beállítására szolgál. Beállíthatjuk, hogy árnyékszimulálás során vessen-e árnyékot az adott alakzat, illetve kapjon-e árnyékot más objektumoktól. Rendelkezésre áll egy *IsInstance* checkbox opció is, amelynek hatására az alakzat renderelése során a játékmotor egy másik renderelési technikát fog alkalmazni, ami sokkal hatékonyabb végkimenetelt eredményez. Erre későbbiekben részletesen is kitérek. Az aktuálisan kiválasztott alakzathoz tartozó előnézeti textúra is megjelenik a panelen, melyre bal gombbal kattintva felugrik az alakzat választó ablak, ez látható a 15. ábrán. Az ablak listázza az összes elérhető egyszerű alakzatot, ezeket kiválasztva egyszerűen megváltoztathatjuk az alakzatunk kinézetét.



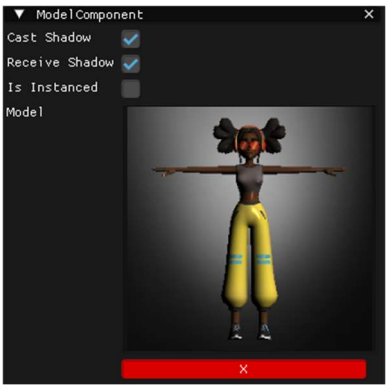
14. ábra - Shape komponens panel



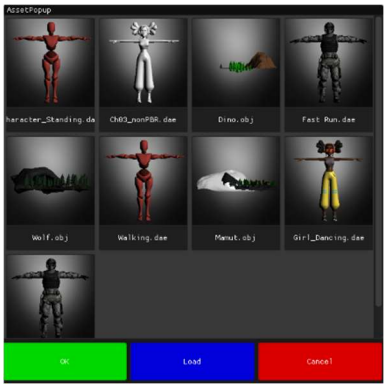
15. ábra - Shape választó ablak

2.5.1.2.4 Model Panel

A 16. ábrán látható panel a komplexebb geometriákból álló entitásokhoz tartozó modellek paramétereinek beállítására szolgál. A panel kialakítása nagyon hasonló a Shape komponens panelnél bemutatottra, lehetőségünk van az árnyék vetési és kapási paraméterek beállítására. A játékmotor támogatja a modellek Instance renderelését is, modellek esetén nagyon ajánlott az opció bekapcsolása. Az aktuálisan kiválasztott modelhez tartozó előnézeti textúra is megjelenik a panelen, melyre bal gombbal kattintva felugrik a model választó ablak, ezt mutatja be a 17. ábra. Az ablak listázza az összes már betöltött modelt, ezeket kiválasztva egyszerűen megváltoztathatjuk a megjelenítendő modellünket. Amennyiben nincsen kiválasztott model egy teljesen fekete textúra fog megjelenni.



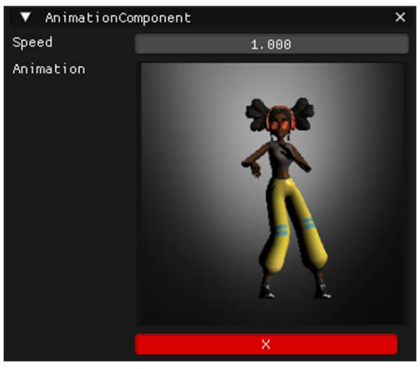
16. ábra - Model panel



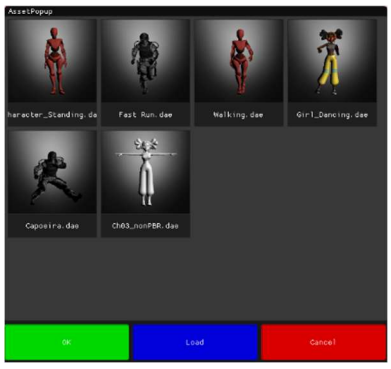
17. ábra – Model választó ablak

2.5.1.2.5 Animation Panel

A 18. ábrán látható panelen a felhasználónak lehetősége van a már kiválasztott modelhez animációt betölteni, illetve az animáció sebességét beállítani. Az aktuálisan kiválasztott animációhoz tartozó mozgó előnézeti textúra is megjelenik a panelen, melyre bal gombbal kattintva felugrik az animáció választó ablak, amelyet a 19. ábra mutat be. Az ablak listázza az összes már betöltött animációt, ezeket kiválasztva egyszerűen megváltoztathatjuk a megjeleníteni kívánt animációt.



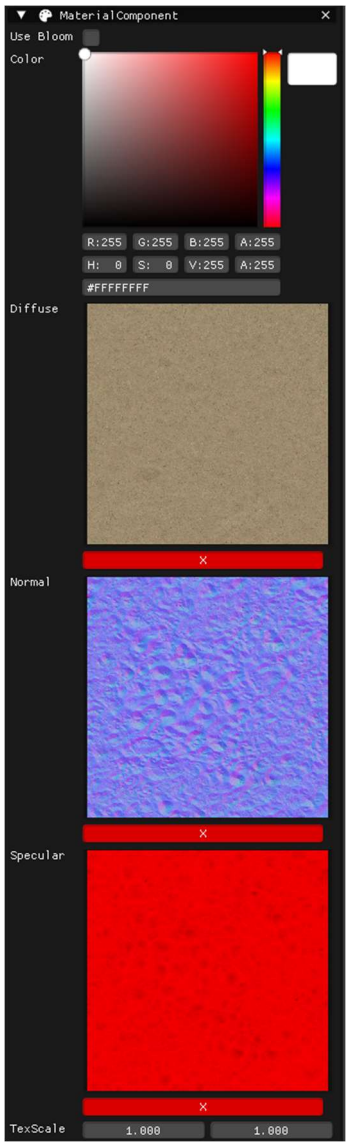
18. ábra - Animáció panel



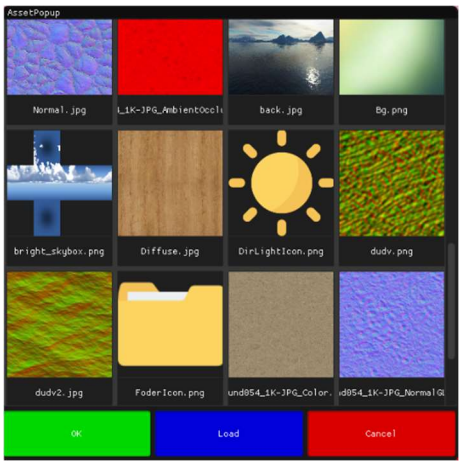
19. ábra – Animáció választó ablak

2.5.1.2.6 Material Panel

A színtérben megjelenő objektumokhoz tartozó anyagtulajdonsági paramétereket lehet testreszabni a 20. ábrán látható fülön. A felhasználónak lehetősége van az egyszerű alakzatok színét beállítani. Ehhez használhatja a dinamikus színválasztó felületet, de akár megadhat előre beállított rgb, illetve hexadecimális színkódot is. Amennyiben az objektumhoz világító hatást szeretnénk rendelni, akkor ezt a színválasztó felett található bloom effektust szabályzó checkboxon keresztül tehetjük meg. A részletgazdagabb anyaghatások eléréséhez természetesen lehetősége van a felhasználónak különféle textúrák betöltésére is. A játékmotor háromfajta textúra típust támogat: A diffúz textúra az anyag színét változtatja meg, a normál textúra a felülethez tartozó normálvektorokat módosítja, melynek hatására „hamis" valósághű mélyedés és árnyékolás érhető el a felületen, illetve a spekuláris textúra, ami a felületről visszaverődő fénymennyiséget szabályozza. A színválasztó felület alatt találhatóak meg a betöltött textúrák előnézetei. Amennyiben még nincsen betöltött textúra, egy teljesen fekete előnézet fog megjelenni. Az adott textúra típus előnézetére kattintva előugrik a textúra választó ablak, amely ablak listázza az összes eddigi betöltött textúrát, ezeket kiválasztva a felhasználó egyszerűen megváltoztathatja a megjeleníteni kívánt textúrát. Végezetül legalul található a textúra koordinátát szabályzó skála paraméterek. Elég sűrűn előfordulhat, hogy az adott alakzat mérete a színtérben túl nagy az adott textúra felbontásához képest, ami pixeles elmosódott megjelenéshez vezet. A textúra skála ezt a problémát oldja meg, XY irányba többször is megjeleníthetjük a textúránkat az objektumon, így egy kisebb részletekből álló valóságot jobban tükröző hatást érünk el.



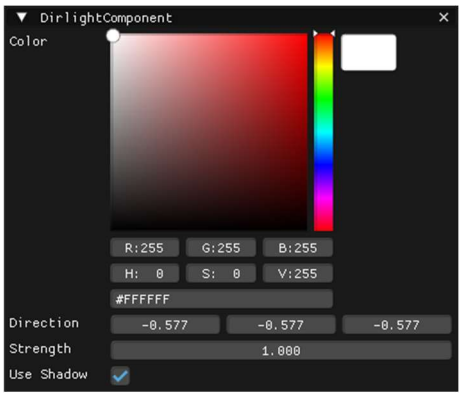
20. ábra - Material panel



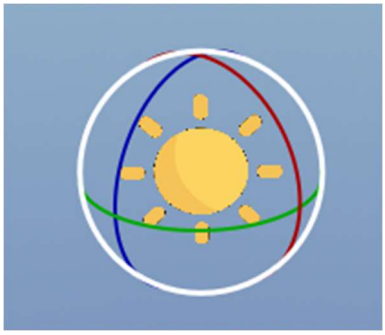
21. ábra - Textúra választó ablak

2.5.1.2.7 Direction Light Panel

Az irányfényforráshoz tartozó paramétereket lehet a 22. ábrán látható fülön testreszabni. Az anyagtulajdonsági résznél bemutatotthoz hasonlóan a felhasználónak lehetősége van a fényforrás színének módosítására a színválasztó felületen keresztül. A színválasztó alatt található meg a fény irányát befolyásoló *Direction* paraméter, illetve a fény erősségét szabályzó *Strength* paraméter. A fény irányát a felhasználó a Viewporton keresztül is beállíthatja, a színtérben lévő irányfényforráshoz tartozó ikonra kattintva megjelenik a forgatást szabályzó gizmo, ami mozgatásával a fény iránya megfelelően változtatható. Amennyiben szeretnénk árnyékot is szimulálni a fényforráshoz, akkor a *Use Shadow* checkbox aktiválásával tehetjük ezt meg.



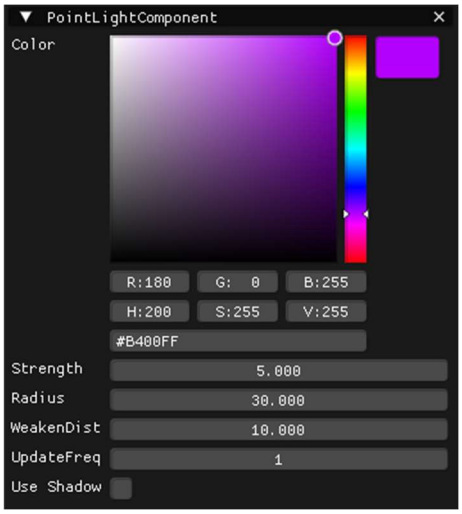
22. ábra – Irányfényforrás panel



23. ábra - Irányfényforrás Gizmo

2.5.1.2.8 Point Light Panel

A pontfényforráshoz tartozó paramétereket lehet a 24. ábrán is látható panelen beállítani. Az irányfényforrásnál bemutatotthoz hasonlóan a felhasználó módosíthatja a pontfényforrás színét, illetve a fényhatás erősségét. A pontfényforrás térbeli pozíció paraméterrel rendelkezik, ami a háttérben össze van kötve a Transform komponensnél bemutatott *Translate* paraméterrel. Tehát amennyiben a fényforrásunk hatását a színtér egy másik pontján szeretnénk alkalmazni, akkor a fent említett paramétert kell átállítanunk, amit természetesen a pontfényforráshoz tartozó ikonra kattintva, a megjelenő eltolást módosító gizmo segítségével is megtehetünk, ezt mutatja be a 20. ábra. Amennyiben a pontfényforráshoz szeretnénk árnyékot szimulálni, akkor a *Use Shadow* checkbox aktiválásával tehetjük ezt meg.



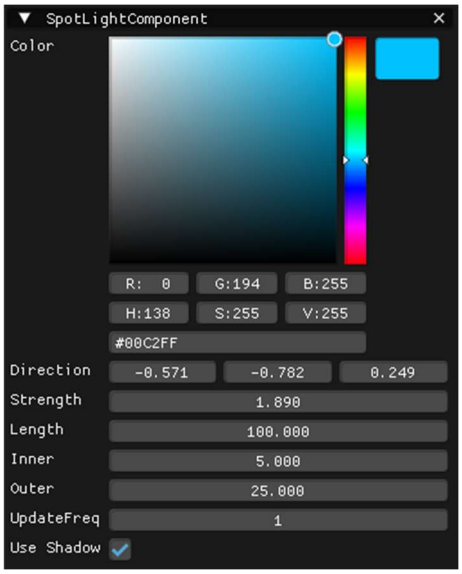
24. ábra - Pontfényforrás panel



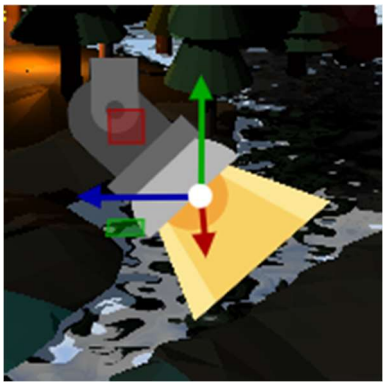
25. ábra - Pontfényforrás gizmo

2.5.1.2.9 Spot Light Panel

A spotfényforrás (zseblámpa szerű) a legösszetettebb a játékmotor által támogatott fényforrások közül. A felhasználó testre szabhatja a fényforrás színét a színválasztó felületen keresztül, illetve beállíthatja a szimulált fényhatás erősségét is. A spot fényforrás pozíció és irány paraméterekkel is rendelkezik. A fentiekben említett módon a pozíciót a Transform komponensen keresztül, míg az irányát az itt látható *Direction* paraméteren keresztül módosíthatjuk. A fényforráshoz tartozó ikonra kattintva megjelenő eltolást és forgatás módosító gizmo is használható a paraméterek egyszerű beállításának érdekében. A spotfényforrás kifejtett hatását egy kúp alakzat definiálja, mely kúp magasságát változtathatjuk meg a *Length* paraméteren keresztül, illetve a kúp bezárt szögét az *Outer* paraméteren keresztül. A fény szimulációjához tartozik a fényerősség csökkentése egy bizonyos szögértéket átlépve, ezt definiálja az *Inner* paraméter. A paraméteren belül teljes erősséggel, míg a paraméteren kívül szép lassan elhalványuló erősséggel jelenik meg a fényhatás. Amennyiben a spot fényforráshoz szeretnénk árnyékot szimulálni, akkor a *Use Shadow* checkbox aktiválásával tehetjük ezt meg.



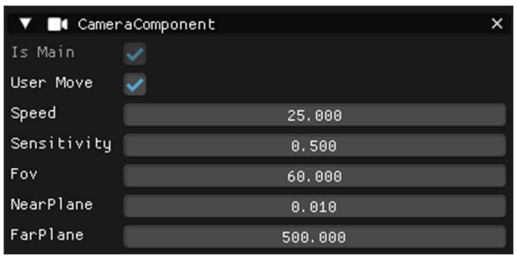
26. ábra - Spotfényforrás panel



27. ábra - Spotfényforrás gizmo

2.5.1.2.10 Camera Panel

A játéktérhez adott kamerák paramétereinek beállítását teszi lehetővé a 28. ábrán látható panel, ahol a kamera mozgási sebességét, érzékenységét, felbontását is testre lehet szabni. A felhasználói inputokat akár letilthatjuk, illetve az *Is Main* opció definiálja a játéktér főkameráját, amin keresztül látjuk a világot.



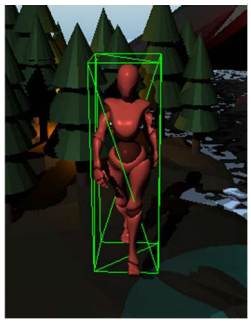
28. ábra - Kamera panel

2.5.1.2.11 Box Collider Panel



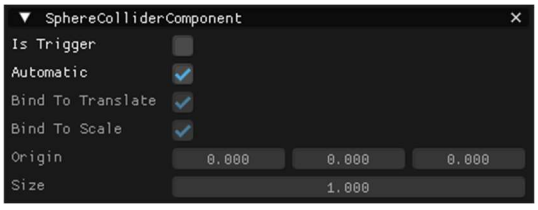
29. ábra – Téglatest ütközési alakzat panel

A fizikai szimulációban résztvevő entitásokhoz tartozó téglatest alakú ütközési alakzat paramétereinek beállítását teszi lehetővé a 29. ábrán látható panel. Az *Automatic* checkbox aktiválásával a felhasználó a játékmotorra bízza a téglatest paramétereinek meghatározását, amit a betöltött alakzathoz/modelhez tartozó adatok alapján számol ki. Amennyiben nem megfelelő ez a viselkedés a felhasználónak, akkor az *Origin* paraméterrel tetszőlegesen pozícionálható, a *Size* paraméterrel a téglatest mérete állítható. Alapértelmezetten az ütközési alakzat leköveti a hozzátartozó eltolás/skálázás változását, így elegendő álló helyzetben beállítani megfelelően a téglatest paramétereit, majd minden mozgást/újraméretezést automatikusan lekövet. Amennyiben ezt a felhasználó nem kívánja használni, akkor a *Bind to* paramétereknél kikapcsolhatja. Amennyiben az alakzatot csak trigger ütközési objektumként szeretnénk felhasználni, akkor az *Is Trigger* opció aktiválásával beállíthatjuk.



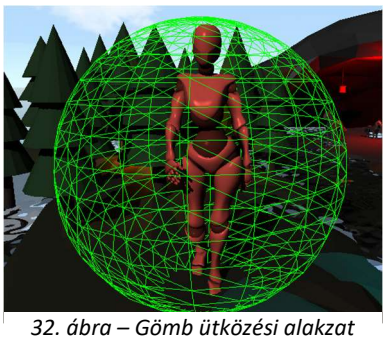
30. ábra – Téglatest ütközési alakzat

2.5.1.2.12 Sphere Collider Panel



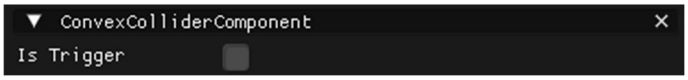
31. ábra – Gömb ütközési alakzat panel

A fizikai szimulációban résztvevő entitásokhoz tartozó gömb alakú ütközési alakzat paramétereinek beállítását teszi lehetővé a 31. ábrán látható panel. A komponenshez tartozó paraméterek viselkedése teljesen megegyezik a BoxCollider komponensnél ismertetekkel. Az *Origin* és *Size* paraméter a gömb alakzathoz tartozó paraméterek, amiket akkor állíthat be a felhasználó, ha kikapcsolja az automatikus paraméter generálási opciót.



32. ábra – Gömb ütközési alakzat

2.5.1.2.13 Convex Collider Panel



33. ábra - Konvex ütközési alakzat panel

A fizikai szimulációban résztvevő entitásokhoz tartozó konvex alakú ütközési alakzat paramétereinek beállítását teszi lehetővé a 33. ábrán látható panel. A komponens az entitás Shape/Model komponensénél betöltött alakzatot veszi inputként, és abból hoz létre egy konvex ütközési formát. A panelen a *trigger* opción kívül nincsen semmilyen további testreszabható paraméter.

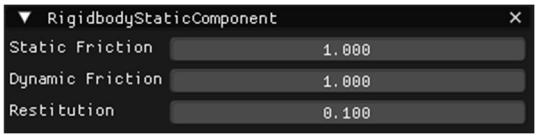
2.5.1.2.14 Mesh Collider Panel



34. ábra - Háromszög ütközési alakzat panel

A fizikai szimulációban résztvevő entitásokhoz tartozó háromszögeken alapuló alakú ütközési alakzat paramétereinek beállítását teszi lehetővé a 34. ábrán látható panel. A komponens az entitás Shape/Model komponensénél betöltött alakzatot veszi inputként, és abból hoz létre egy háromszögeken alapuló, pontos ütközési formát. A panelen a *trigger* opción kívül nincsen semmilyen további testreszabható paraméter.

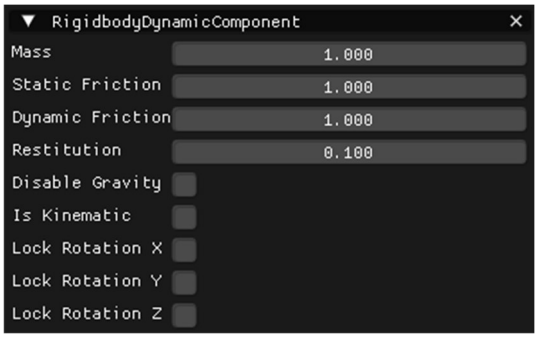
2.5.1.2.15 Rigidbody Static Panel



35. ábra - Statikus szilárdtest panel

A 35. ábrán látható panelen az állandó helyzetű, gravitációval nem kölcsönható statikus fizikai szilárdtestek paramétereit állíthatjuk be. A komponens hatására fog az entitás részt venni a fizikai szimulációban. A panelen a statikus és dinamikus súrlódási paramétereket, illetve pattogás erősségét állíthatjuk be.

2.5.1.2.16 Rigidbody Dynamic Panel



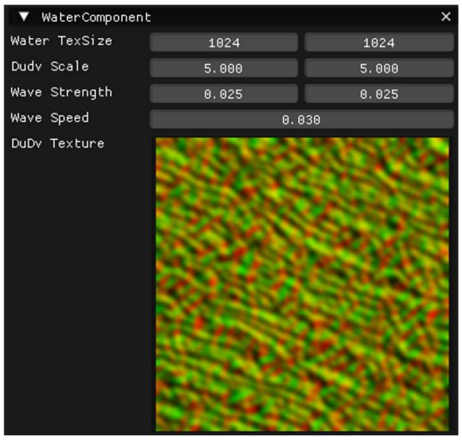
36. ábra - Dinamikus szilárdtest panel

A 36. ábrán látható panelen az erőhatásokra reagáló, irányukat és sebességüket változtató dinamikus fizikai szilárdtestek paramétereit állíthatja be a felhasználó. A komponens hatására az entitás részt vesz a fizikai szimulációs térben. A panelen beállíthatjuk a test tömegét, statikus és dinamikus súrlódási paramétereit, illetve a

pattogást szabályzó erőhatás erejét is. Továbbá lehetősége van a felhasználónak kikapcsolni a gravitáció szimulálást, illetve az XYZ irányba történő ütközések által felmerülő forgatási erőhatásokat.

2.5.1.2.17 Water Panel

A 37. ábrán látható Water komponens panel segítségével fog tudni a felhasználó 2D vízfelszínt szimulálni a színtérben. Az első opció a vízszimulálás előzetes lépéseiben felhasznált belső textúrák méretét módosítják, lehetővé téve akár az extra részletesség, akár a jobb teljesítmény alapú megvalósítást. A víz mozgását az úgynevezett dudv textúra határozza meg, amelyre egy példát a 38. ábra mutat. Az éppen aktuálisan kiválasztott textúra előnézete megjelenik a panelen, melyre kattintva előugrik az anyaginformációs panelnél bemutatott textúra választó ablak, ahonnan szabadon betölthetőek további dudv textúrák. Amennyiben még nincsen kijelölt textúra a komponenshez, akkor egy fekete terület jelenik majd meg. A *Dudv Scale* hasonlóan segít XY irányba megtöbbszörözni a textúrát, így nagy felületen is kellően részletes tud maradni a vízszimuláció. A *Wave Strength* és *Speed* paraméterek a vízmozgás gyorsaságát, illetve hullámok erősségét, méretét állítják be.



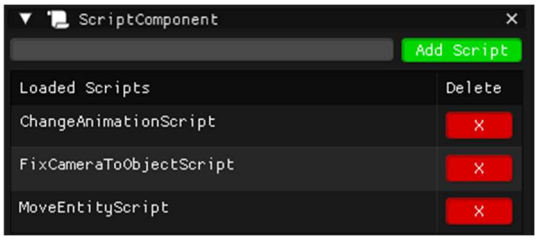
37. ábra – Vízkomponens panel



38. ábra - Vízszimuláció

2.5.1.2.18 Script Panel

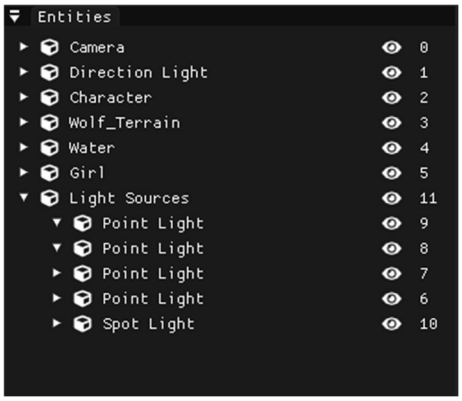
A felhasználó által fejlesztett, játék specifikus script kódok menedzselésére szolgál a 39. ábrán látható Script komponens Panel. Első sorban megtalálható egy input mező, ahova a felhasználó az általa írt script kód nevét írhatja be, majd az *Add Script* gombot lenyomva hozzáadhatja a scriptet az aktuális entitás scriptjei közé. Az eddigi hozzáadott scriptek táblázat formátumban jelennek meg, így a felhasználó könnyedén nyomon követheti, hogy milyen scriptek vannak az entitáshoz hozzácsatolva.



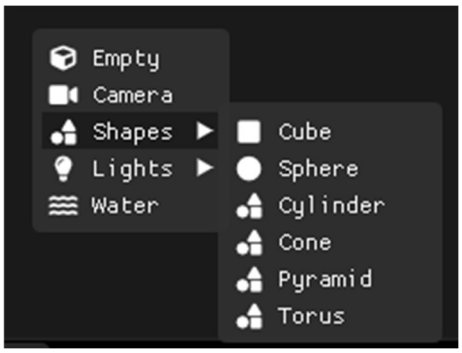
39. ábra - Script panel

2.5.1.3. Entity Panel

Az Entity panelen van lehetősége a felhasználónak az adott játékjelenethez új entitásokat hozzáadni, az entitások közötti kapcsolatokat menedzselni, illetve entitásokat törölni vagy éppen teljesen lemásolni. A panelen megjelenő entitások nevét a komponens panelen bemutatott Tag komponens segítségével változtathatjuk meg. Az entitások lenyitható/összecsukható vízszintes fa struktúrában jelennek meg a panelen, az entitás nevére bal gombbal hosszan kattintva a hozzá tartozó elemet ráhúzhatjuk egy másik entitáshoz tartozó elemre, így kialakítva a gyerek-szülő kapcsolatot. Amennyiben a gyerekhez nem szeretnénk a szülőt hozzárendelni, akkor a panel bármely üres pontjára húzva törlődik a kapcsolat. Az entitás nevére bal gombbal kattintva kiválaszthatjuk azt aktív entitásnak, vagyis a komponens panelen az entitáshoz tartozó komponensek fognak megjelenni. Amennyiben az entitás törlését/lemásolását szeretné a felhasználó végrehajtani, akkor az entitás nevére jobb klikkel kattintva felugrik a 42. ábrán látható ablak, amelyen belül a két opció egyikére kattintva elvégezhető a kívánt művelet. A panel jobb oldalán megtalálható az entitáshoz tartozó számazonosító, amely információ a felhasználónak scriptírásnál sokat segíthet. A számazonosító mellett egy szem ikon jelzi, hogy az adott kameraállás szerint az entitás megjelenik a színtérben, vagy nem. Amennyiben a felhasználó új entitásokat szeretne a színtérhez hozzáadni, akkor a panel bármely üres pontjára jobb gombbal kattintva felugrik a 41. ábrán is látható, az entitás hozzáadását kezelő ablak. Az ablakon belül teljesen üres, vagy akár specifikus feladatot ellátó, előre összeállított komponensekkel rendelkező entitásokat is választhatunk.



40. ábra - Entity Panel



41. ábra - Új entity hozzáadása



42. ábra - Entity törlés/másolás

2.5.1.4. A FileSystem Panel

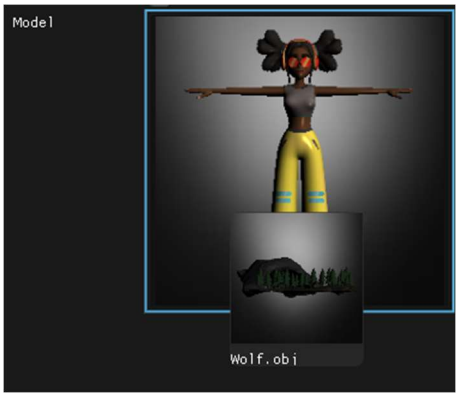


43. ábra - FileSystem Panel

A 43. ábrán látható FileSystem Panel egy beépített fájlkezelőként funkcionáló ablak, melynek segítségével a felhasználó a számítógépén lévő mappák/fájlok között böngészhet, illetve megfelelő formátumú fájlokat direkten betölthet a játékmotorba. A Panel két gyerekpanelből áll. Az első gyerekpanel a baloldalt található fehér mappa ikonokat tartalmazó egység, amely a beállított projekt kezdőmappájában megtalálható mappákat listázza ki az Entity Panelnél bemutatott lenyitható/összecsukható vízszintes faszerkezetű elemekkel. Az adott mappa nevére kattintva beállíthatjuk a megjeleníteni kívánt elérési útvonalat. A másik gyerekpanel a kiválasztott útvonalhoz tartozó mappákat/fájlokat listázza ki vizuális módon. A megjelenő mappaikonokra kattintva annak tartalmát nézhetjük meg, illetve az fájlikonra kattintva és jobb klikket letartva húzó mozgással a kiválasztott entitás komponens paneljén megjelenő megfelelő elemére is rá húzhatjuk, ami azonnal betölti és elmenti a kívánt textúrát/modelt/animációt a komponensben, ez látható a 45. ábrán. A gyerekpanel tetején található „<” és „>” gombok segítségével hátra/előre lépkedhetünk a mappastruktúrában, míg a pirosan/zölden megjelenő *Enable Preview* opció bekapcsolásával a textúrák, modellek, animációk előnézeti megjelenését kapcsolhatjuk be, ezt szemlélteti a 44. ábra.

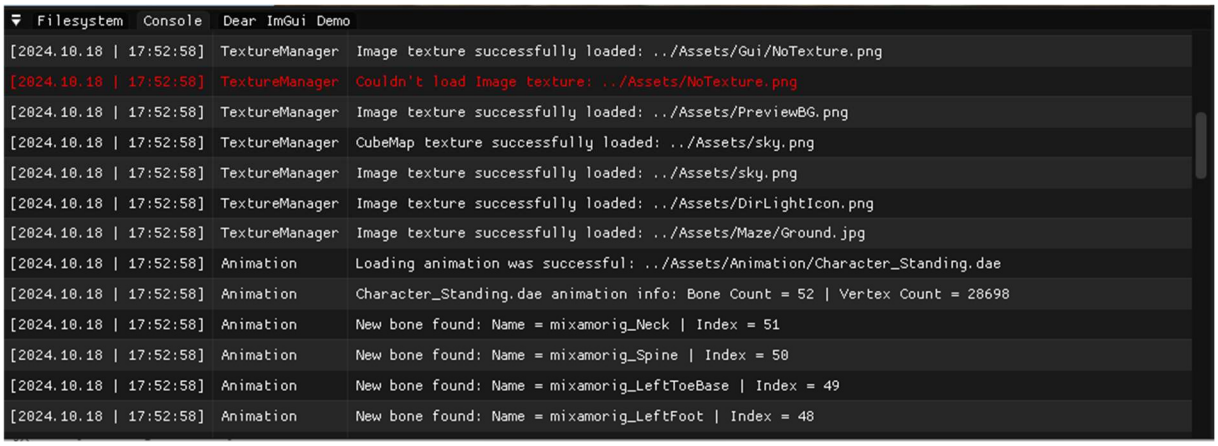


44. ábra - Előnézeti textúrák megjelenítése



45. ábra - Ikon ráhúzása komponensre

2.5.1.5. Console Panel



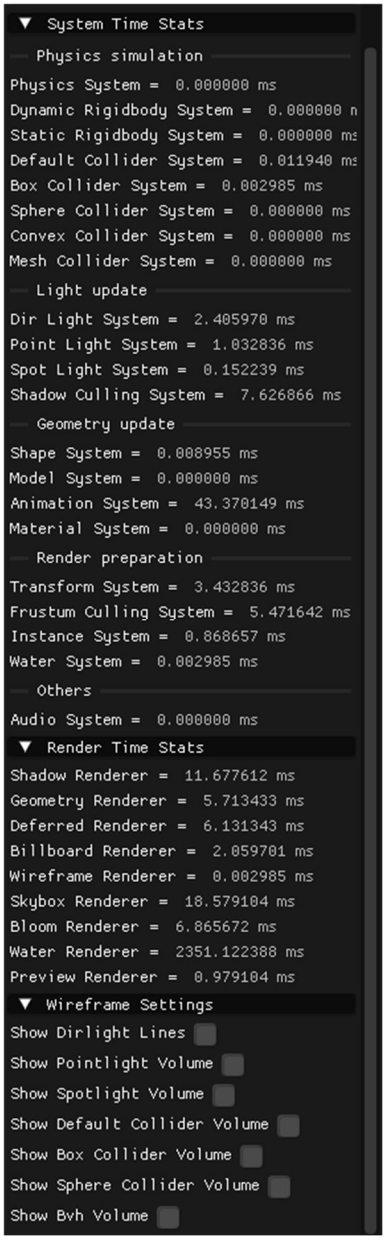
46. ábra - Consol Panel

A 46. ábrán látható Consol Panel listázza a játékmotor belső szerkezetében történő betöltéseket és különösebb történéseket, tehát az ablak debugolásra nagyon hasznos lehet. A játékmotor alapvetően a textúrák, modellek, animációk betöltésének sikerességét, hasznos adatait vagy éppen sikertelenségét jelzi a felhasználó számára. Szélsőséges esetben akár a Systemekben felhasznált ütközésvizsgáló algoritmusok (GJK) helytelen viselkedését is jelezhetni. A felhasználónak is lehetősége van a Consol panelre írni, amennyiben valamilyen fontos belső scriptműködés/debuggolásra lenne szükség, akkor a Logger osztály megfelelő metódusain keresztül ezt megteheti scriptekből hívva is.

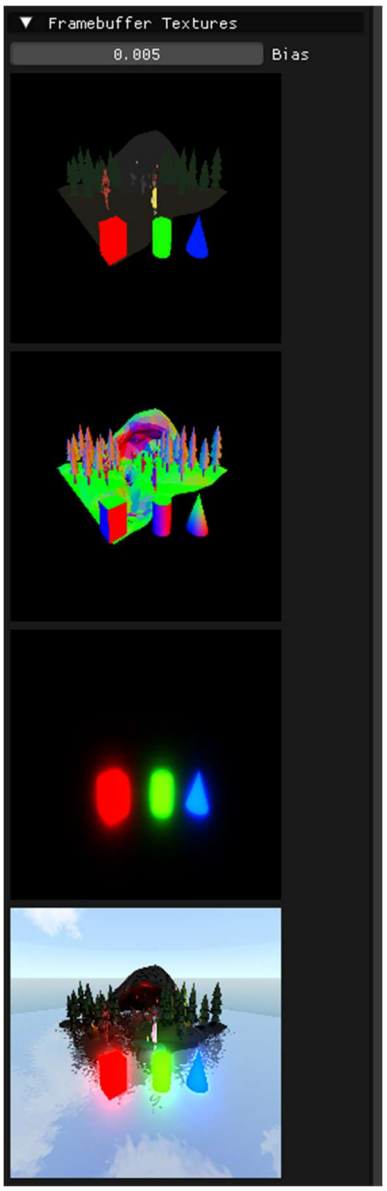
2.5.1.6. Settings Panel

A Settings Panel alapvetően a felhasználónak belső információkat nyújt akár a renderelés fázisairól, akár a komponenseket frissítő rendszerek gyorsaságáról. A panelen lehetősége van a felhasználónak további vizualizációs renderelési folyamatok be és kikapcsolására is. A panelen lenyitható *FrameBuffer Textures* ablak alatt találhatóak meg a deferred rendereléshez tartozó előzetes textúrák, bloom effektet kiváltó textúra, végül a ténylegesen megjelenő színtér textúra is. A „Bloom Textures” fül alatt a bloom effektushoz szimulálásához szükséges méretében skálázódó textúrák találhatóak meg. A 47. ábrán látható *System Time Stats* fülön az össszes belső rendszer futásához szükséges időmennyiség jelenik meg csoportosítva, a „Render Time Stats” fülön pedig a különböző renderelési feladatokat végző Rendererekhez tartozó időmennyiség jelenik meg. Ez egyrészt hasznos adat lehet a felhasználónak is, illetve a fejlesztőnek is, hiszen nyomon követhető az adott játék belső mechanizmusához szükséges számítási időmennyiségek, amik elősegíthetik a hibák, optimalizálandó helyek megtalálását. Az utolsó, *WireFrame Settings* fül alatt az objektumokhoz tartozó körbefoglaló alakzatok

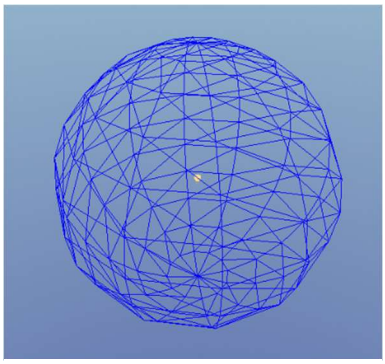
vizuális megjelenítését állíthatjuk meg. Ez lehet egyrészt a fényforrás típushoz tartozó, annak hatását reprezentáló alakzat váza, vagy a fizikai szimulációban használt default, box, sphere ütközési alakzat váza.



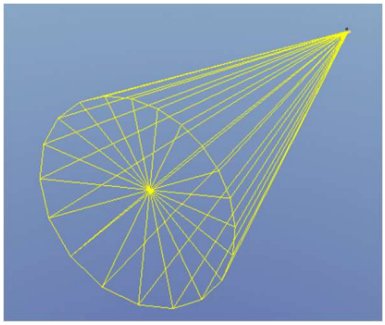
47. ábra - Időmennyiségek/beállítások



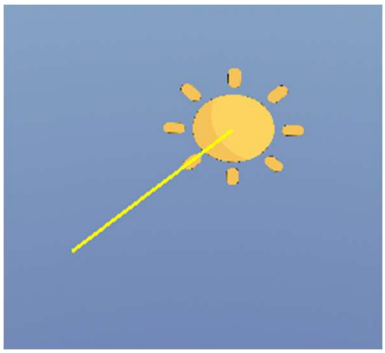
48. ábra - Render lépések vizualizációja



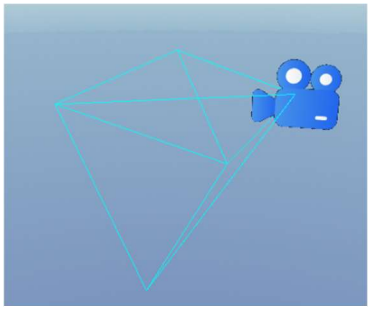
49. ábra - Pontfényforrást határoló alakzat



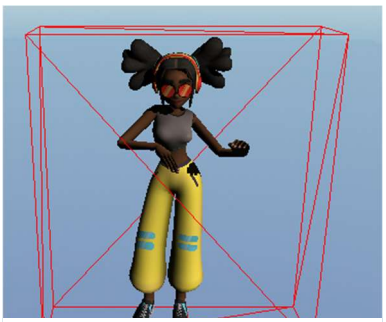
50. ábra - Spotfényforrást határoló alakzat



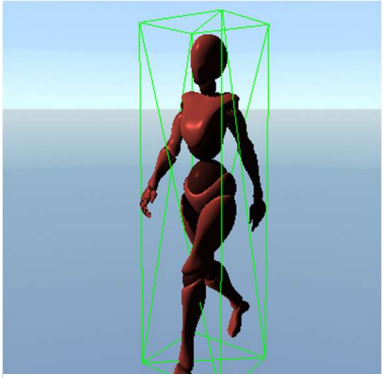
51. ábra - Irányfényforrás iránya



52. ábra - Kamera látógúla



53. ábra - Default Collider



54. ábra - Box Collider

3. Fejlesztői dokumentáció

3.1. Felhasznált külső könyvtárak

3.1.1. GLEW (OpenGL Extension Wrangler Library)

Ez a könyvtár az OpenGL kiterjesztéseinek, funkcióinak betöltésére szolgál. A könyvtárnak köszönhetően nem kell manuálisan lekérdezni és kezelni az egyes OpenGL kiterjesztéseket, mert a GLEW ezt helyettünk elvégzi. Így biztosíthatjuk, hogy grafikai alkalmazásunk kompatibilis legyen a különböző hardverekkel és meghajtókkal, valamint hozzáférjünk a legújabb OpenGL funkciókhoz.

3.1.2. GLFW (Graphics Library Framework)

Ez a könyvtár biztosítja a grafikai alkalmazás ablakkezelését és az OpenGL integrációját, lehetővé téve a színtér megjelenítését az ablakban. Ezen kívül alapvető eseménykezelési funkciókat is nyújt, például az egér- és billentyűzetesemények regisztrálását, így egyszerűen kezelhetjük a felhasználói interakciókat.

3.1.3. GLM (OpenGL Mathematics)

Ez a könyvtár a grafikai programozáshoz szükséges matematikai eszköztárt kínálja. Számos adattípust biztosít, amelyek segítségével könnyedén elvégezhetőek a vektor- és mátrixműveletek, amelyek elengedhetetlenek a 3D grafikában. A transzformációk, mint az eltolás, forgatás és skálázás, valamint a kameramozgások és a perspektív projekciók szintén a könyvtárba beépített funkciók segítségével valósíthatók meg. A könyvtár adatszerkezetei az OpenGL és GLSL által használt típusok memóriamodelljét követik, így ezek egyszerűen feltölthetők a GPU memóriájába, ahol a shader programok közvetlenül használhatják őket.

3.1.4. IMGUI (Immediate Mode Gui):

Ez a könyvtár felelős a felhasználói interfész gyors és egyszerű létrehozásáért, valós idejű frissítéssel. A megjelenő interfész a grafikai alkalmazásunk által használt render API felhasználásával jelenik meg. Az ImGui docking változatát használtam, amely lehetőséget ad a különböző ablakok egymásba „dokkolására” vagy akár azok elválasztására és különálló módon történő mozgatására.

3.1.5. STB_IMAGE

Ez egy egyszerű és hatékony C++ képbetöltő könyvtár, amely támogat számos formátumot, például PNG, JPG, BMP. Ennek segítségével képeket töltök be byte formátumba, amelyeket OpenGL textúrákká alakítok majd át.

3.1.6. ASSIMP (Open Asset Import Library):

A könyvtár lehetővé teszi a 3D modellek betöltését és feldolgozását, számos népszerű fájlformátumot támogat, mint például OBJ, FBX, és Collada. A betöltött modellekhez kapcsolódó adatokat – például geometriát, textúrákat és animációkat – különböző adatszerkezetekbe dolgozza fel, így könnyedén hozzáférhetek a háromdimenziós objektumok összes szükséges információjához.

3.1.7. IRRKLANG

Ez a könyvtár felelős a hangok, például zenei anyagok és hangeffektek betöltéséért és lejátszásáért. A könyvtár támogat rengeteg széles körben használt hangformátumot, köztük a legnépszerűbbeket is .wav, .mp3 formátumokat. Az IrrKlang különlegessége, hogy nemcsak 2D hangokat, hanem 3D térbeli hangokat is képes lejátszani, így lehetőséget kínál a hanghatások pontos térbeli pozícionálására.

3.1.8. PHYSX

A fizikai szimulációért felelős könyvtár, amelyet az NVIDIA fejlesztett ki. A könyvtár lehetővé teszi a valósághű fizikai hatások szimulálását, például az ütközések, gravitáció és erők hatását a 3D környezetben. Támogatja továbbá a merev testek (rigid body) dinamikáját, folyadékok, részecskerendszerek és sok más számos hasznos fizikai jelenséget.

3.1.9. NLOHMANN JSON

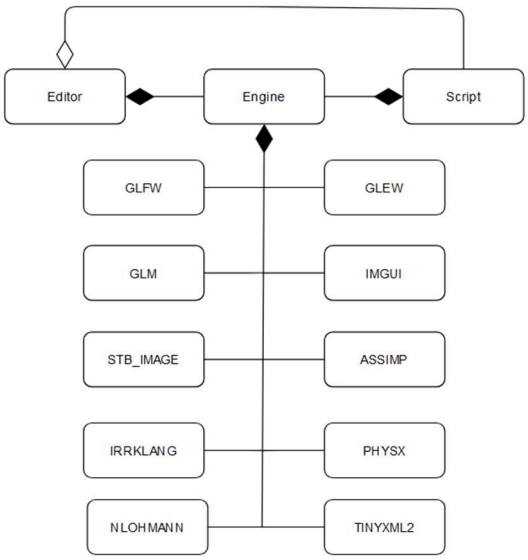
Ez a könyvtár felelős a json formátumú fájlok betöltésére, feldolgozására, illetve a színtér elemeinek elmentésére.

3.1.10. TINYXML2

A könyvtár segítségével töltök be xml fájlokat, és módosítom annak tartalmát. A játékmotor kapcsán a felhasználói Visual Studio 2022 script projekt include és lib elérési útvonalainak automatikus módosítására használtam fel.

3.2. Játékmotor felépítése

A játékmotor két fő komponensből áll. A legfontosabb komponens az Engine projekt, ami a játékmotor belső szerkezetét foglalja magába, beleértve az entitásokat, komponenseket, systemeket, különféle renderelést vezénylő rendszereket, manager osztályokat, illetve minden alapvető osztály definíciót amire a főbb rendszerek épülnek, tehát OpenGL absztrakciókat, model és animáció feldolgozást, egyszerű geometriák generálását, belső ütközésvizsgáló rendszereket, logger osztályt stb... Az engine projekt típusát tekintve egy DLL, erre azért van szükség mert a projektet két másik projekt is referenciaként felhasználja, és a statikus, globális változók így lesznek megosztottak a projektek között. Illetve az Engine tölti be az összes előbbiekben bemutatott külső könyvtárat is. A másik fő komponens az Editor project, ami a játékmotor futtatható állománya lesz, azaz projekt típusát tekintve EXE. Az editor valósítja meg az ablak létrehozást, grafikus interfész létrehozását, a játéktér frissítését és a színtér renderelését. A harmadik komponens, ami nem kötelező eleme a játékmotornak az a felhasználó által írt scriptek, amelyeket egy Visual Studio 2022-ben létrehozott DLL projekt foglal össze, és ezt a DLL állományt tölthetjük be a játékmotorba. A script projekt működése teljes mértékben az Engine projektre épül, azaz referenciaként használja fel azt.



55. ábra - A játékmotor felépítése

A DLL projektekben megtalálható osztálydefiníciók megfelelő importálása és exportálása érdekében minden osztálydefiníció elé az Engine projekt esetén az ENGINE_API, míg a script projektek esetén a SCRIPT_API makró került beillesztésre. Az EXPORT_ENGINE_DLL és az EXPORT_SCRIPT_DLL makrót a projektek maguk definiálják a beállításokban, így biztosítva, hogy a projekten belül exportálva, míg a projekten kívül importálva legyenek az osztály definíciók.

```
#pragma once
#pragma warning(disable: 4251)

#ifdef EXPORT_ENGINE_DLL
#define ENGINE_API __declspec(dllexport)
#else
#define ENGINE_API __declspec(dllimport)
#endif
```

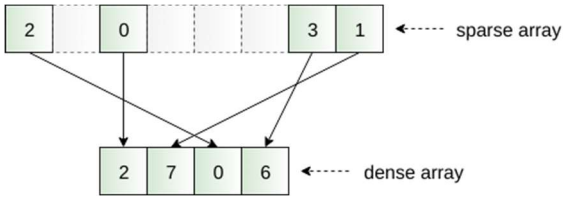
56. ábra - Dll import és export beállításai

3.3. Adatorientált Entity-Component-System elmélete és megvalósítása

3.3.1. Objektumorientáltság problémái és adatorientáltság elmélete

Az objektumorientált programozás összekapcsolja az adatokat és a viselkedést egyetlen egységbe, az objektumba, ami sok szempontból hasznos, de a CPU működése szempontjából problémás lehet. Az objektumok különféle típusú komponenseket tartalmaznak, amelyek a memóriában szétszórtan helyezkednek el, és gyakran pointerekkel érhetők el. Ez a CPU számára előre nem látható memóriaelérést okoz, így gyakoriak a cache miss-ek, amelyek lassítják a program futását. Ezenkívül az objektumorientált rendszerek gyakran alkalmaznak virtuális táblákat és típuskonverziókat, amelyek minden egyes lekérdezésnél extra időt igényelnek. Az adatorientált megközelítés, különösen az Entity-Component-System (ECS) modell, erre kínál megoldást azáltal, hogy az adatokat és a viselkedést elkülöníti. Az ECS modellben három fő elem található: az entitás, a komponens és a rendszer. Az entitás csupán egy egyedi azonosító, amely megkülönbözteti a rendszerben lévő objektumokat, de nem tartalmaz adatot vagy viselkedést. A komponens egy adott tulajdonságot ír le és tárolja annak adatait, például egy specifikus jellemzőt, amellyel az entitást bővítjük. Az entitások így az őket jellemző komponensek hozzáadásával kapják meg a szükséges tulajdonságokat. A komponensek tárolási módjánál két elterjedt megközelítés az *Array of Structures (AOS)* és a *Structure of Arrays (SOA)*, amelyek eltérő memóriaszervezési elveken alapulnak. Az AOS esetén a komponensek adatai egy struktúrában vannak összegyűjtve, és ezeket tároljuk vektorokban. Ezzel szemben az SOA esetén a komponens adatai külön-külön vektorokban tárolódnak a struktúrán belül. Szakdolgozatomban én az AOS verziót valósítottam meg. A rendszer felelős az egyes komponensekhez kapcsolódó viselkedésért. Az adott rendszer végigiterál az azonos típusú komponenseken, és végrehajtja a szükséges frissítéseket vagy számításokat. Mivel az ECS modellben az azonos típusú komponenseket vektorokban, egymás mellett tárolják, a CPU könnyebben fér hozzá a sorosan elhelyezett adatokhoz, minimalizálva a cache miss-ek számát és növelve a feldolgozás sebességét. Az ECS modell párhuzamos feldolgozást is támogat, mivel a rendszerek különálló komponenseken dolgoznak, így egyszerre több rendszer is futtatható anélkül, hogy versenyhelyzet alakulna ki, és mivel a komponensek vektorokban tárolódnak, így azok párhuzamos (SIMD) szerű frissítése is megvalósítható. Összességében az ECS modell hatékonyabb memóriahasználatot és nagyobb teljesítményt biztosít, különösen nagy adatmennyiségek gyors kezelése és frissítése esetén.

3.3.2. Component Pool megvalósítása



57. ábra - SparseSet adatstruktúra

Egy adott típusú komponens összes előfordulásának a memóriában egybefüggő tárolásáért, illetve kezeléséért felelős a Pool generikus osztály. Az adatorientált elveket kielégítő adattároláshoz az úgynevezett **SparseSet** adatstruktúrát alkalmaztam. Az adatstruktúra lényege, egy olyan indexelés bevezetése, amely segítségével egy adott tulajdonsággal rendelkező adatokat egybefüggően, rugalmasan, de mégis valós **O(1)** idejű indexeléssel lehessen elérni. Ahogyan az 57. ábrán is látható, az indexelési mechanizmushoz két tömbre lesz szükségem, egy ritka (sparse) és egy kompakt (dense) tömbre. Az első ritka tömb index pozíciói az egyedi azonosítóval rendelkező objektumokat szimbolizálják, és az adott pozícióban tárolt érték a kompakt tömb megfelelő index pozíciójára mutat. A második tömb hasonló módon számazonosítókat fog tárolni, melyek az első tömbnek azon index pozícióit azonosítják, mellyel asszociált objektumok rendelkeznek az előre meghatározott tulajdonsággal. Ezek az index pozíciók valójában maguk az objektum azonosítók. Az indexelési mechanizmushoz szükséges tömbök mellett természetesen az adott tulajdonsághoz tartozó speciális adatokat is tárolok, ezért szükségem van egy újabb harmadik kompakt tömb felvételére, ami hosszát tekintve mindig megegyezik a második tömb (dense array) hosszával, illetve annak minden módosulását megfelelően kezelem és követem. Amennyiben az adott objektumhoz tartozó speciális adatra van szükségem, akkor az első tömbben tárolt indexeket felhasználva a harmadik tényleges adatokat tároló tömbbe indexelhetek, ezt mutatja az 58. ábra.

```
int object_id = 2;
std::vector<int> sparse_array = { 2, -1, 0, -1, -1, -1, 3, 1 };
std::vector<int> dense_array = { 2, 7, 0, 6 };
std::vector<Data> dense_data_array = {{0, 0}, {1, 1}, {2, 2}, {3, 3}};

//Adott tulajdonság lekérdezése
bool hasData = sparse_array[object_id] != -1;

//Objektumhoz tartozó adat elérése
Data& data = dense_data_array[sparse_array[object_id]];

//Kompakt iteráció
for (int i = 0; i < dense_data_array.size(); ++i)
{
    Data& data = dense_data_array[i];
    int object_id = dense_array[i];
}
```

58. ábra – SparseSet felhasználása

A struktúra egyedüli hátulütője, hogy a ritka tömb (sparse array) hossza minden esetben a legnagyobb azonosítóval rendelkező entitás értékével kell megegyeznie. Ez azért problémás, mert ha rengeteg objektumazonosító áll rendelkezésre (10000 db) és csak egy töredéke rendelkezik az adott tulajdonsággal (5 db), akkor a ritka tömbben rengeteg entitásazonosítóhoz (9995) „feleslegesen” foglaltunk le indexpozíciókat tartalmazó adatnak

helyet a memóriában. Potenciális megoldás lenne erre a problémára, ha a ritka tömb helyett egy asszociációs lista segítségével tartanám nyilván azokat az objektum azonosítókat, amikhez tartozik az adott tulajdonság, viszont ebben az esetben elveszteném a konstans idejű O(1) indexelésen alapuló lekérdezéseket, hiszen minden asszociációs lista (std::unordered_map) hash számítás vagy bináris keresés költségét vonja maga után.

Mivel a ritka tömb esetemben az entitások azonosítóját (4 bájtos unsigned int) fogják tárolni, és mivel a mai modern hardverek nem küzdenek memória méret problémákkal, így én az először bemutatott gyors eléréssel bíró változatot valósítottam meg a Pool osztály implementációjában.

A Pool osztály az 59. ábrán is látható **PoolBase** interfészből származik, amely kötelezi az *AdjustPool*: ritka tömb megfelelő átméretezésért felelős, az *AddNewEntity*: új entitás azonosító hozzáadását megvalósító, és *RemoveEntity*: létező entitás azonosító törlését implementáló tisztán virtuális metódusokat.

```
class ENGINE_API PoolBase
{
public:
    virtual ~PoolBase() = default;
    virtual void AdjustPool(unsigned int size) = 0;
    virtual void AddNewEntity(Entity entity) = 0;
    virtual void RemoveEntity(Entity entity) = 0;
};
```

59. ábra - PoolBase interfész

A 60. ábrán látható **Pool** osztály *m_SparseEntities* vektora felel meg a fentiekben részletesen bemutatott ritka index tömbnek, az *m_DenseEntities* vektor felel meg a kompakt index tömbnek. Ezek mellett az osztály még két további kompakt vektort foglal magába. Az *m_DenseComponents* vektor T típusú komponensek, míg az *m_DenseFlags* vektor komponensekhez tartozó speciális bitflagek tárolásáért felelősek.

```
class Pool : public PoolBase
{
public:
    virtual void AdjustPool(unsigned int size) override;
    virtual void AddNewEntity(Entity entity) override;
    virtual void RemoveEntity(Entity entity) override;

    auto& GetDenseFlagsArray() { return m_DenseFlags; }
    auto& GetDenseComponentsArray() { return m_DenseComponents; }
    auto& GetDenseEntitiesArray() { return m_DenseEntities; }
    auto& GetSparseEntitiesArray() { return m_SparseEntities; }

    bool  IsFlagSet(Entity entity, const Flag flag);
    Flag& GetFlag(Entity entity);
    void  SetFlag(Entity entity, const Flag flag);
    void  ResFlag(Entity entity, const Flag flag);

    unsigned int GetIndex(Entity entity);
    bool HasComponent(Entity entity);
    T&   GetComponent(Entity entity);
    void AddComponent(Entity entity, const T& component = T{});
    void RemoveComponent(Entity entity);
private:
    std::vector<Flag>   m_DenseFlags;
    std::vector<T>      m_DenseComponents;
    std::vector<Entity> m_DenseEntities;
    std::vector<Index>  m_SparseEntities;
};
```

60. ábra - Pool osztálydefiníció

A speciális bitflagek majd a komponenseket frissítő rendszereknél (Systems) fognak fontos szerepet játszani, mivel ezek segítségével lesz eldönthető, hogy egy adott komponenshez tartozó OpenGL erőforrásokat újragenerálom-e (REGENERATE_FLAG), komponens valamely változójának megváltoztatása által kifejtett belső hatását újraszámolom-e (UPDATE_FLAG), illetve a frissített komponenseket felhasználó további rendszerek belső mechanizmusai aktiválódjanak-e (CHANGED_FLAG). Flagek definíciói láthatóak a 61. ábrán.

```
using Flag = unsigned char;
const unsigned char NULL_FLAG = 0b00000000;
const unsigned char UPDATE_FLAG = 0b00000001;
const unsigned char CHANGED_FLAG = 0b00000010;
const unsigned char REGENERATE_FLAG = 0b00000100;
const unsigned char ERROR_FLAG = 0b11111111;
```

61. ábra - Komponens flagek

A **HasComponent** metódussal vizsgálom meg, hogy egy adott entitás azonosítóhoz történt-e már komponens regisztrálás. Először az entitás azonosító értékéről győződök meg, hogy valid, majd a m_SparseEntities vektorba indexelve nézem meg, hogy annak tartalma a kompakt vektor egy indexét tartalmazza-e. Amennyiben az érték null (UINT_MAX), akkor nincsen az entitáshoz komponens rendelve. A metódus forráskódja a 62. ábrán látható.

```
template<typename T>
bool Pool<T>::HasComponent(Entity entity)
{
    return entity < m_SparseEntities.size() && m_SparseEntities[entity] != null;
}
```

62. ábra – Komponensek meglétének vizsgálata

Az **AddComponent** metódus tartalmazza az entitásokhoz tartozó komponensek beregisztrálását a Sparse Set-be. Kezdetben ellenőrzöm az entitás azonosítóját, illetve, hogy rendelkezik-e már beregisztrált komponenssel. Amennyiben nem, akkor az m_SparseEntities vektor entity azonosítójának indexpozíciójára elmentem az m_DenseEntities előzetes méretét, amely index segítségével fogom tudni elérni az entitáshoz tartozó komponenst. Komponens beszúrásakor a már kompaktan tárolt komponenseket nem szeretném felbolygatni, ezért az új komponenst mindig az m_DenseComponents végére, míg az entity azonosítóját az m_DenseEntities végére szúrom be, így megfelelően párhuzamban tartva a két kompakt vektort. Végül beállítom a flageket, amik lehetővé teszik, hogy új komponens regisztrálásakor a rendszerek azonnal reagáljanak, és minden szükséges további lépést elvégezzenek. A metódus a 63. ábrán tekinthető meg.

```
template<typename T>
void Pool<T>::AddComponent(Entity entity, const T& component)
{
    if (entity < m_SparseEntities.size() && m_SparseEntities[entity] == null)
    {
        m_SparseEntities[entity] = m_DenseEntities.size();
        m_DenseEntities.push_back(entity);
        m_DenseComponents.emplace_back(component);
        m_DenseFlags.push_back(UPDATE_FLAG | REGENERATE_FLAG);
    }
}
```

63. ábra - Komponensek regisztrálása

A 64. ábrán látható **GetComponent** metódus tartalmazza az entitásokhoz tartozó komponensek lekérését. A potenciálisan hibás működést meggátolva, előzetesen megvizsgálom az azonosító értéket, illetve, hogy rendelkezik-e az entitás beregisztrált komponenssel. Amennyiben nem, akkor egy statikusan létrehozott objektumot küldök vissza, hiszen egy T& objektumot vár el kimenetnek a metódus, emiatt nem áll rendelkezésemre nullptr visszaküldeni, viszont ez nem jelent különösebb problémát. Amennyiben az entitáshoz van beregisztrált komponens, akkor a SparseSet indexeket felhasználva elérem azt el.

```
template<typename T>
T& Pool<T>::GetComponent(Entity entity)
{
    if (HasComponent(entity))
        return m_DenseComponents[m_SparseEntities[entity]];

    static T defaultComponent;
    return defaultComponent;
}
```

64. ábra - Komponensek lekérdezése

A legkomplexebb funkcionalitást a **RemoveComponent** függvény tartalmazza, ami egy adott entitáshoz tartozó komponens törlését implementálja, ennek az algoritmusa látható a 65. ábrán. A törlés, a beszúráshoz hasonlóan a már meglévő kompakt módon tárolt komponenseket nem fogja túlságosan felbolygatni, így biztosítva a hatékony működést. A törlés algoritmusa tehát a törlendő komponenst megcseréli a kompakt tömbök utolsó elemével. Ennek hatására a törlendő adatok a vektorok végén fognak elhelyezkedni és a *pop_back* metódussal törölhetőek a vektorokból. A megcserélt adathoz tartozó entitás indexpozícióit megfelelően frissíteni kell a SparseSet index tömbökben, illetve a törölt entitáshoz tartozó indexpozíciót pedig null-ra állítom, így jelezvén, hogy nem tartozik az entitáshoz többé komponens.

```
template<typename T>
void Pool<T>::RemoveComponent(Entity entity)
{
    if (HasComponent(entity))
    {
        auto deleteIndex = m_SparseEntities[entity];
        auto swapIndex = m_DenseEntities.size() - 1;
        m_DenseFlags[swapIndex] |= UPDATE_FLAG;
        m_SparseEntities[m_DenseEntities[swapIndex]] = deleteIndex;
        std::swap(m_DenseEntities[deleteIndex], m_DenseEntities[swapIndex]);
        std::swap(m_DenseComponents[deleteIndex], m_DenseComponents[swapIndex]);
        std::swap(m_DenseFlags[deleteIndex], m_DenseFlags[swapIndex]);
        m_DenseEntities.pop_back();
        m_DenseComponents.pop_back();
        m_DenseFlags.pop_back();
        m_SparseEntities[entity] = null;
    }
}
```

65. ábra - Komponens törlése

Az entitásokhoz tartozó bitflagek beállításáért a *SetFlag*, visszaállításáért *ResFlag*, az adott bit aktiválódásának lekérdezését az *IsFlagSet* és a teljes bitsorozat visszaadásáért *GetFlag* metódusok felelősek.

3.3.3. Registry megvalósítása

A 66. ábrán látható **Registry** osztállyal implementálom az entitás azonosítók megfelelő létrehozását, tárolását, kezelését, adott entitások közötti szülő-gyerek viszony fenttartását. A registry felelős továbbá az entitásokhoz beregisztrált komponenseket tároló Poolok létrehozásáért, kezelésért és a Pool osztálynál bemutatott Add/Has/Remove funkcionalitás biztosításáért. Az entitások és komponensei json formátumba történő elmentésére és betöltésére is biztosítok metódusokat az osztályban.

```
class ENGINE_API Registry
{
public:
    Registry();
    Registry(nlohmann::json& data);
    ~Registry();

    Entity CreateEntity();
    void DestroyEntity(Entity entity);
    void SetActiveEntity(Entity entity) { m_ActiveEntity = entity; }

    void SetParent(Entity entity, Parent parent);
    bool IsDeepConnected(Entity entityA, Entity entityB);
    bool HasParent(Entity entity) { return m_Parents[entity] != null; }

    template<typename T>
    Pool<T>* GetComponentPool();
    template<typename T>
    bool HasComponent(Entity entity);
    template<typename T>
    T& GetComponent(Entity entity);
    template<typename T>
    void AddComponent(Entity entity, const T& component = T{});
    template<typename T>
    void RemoveComponent(Entity entity);
    template<typename T>
    void SetFlag(Entity entity, const Flag flag);
    template<typename T>
    unsigned int GetIndex(Entity entity);
    template<typename T>
    unsigned int GetSize();

    nlohmann::json Serialize();
    void DeSerialize(nlohmann::json& data);
    void DeSerializeEntity(nlohmann::json& data);
    nlohmann::json SerializeEntity(Entity entity);

    //Getterek
private:
    Entity m_NextEntity{ 0 };
    Entity m_ActiveEntity = null;
    std::vector<Entity> m_ActiveEntities;
    std::vector<Parent> m_Parents;
    std::vector<std::vector<Child>> m_Children;
    std::deque<Entity>  m_InactiveEntities;
    std::unordered_map<std::type_index, PoolBase*> m_Pools;
};
```

66. ábra - Registry osztálydefiníció

Az entitásokat azonosító *Entity* típus, a gyerek-szülő kapcsolatot reprezentáló Child/Parent típusok valójában mind egy pozitív egész számot fognak tartalmazni, tehát mind a három típus egy alias lesz az unsigned int típusra. Az azonosítók a Pool osztálynál bemutatott tömb indexelési feladatokat fognak ellátni, ezért döntöttem az unsigned int típus mellett, hiszen az int típus negatív értékei nem lennének kihasználhatóak indexelésre. A **null** entitás fogja jelenteni a nem létező entitás azonosítót, ami UINT_MAX makró értékével lesz egyenlő, ez tekinthető meg a 67. ábrán is.

```
using Index = unsigned int;
using Entity = unsigned int;
using Parent = unsigned int;
using Child = unsigned int;
const unsigned int null = UINT_MAX;
```

67. ábra - Entitás reprezentációja

Az osztály *m_NextEntity* változója tartalmazza a sorban következő entitás azonosítót, az *m_ActiveEntity* tárolja az aktívvá titulált entitás azonosítót, amely segítségével tudom definiálni például a felhasználó által módosítani kívánt entitást. Az *m_ActiveEntities* vektor tartalmazza az összes eddigi létrehozott (nem törölt) entitás azonosítót, míg az *m_InactiveEntities* tartalmazza azokat a volt entitás azonosítókat, amelyeket töröltem a registryből. Az *m_Parents* vektor az entitásokhoz tartozó szülő entitás azonosítót tartalmazza, az *m_Children* vektor pedig az entitásokhoz asszociált gyerek entitás azonosítókat

tartalmazzák vektoros formában. Az m_Parents és m_Children vektorok a SparseSet ritka tömbjének a tulajdonságaival rendelkeznek, tehát a hosszuk a maximális entity azonosítójával arányos, illetve adateléréshez az entitás azonosítókat használhatom a vektorokba indexeléshez, ami sokkal hatékonyabb, átláthatóbb megoldást biztosít a gyerek-szülő kapcsolat fenttartására, mint az objektum-orientált megvalósításban történő oda-vissza mutató pointerekkel.

Új entitásazonosító létrehozásáért a **CreateEntity** metódus a felelős, melynek forráskódja a 68. ábrán látható. Amennyiben találok a múltban törölt, inaktív entitás azonosítót az m_InactiveEntities listában, akkor annak egy elemét fogom felhasználni új entitás azonosítóként. Ez biztosítja, hogy a ritka típusú entitás tömbök hossza csak abban az esetben változzon, ha tényleg új entitás azonosítóra van szükség (plusz ezekhez az azonosítókhoz már tartozik memória lefoglalva). Amennyiben nincsen inaktív entitás azonosító, akkor az m_NextEntity értékét fogom felhasználni, ami az éppen sorban következő legnagyobb entitás azonosítót tartalmazza, így az m_Parents/m_Children és Component Poolokban szükség lesz a ritka tömbök átméretezésére, hiszen azok biztosan nem tartalmazták az azonosítónak megfeleltethető indexpozíciókat. Utolsó lépésként beregisztrálom az entitást aktív entitások közé, és visszaadom az azonosítót, amit felhasználva már a külső rétegek képesek lesznek komponenseket hozzáadni az entitáshoz.

```
Entity Registry::CreateEntity()
{
    Entity newEntity{ null };

    if (m_InactiveEntities.size() != 0)
    {
        newEntity = m_InactiveEntities.front();
        m_InactiveEntities.pop_front();
    }
    else
    {
        newEntity = m_NextEntity++;
        m_Parents.push_back(null);
        m_Children.push_back(std::vector<Child>{});
    }

    for (auto& [typeID, pool] : m_Pools)
        pool->AddNewEntity(newEntity);

    m_ActiveEntities.push_back(newEntity);
    return newEntity;
}
```

68. ábra - Entitás létrehozása és regisztrálása

A **DestroyEntity** metódus felelős egy adott entitás megsemmisítésért, az ahhoz tartozó viszonyokat, komponenseket is beleértve. A metódus implementációja tekinthető meg a 69. ábrán. A metódus először minden Poolon végigiterál, és meghívja a RemoveEntity metódusát, ami

```
void Registry::DestroyEntity(Entity entity)
{
    for (auto& [typeID, pool] : m_Pools)
        pool->RemoveEntity(entity);

    for (auto& child : m_Children[entity])
        SetParent(child, null);

    m_Parents[entity] = null;
    m_Children[entity].clear();

    auto it = std::find(m_ActiveEntities.begin(), m_ActiveEntities.end(), entity);
    m_ActiveEntities.erase(it);
    m_InactiveEntities.push_front(entity);
}
```

69. ábra - Entitás törlése

megfelelően törli az entitáshoz tartozó adott típusú komponenst. Majd a gyerek-szülő viszony megsemmisítését hajtom végre, amely során az entitáshoz tartozó gyerek entitásokon végigiterálva a SetParent(child, null) metódust felhasználva minden kapcsolatot felbontok az entitás és gyereke között, és beállítom azt megfelelően semlegesre. Végső lépésként törlöm az entity azonosítót az aktív entitások vektorából, és hozzáadom az inaktív entitások listájához.

A **SetParent** metódus felelős a szülő-gyerek viszony megfelelő kialakításáért, amelynek az implementációja látható a 70. ábrán. A függvény paraméterként a gyerek azonosítóját és annak szülő azonosítóját kapja meg. A metódus már meglévő gyerek-szülő kapcsolatokba történő viszonyok módosítását is megfelelően kezeli. Első lépésként megvizsgálom, hogy a gyerek azonosítóhoz tartozik-e már szülő azonosító az m_Parent vektorban. Amennyiben tartozik hozzá, akkor a szülő azonosítóhoz tartozó gyerekeket tartalmazó vektorban a paraméterként átadott gyerek azonosító biztosan szerepel, tehát abból azt törölni kell. Törlés után beállítom a gyerek szülő azonosítóját null-ra, és ezután már nem áll fent semmilyen viszony a gyerek és annak régi szülője között. Ezt követően beállítom az m_Parents vektorban a gyerekhez tartozó új szülő azonosítót, és amennyiben az új szülő azonosító nem null, akkor szükség van a szülőhöz tartozó gyerekeket tartalmazó vektor frissítésére, azaz a gyerek azonosító felvételére.

```
void Registry::SetParent(Entity entity, Parent parent)
{
    if (m_Parents[entity] != null)
    {
        auto& children = m_Children[m_Parents[entity]];
        auto it = std::find(children.begin(), children.end(), entity);
        children.erase(it);
        m_Parents[entity] = null;
    }

    m_Parents[entity] = parent;

    if (parent != null)
        m_Children[parent].push_back(entity);
}
```

70. ábra - Gyerek-Szülő viszony létrehozása

Az **IsDeepConnected** metódus egy iterációs megoldást biztosít a többszörösen beágyazott gyerek-szülő kapcsolatok felderítésére, ennek az algoritmusa olvasható a 71. ábrán. Számos esetben szükség lehet a megfelelő funkcionalitás biztosításához ennek a tulajdonságnak az ismeretére. A metódus egy szülő és egy gyerek azonosítót kap paraméterül és meghatározza, hogy több rétegen átívelően is van-e közöttük

```
bool Registry::IsDeepConnected(Entity entityA, Entity entityB)
{
    while (m_Parents[entityB] != null)
    {
        if (m_Parents[entityB] == entityA)
            return true;

        entityB = m_Parents[entityB];
    }

    return false;
}
```

71. ábra - Gyerek-Szülő viszony megvizsgálása

kapcsolat. Az iterációs algoritmus a gyerekhez tartozó szülőazonosítót vizsgálja meg, hiszen ez könnyedén, indexeléssel elérhető. Amennyiben ez nem null, akkor tartozik a gyerekhez szülő azonosító, és amennyiben ez az azonosító a paraméterként megadott szülő azonosító, akkor az algoritmus terminál és igazat ad vissza. Ha nem egyezik a két azonosító, akkor a következő iterációban a gyerek azonosító az előző iteráció szülő azonosítója lesz, a szülő azonosító pedig az ehhez tartozó szülő azonosító lesz. Az algoritmus addig iterál, amíg valamely entitáshoz tartozó szülő azonosító nem null, hiszen ellenkező esetben nincsen további gyerek-szülő kapcsolat.

A Registry osztállyal biztosítok a Pool osztályhoz hasonló komponensek kezelésére szolgáló metódusokat is.

Az **AddComponent** template metódus felelős a komponens Poolok létrehozásáért, illetve az adott Poolban az entitáshoz tartozó komponens beregisztrálásáért, ennek kódja tekinthető meg a 72. ábrán. Amennyiben az m_Pools asszociációs lista (std::unordered_map) még nem tartalmazza a T típushoz köthető Pool objektumot, akkor létrehozok egy példányt belőle, beállítom az AdjustPool metódusán keresztül a ritka indexlistájának hosszát az m_NextEntity változó értékének felhasználásával, ami a sorban következő legnagyobb még fel nem használt entitás azonosítóját tartalmazza. Ezek után a létrehozott Pool objektumon keresztül feltöltöm az entitáshoz tartozó komponenst az osztály AddComponent metódusa felhasználásával.

```
template<typename T>
void Registry::AddComponent(Entity entity, const T& component)
{
    std::type_index typeID = Unique::typeIndex<T>();

    if (m_Pools.find(typeID) == m_Pools.end())
    {
        PoolBase* pool = new Pool<T>{};
        pool->AdjustPool(m_NextEntity);
        m_Pools.insert(std::make_pair(typeID, pool));
    }

    auto pool = static_cast<Pool<T>*>(m_Pools[typeID]);
    pool->AddComponent(entity, component);
}
```

72. ábra - Komponens regisztálása

A **GetComponent** metóduson keresztül lehet elérni az adott T típusú komponensét egy entitásnak. A metódus algoritmusa látható a 73.

```
template<typename T>
T& Registry::GetComponent(Entity entity)
{
    std::type_index typeID = Unique::typeIndex<T>();
    if (m_Pools.find(typeID) != m_Pools.end())
        return static_cast<Pool<T>*>(m_Pools[typeID])->GetComponent(entity);

    static T component;
    return component;
}
```

73. ábra - Komponens lekérése

ábrán. A mechanizmus nagyon hasonló a Pool osztálynál már bemutatottal. Annyit érdemes megjegyezni, hogy ha a Registry osztályon keresztül szeretnénk lekérdezni a komponensét egy entitásnak, akkor előzetesen szükség van egy a komponens Pool létezését vizsgáló, és egy a PoolBase* típusról Pool<T>* típusra statikusan konvertáló műveletre, amik lassíthatják a komponensek O(1) elérését. Ezért minden komponenseket érintő, rendszeresen (framenként) végrehajtandó művelethez érdemes először magát az adott komponens Pool-t lekérdezni, majd annak validságának vizsgálata után a Pool osztályhoz tartozó metódusokat felhasználni, így kiküszöbölve a std::unordered_map-ban lévő keresést, és statikus konverziót. A Poolok lekérdezési kódja látható a 74. ábrán. (Minden System ezt a megvalósítást követi).

```
template<typename T>
Pool<T>* Registry::GetComponentPool()
{
    std::type_index typeID = Unique::typeIndex<T>();

    if (m_Pools.find(typeID) != m_Pools.end())
        return static_cast<Pool<T>*>(m_Pools[typeID]);

    return nullptr;
}
```

74. ábra - KomponensPool lekérdezése

A **RemoveComponent**, Pool **Flageket** kezelő metódusok teljesen a fentiekben bemutatott gondolatmenetre épülnek, ezért ezek bemutatását már nem részleteztem.

3.4. Játékmotor komponensei és rendszerei

3.4.1. Komponensek és rendszerek általános felépítése

A komponensek alapvetően csak a tulajdonsághoz tartozó adatokat tárolják. Az entitásokhoz csatolt komponensek pedig megát a tulajdonságot definiálják. Például az egyszerű alakzat komponens eltárolja az adott alakzat egy példányát és további rendereléshez szükséges paramétereket. A komponensekre sűrűn a shadereknek is szüksége van, ilyen esetben egy GLSL taggel ellátott struktúrát hozok létre, amely a hozzájuk tartozó komponensek rendereléshez szükséges adatait tárolják a lehető legkompaktabb módon. Minden rendszer számára definiálva van egy *OnStart*, *OnUpdate*, *OnEnd* statikus metódus. Az *OnUpdate* metódus lesz felelős a komponensek frissítéséért. Az *OnStart* és *OnEnd* metódus értelemszerűen a frissítés kezdete előtti, illetve frissítés utáni viselkedést definiálhat, például az *OnEnd* metódus felelhet a komponens flagek visszaállításáért. Minden rendszerben a hozzátartozó komponensek **json** formátumba történő elmentéséhez és betöltéséhez is definiálok metódusokat, ezeket a *Serialize* és *DeSerialize* statikus metódusokban implementáltam. A rendszerek frissítő iterációi előtt a szükséges OpenGL buffereket mappolom, az így megkapott buffer pointereken keresztül közvetlenül frissítem az adatokat a GPU bufferekben. A buffer pointeres megoldás lehetővé teszi a párhuzamos működés mellett is az adatok GPU-ra való feltöltését. A párhuzamos iteráció eléréséért ***std::execution::par*** paramétert állítom be az ***std::for_each*** iteráló algoritmusnak.
A rendszerek általános frissítési eljárása az 75. ábrán tekinthető meg.

```
auto componentPool = registry->GetComponentPool<Component>();

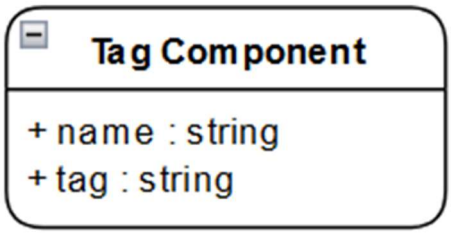
std::for_each(std::execution::seq, componentPool->GetDenseEntitiesArray().begin(), componentPool->GetDenseEntitiesArray().end(),
    [&](const Entity& entity) -> void {
        //Regenerate OpenGL resources if needed
        //Update components and recalculate parameters
        //Update data in GPU buffer
    }
);
```

75. ábra - Komponenseket frissítő rendszerek általános működési megvalósítása

3.4.2. Alapvető komponensek és rendszerek

3.4.2.1. Név azonosító

A 76. ábrán látható **Tag Component** tárolja az entitás nevét, illetve tag azonosítóját. A tag azonosítók a felhasználói scriptekben nyernek értelmet, ha adott entitásokat egy csoportként szeretnék azonosítani, és a csoportnak különösebb funkcionalitást definiálni.

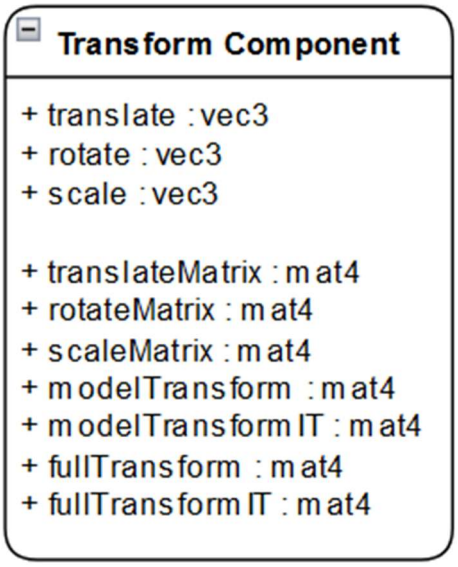


76. ábra - Tag komponens struktúra

A **Tag System** osztályban különösebb funkcionalitást nem implementáltam.

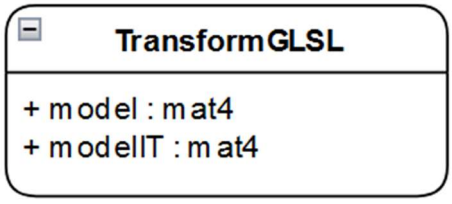
3.4.2.2. Transzformáció

A **Transform Component** tárolja az entitás háromdimenziós játéktérben lévő pozícióját, forgatását, illetve skálázási paramétereit és mátrixait, melynek osztálydiagramja látható a 77. ábrán. A komponens hatására az entitás megjelenik a színtér megfelelő pontján. A transzformációs mátrixokat külön-külön, szorzatukat és azok inverz transzponáltjait is eltárolom az IT postfix-szel ellátott változókban.



77. ábra - Transform komponens struktúra

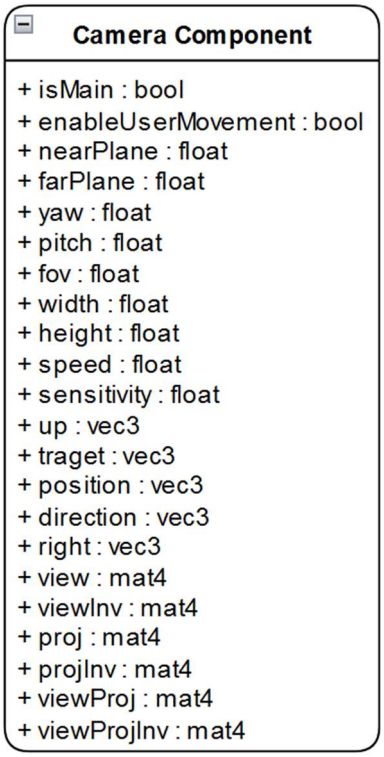
A **TransformGLSL** struktúra tartalmazza az alakzatok rendereléséhez szükséges transzformációs mátrixokat, ezt mutatja be a 78. ábra.



78. ábra - TransformGLSL struktúra

A **Transform System**ben implementáltam a komponens transzformációs adatainak frissítését. Az osztály *OnUpdate* metódusában párhuzamos végrehajtással végigiterálok a Transform komponenssel rendelkező entitásokon, és amennyiben a komponens adatai változtak (UPDATE_FLAG aktív), a transzformációs mátrixokat újra kiszámolom. A transzformáció vizuális helyessége érdekében balról jobbra először az eltolási, majd a forgatási és végül a skálázási transzformációkat szorzom össze, amelyek eredményét a *modelTransform* változóban tárolok el. Végül, a rendereléshez szükséges transzformációs mátrixokat feltöltöm a GPU buffer megfelelő részébe.

3.4.2.3. Kamera



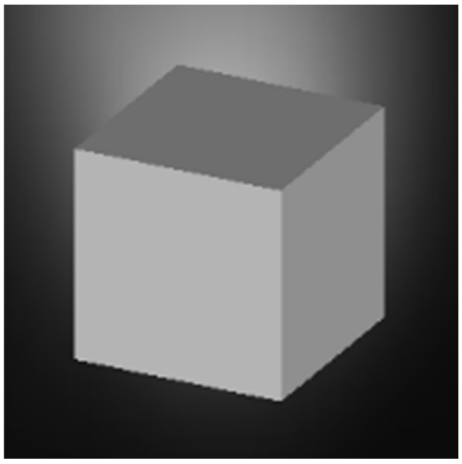
79. ábra - Kamera komponens struktúra

A **Camera Component** osztály tartalmazza a játéktérben lévő kamerák szimulációjához szükséges adatokat, amely osztálydiagramja látható a 79. ábrán. A kamera térbeli pozícióját a *position* változóban, a kamera által fókuszált térbeli pontot a target változóban tárolom, illetve a kamera felfelé mutató vektorát az *up* változó tartalmazza. Ezek az értékek lesznek szükségesek a kamera nézeti mátrixának létrehozásához. A *direction* és *right* vektorok a kamera billentyűzettel való mozgatásában fognak szerepet betölteni. A *yaw* és *pitch* értékek segítségével fogom a kamera egérrel történő forgatását szimulálni. A *speed* változó a kamera mozgási sebességét, míg a *sensitivity* változó a kamera forgatási érzékenységét definiálja. A kamera projektív mátrixának létrehozásához szükséges adatokat tárolom a következő változókban: A *nearPlane* és *farPlane* változók a kamera látógúlájának közeli és távoli vágósíkját definiálják. A látógúla nyílásszögét a *fov* változó tartalmazza, illetve a kamera képarányának kiszámításához szükséges XY paramétereket a *width* és *height* változóban tárolom. A kiszámolt nézeti és projektív mátrixokat, szorzatukat és ezeknek inverz változatát is eltárolom a komponensben, mivel más rendszereknek, illetve shadereknek szüksége lesz ezekre az adatokra. Az *isMain* opcióval definiálom a játéktér főkameráját, és az *enableUserMovement* opcióval pedig letiltható a billentyűzettel és egérrel történő mozgás.

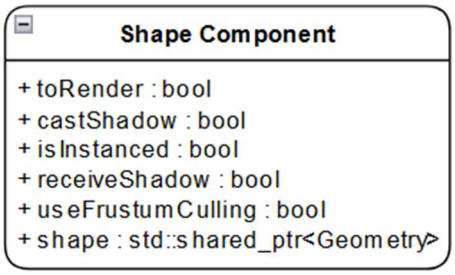
A **Camera System** osztály felelős a kamerák szimulálásáért. A kamerák adatainak frissítését az *OnUpdate* metódusban implementálom. Az InputManager osztály *IsKeyHeld* és *IsButtonHeld* metódusával meghatározható, hogy történt-e bármilyen felhasználói egér/billentyűzet lenyomás, amik hatására a kamerát mozgatni kell. Amennyiben történt ilyesfajta interakció, akkor a kamera pozícióját, irányát és forgatási vektorát frissítem. Ezután kiszámolom a nézeti és projektív transzformációs mátrixokat. Végezetül, a rendereléshez szükséges játéktér főkamerájához tartozó adatokat feltöltöm a GPU megfelelő bufferébe.

3.4.3. Geometriák

3.4.3.1. Egyszerű alakzatok (Shape)



80. ábra - Egyszerű alakzat



81. ábra - Shape komponens struktúra

A 81. ábrán látható **Shape Component** tárolja az egyszerű alakzatokat (például kocka, gömb) és azok paramétereit, és hatására ezek az alakzatok megjelennek a játéktérben. Az alakzatokat a *ResourceManager* osztály generálja le, majd azok referenciáját adja tovább a komponensnek, amelyeket a *shape* változóban tárolok. Ennek köszönhetően elkerülhető a felesleges redundáns adattárolás, mivel az alakzatok felhasználása a játékmotorban nem igényel több azonos példányt. A *castShadow* és *receiveShadow* változók az alakzatok kölcsönhatását szabályozzák a fényforrások árnyékgenerálásában, míg a *toRender* és *useFrustumCulling* paraméterek befolyásolják, hogy az objektum megjelenik-e a renderelés során. Végül, az isInstanced változó határozza meg az objektum renderelési módját, amelyről még a renderelés szakaszban bővebben lesz szó.

A **Shape System** alapvetően nem végez semmilyen komplexebb végrehajtást. Az *OnUpdate* metódusban a Shape komponenssel rendelkező entitásokon végigiterálok, majd amennyiben még nem rendelkeznek Default Collider komponenssel, akkor azt hozzájuk rendelem. A Default Collider segítségével fogják az optimalizációs rendszerek eldönteni, hogy az objektum megjelenjen-e a színtérben, és ez csak akkor érvényes, ha az entitás rendelkezik Shape komponenssel, ezért végzem el ezt a lépést itt. A rendszer továbbá feltölti a komponens *castShadow* és *receiveShadow* változóit a GPU-ra, hiszen ezen adatokra az árnyékgeneráló shadereknek szüksége lesz.

Az egyszerű alakzatokat a parametrikus képleteik alapján generálom le kódban. Az alakzatok közös funkcionalitását a *Geometry* ősosztályban implementáltam, az alakzatot leíró specifikusabb adatok legenerálása (pozíció, normál vektor, stb) pedig külön osztályokban (*Cube*, *Cylinder*, *Torus*, stb) tekinthetőek meg. Minden legenerált alakzathoz kiszámítom a

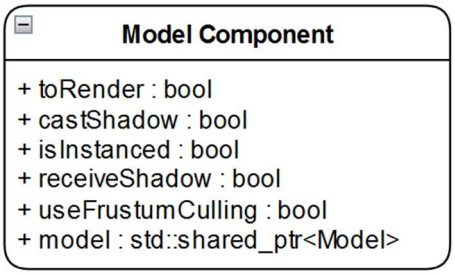
tértengelyek mentén a minimális és maximális pozíció értékeket, amelyeket majd ütközésvizsgálatnál, illetve ütközési alakzat generálásnál fogok felhasználni.

3.4.3.2. Komplex alakzatok (Model)

A 83. ábrán látható **Model Component** tárolja a fájlból betöltött komplex alakzatokat és azok paramétereit, és hatására ezek az alakzatok megjelennek a játéktérben. A komponens a felépítését tekintve nagyon hasonló a Shape komponensnél bemutatottra. A modellek betöltését és tárolását a *ModelManager* osztályban implementáltam. A betöltött model példányokat egyszer tárolom el, amelyek referenciáját kapja meg a komponens *model* változója. A struktúra további változói teljesen megegyeznek az egyszerű alakzatoknál már ismertetettel.



82. ábra - Komplex alakzat



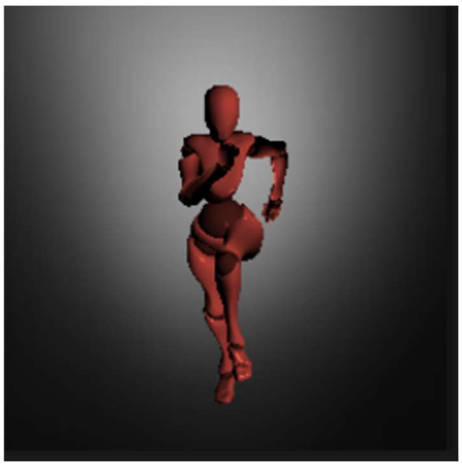
83. ábra - Model komponens struktúra

A **Model System** működési elve teljesen megegyezik a Shape Systemnél láthatóakkal. A rendszer csupán annyiban különbözik, hogy az *OnUpdate* metódusban a Model komponenssel rendelkező entitásokon iterálok végig.

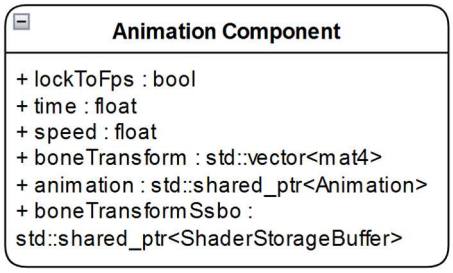
A *Model* osztályban implementáltam a komplex alakzatok fájlformátumból történő betöltését, és adatainak tárolását. A modelt leíró adatok betöltéséhez az Assimp könyvtár függvényeit alkalmaztam. Az osztály *Process* metódusában hierarchikusan végigiterálok a model meshein (részegységein), majd a *ProcessGeometry* metódusban feldolgozom az adott meshez tartozó adatokat (pozíció, normál vektor, textúrák). A feldolgozott meshek vertex adatait folytonosan egy vektorban tárolom el (batchelés), melynek köszönhetően az adott model egy renderhívással megjeleníthető lesz. A különböző meshekhez különböző textúrák és anyagtulajdonságok tartozhatnak, emiatt a vertexek struktúrái tartalmazni fognak egy ID paramétert, amiket az alakzat megjelenítő shaderekben fogok felhasználni, hogy a materiális adatok visszakövetkeztethetőek legyenek. Az osztályban a rendereléshez szükséges GPU erőforrásokat is létrehozom, illetve hasonlóan az egyszerű alakzatokhoz, kiszámítom a model maximális és minimális pozíció értékeit, amiket a későbbi rendszerek fognak felhasználni.

3.4.3.3. Animáció

A 85. ábrán bemutatott **Animation Component** tárolja a fájlból betöltött modellek animációját, hatására a színtérben megjelenített komplex alakzat életre kel, és az animáció által definiált mozgást kezdi el végrehajtani. Az animációk betöltését és tárolását a *ModelManager* osztályban implementáltam. A betöltött animáció példányokat egyszer tárolom el, amelyek referenciáját kapja meg a komponens *animation* változója. A komponens *boneTransform* változójában tárolom az animáció csontjainak adott időpillanathoz tartozó transzformációs mátrixait, amelyeket a *boneTransformSsbo* GPU bufferbe töltök fel. Az animáció sebességét a *speed* változóban, míg az aktuális animációs időpontot a *time* változóban tárolom. A csontokhoz tartozó transzformációs mátrixok frissítési rátáját szabályozom a *lockToFps* változóval.



84. ábra - Animáció



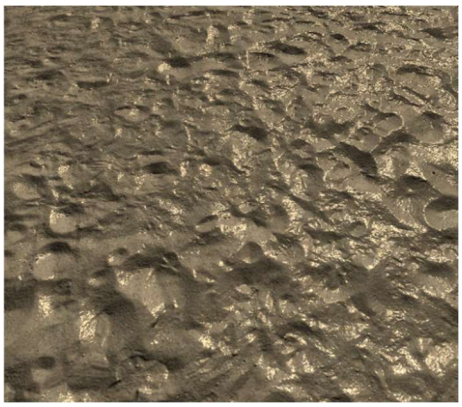
85. ábra - Animáció komponens struktúra

Az **Animation System**ben implementáltam az animációk frissítését. Az osztály *OnUpdate* metódusában az Animation komponenssel rendelkező entitásokon végigiterálok, és az animáció csontjaihoz tartozó transzformációs mátrixokat kiszámolom, majd feltöltöm a megfelelő GPU bufferbe. A mátrixok kiszámítása a *CalculateBoneTransforms* metódusban található, amelyben hierarchikusan a gerinctől a legkisebb csontokig egymásra épülve dolgozom fel a transzformációkat. Ez az eljárás időigényes, ezért minden komponenst párhuzamosan, több szálon dolgozok fel.

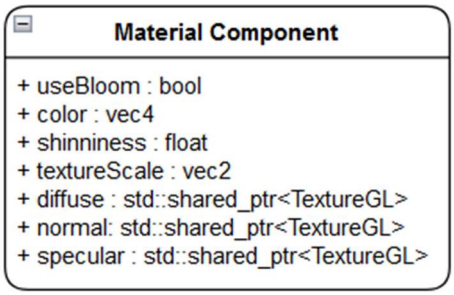
Az Animation osztályban található a modellekhez tartozó animációk fájlformátumból történő betöltése az Assimp könyvtár függvényeinek felhasználásával. Az osztály *Process* metódusában hierarchikusan végigiterálok az animáció csontjain, majd a *ProcessBone* metódust meghívva feldolgozom a csontokhoz tartozó adatokat, aholis elmentem minden egyes vertexhez, hogy mely csontok milyen súllyal hatnak rá. Továbbá elmentem minden csonthoz, hogy a megadott időpontban milyen eltolási, forgatási és skálázási értékekkel rendelkezik, amelyek között interpolálva elérhető a folyamatosnak tűnő mozgó animáció.

3.4.3.4. Anyagtulajdonságok

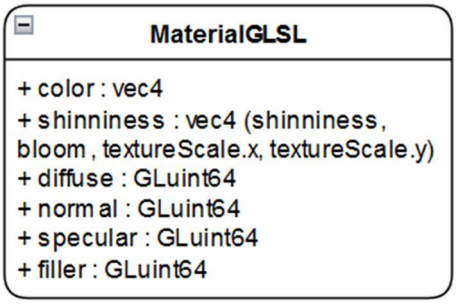
A 87. ábrán látható **Material Component** tárolja az egyszerű (és közvetetten a komplex) alakzatok anyaginformációit, hatására a színtérben az alakzatok színesen és feltextúrázottként jelennek meg. Az alakzat színét a *color* változó, a fényvisszaverődés erősségét a *shinniness* változó fogja tárolni. A *useBloom* opcióval az alakzat világítását szabályozom. A komponens szerves részét képzik a fájlból betölthető textúrák. A textúrákat a *TextureManager* osztállyal töltöm be és tárolom el. Minden betöltött textúrát egyszer tárolok el, majd annak egy referenciáját kapja meg a Material komponens textúra változói, így elkerülve a redundáns adatokat. A színértéket módosító textúrát a *diffuse* változóban, az alakzat felszíni normál vektorait definiáló textúrát a *normal* változóban, illetve a felszíni fényvisszaverődés értékét szabályzó textúrát a *specular* változóban tárolom el. A textúrák használata opcionális, azok nélkül is megfelelően megjelennek az alakzatok. A *textureScale* változóval biztosítom, hogy XY koordináták mentén az alakzatokhoz tartozó textúra koordináták skálázódjanak, melynek hatására a textúrák többször is megjelennek az adott felületen. Akkor játszik fontos szerepet ez az opció, amikor nagy felületen túl pixelesen jelenne meg a textúra.



86. ábra - Anyagtulajdonság



87. ábra - Material komponens struktúra



88. ábra - MaterialGLSL struktúra

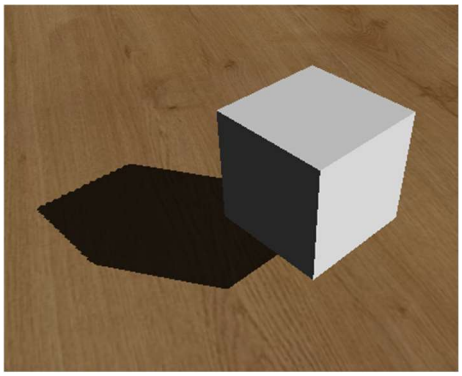
Az anyagtulajdonságokra a shadereknek lesz szüksége. A **MaterialGLSL** struktúrába tömörítem renderelésnél felhasznált komponens adatait, ami a 88. ábrán tekinthető meg. A struktúrát a shader bufferek által definiált memóriaeloszlási szabályokat betartva definiáltam.

A **Material System** alapvetően nem végez semmilyen komplexebb végrehajtást. Az osztály *OnUpdate* metódusában végigiterálok az összes Material komponenssel rendelkező entitáson, és adattag változás esetén frissítem a GPU bufferben tárolt adatokat.

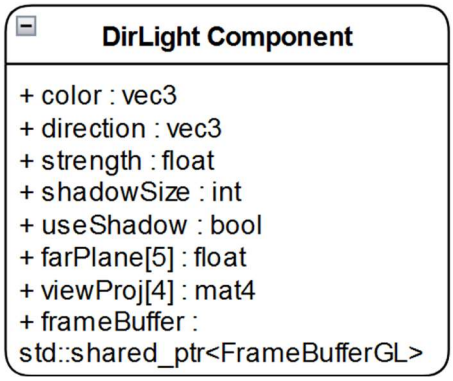
3.4.4. Fényforrások

3.4.4.1. Irány fényforrás

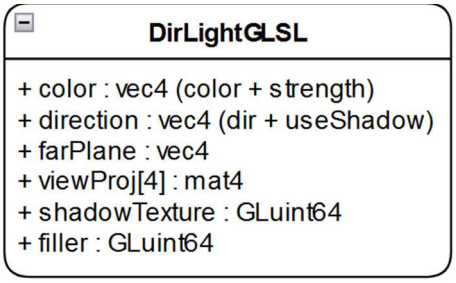
A **DirLight Component** tárolja az irányfényforrások szimulálásához szükséges adatokat, a hatására a játéktérben az alakzatok árnyaltan jelennek meg. A komponens osztálydiagramját mutatja be a 90. ábra. Az irányfényforrás irányát a *direction* változóban, a színét a *color* változóban, illetve a fényforrás szimulált erősségét a *strength* változóban tárolom. A játékmotor támogatja az irányfényforrások többrétegű árnyékszimulációját, a rétegekhez tartozó távoli vágósíkokat a *farPlane* tömbben, a szimuláláshoz szükséges nézeti és projektív transzformációs mátrixok a *viewProj* tömbben találhatóak meg. Az árnyékszimulációhoz szükséges mélységtextúrák méretét a *shadowSize* változó definiálja, illetve framebuffer és textúra erőforrásokat a *frameBuffer* változó tartalmazza. Az árnyékgenerálás opciója a *useShadow* változóban található meg.



89. ábra - Irányfényforrás hatása



90. ábra - Irányfényforrás komponens struktúra



91. ábra - DirLightGLSL struktúra

A **DirLightGLSL** struktúra tartalmazza az irányfényforrás komponensének azon adatait, amelyekre az árnyékszimulációs és a fényszimulációs shadereknek szüksége lesz. A struktúra leírása látható a 91. ábrán. A struktúrát a shader bufferek által definiált memóriaeloszlási szabályokat betartva definiáltam.

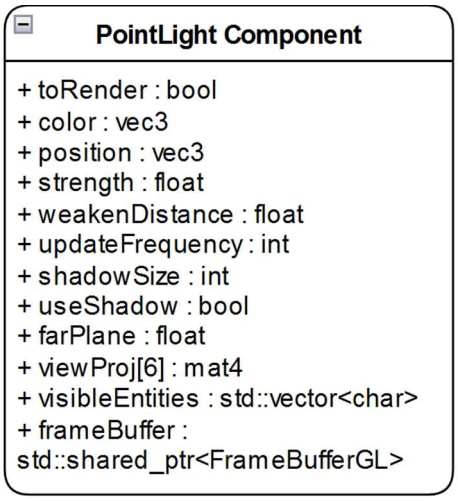
A **DirLight System** osztályban találhatóak az irányfényforrás komponensek adatainak frissítése. Az osztály *OnUpdate* metódusában végigiterálok a komponenseken, amennyiben szükséges az OpenGL-es erőforrásokat újragenerálom, illetve változás esetén kiszámolom újra az árnyékszimuláláshoz szükséges mátrixokat. Végül, a frissített komponensek adatait feltöltöm a megfelelő GPU bufferbe.

3.4.4.2. Pont fényforrás

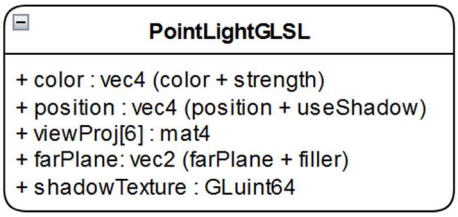
A 93. ábrán látható **PointLight Component** tárolja a pontfényforrások szimulálásához szükséges adatokat, a hatására a játéktérben a fényforráson belül az alakzatok árnyaltan jelennek meg. A komponens felépítése hasonló az irányfényforrásoknál bemutatottra. A fényforrás színét a *color* változóba, térbeli pozícióját a *position* változóban, fényforrás szimulált erősségét a *strength* változóban tárolom, illetve a *weakenDistance* változó tartalmazza azt a távolságot, amelytől kezdve a fényforrás hatása fokozatosan gyengülni kezd. A játékmotor támogatja a pontfényforrások árnyékszimulációját, a szükséges nézeti és projektív transzformációs mátrixok a *viewProj* tömbben találhatóak meg. A nézeti mátrixok távoli vágósíkját a *farPlane* változóban tárolom, ami egyrészt a fényforrás méretét is meghatározza. Az árnyékszimulációhoz szükséges mélységtextúrák méretét a *shadowSize* változó definiálja, illetve a textúra létrehozásához szükséges framebuffer és textúra erőforrásokat a *frameBuffer* változó tartalmazza. Az árnyékgenerálás opciója a *useShadow* változóban található meg, illetve a mélységtextúrák frissítési rátáját az *updateFrequency* változó definiálja. A *toRender* opció definiálja, hogy a fényforrás látható-e a kamera számára, illetve a *visibleEntities* vektor tartalmazza azokat az entitásokat, amelyek a pontfényforrás árnyékszimulálásában részt vesznek. Ezeket a változókat az optimalizációs rendszerek fogják felhasználni, segítségükkel kiküszöbölhetőek a felesleges renderhívások, fényszimulációk.



92. ábra - Pontfényforrás



93. ábra - Pontfényforrás komponens struktúra



94. ábra - PointLightGLSL struktúra

A **PointLightGLSL** struktúra tartalmazza azokat a komponens változókat, amelyekre az árnyékszimulációs és fényszimulációs shadereknek szüksége lesz, ezt látható a 94. ábrán. A struktúrát a shader bufferek által definiált memóriaeloszlási szabályokat betartva definiáltam.

A **PointLight System**ben implementáltam a komponensek frissítését, szimulációs adatok újraszámítását változás esetén. Az osztály *OnUpdate* metódusában végigiterálok a PointLight komponenssel rendelkező entitásokon, szükség esetén újragenerálom az OpenGL-es erőforrásokat, illetve paraméter változás esetén kiszámolom az árnyékszimulációhoz szükséges transzformációs mátrixokat. Végül, a frissített komponensek adatait feltöltöm a megfelelő GPU bufferbe.

3.4.4.3. Spot fényforrás

A 96. ábrán definiált **SpotLight Component** tárolja a spotfényforrások (zseblámpaszerű fény) szimulálásához szükséges adatokat, a hatására a játéktérben a fényforráson belül az alakzatok árnyaltan jelennek meg. A komponens felépítése hasonló az előző fényforrásoknál bemutatottra. A fényforrás színét a *color* változóban, térbeli pozícióját a *position* változóban, az irányát a *direction* változóban, illetve fényforrás szimulált erősségét a *strength* változóban tárolom. A fényforrás hatását leíró kúp alakzat nyílásszögét, és belső szögét, és ezek cosinusát az *angles* változóban tárolom, illetve a kúp magasságát a *length* változó definiálja. A játékmotor támogatja a spotfényforrások árnyékszimulációját, a szükséges nézeti és projektív transzformációs mátrixok a *viewProj* változójában találhatóak meg. A nézeti mátrixok távoli vágósíkját a *farPlane* változóban tárolom, az árnyékszimulációhoz szükséges mélységtextúrák méretét a *shadowSize* változó definiálja. A textúra létrehozásához szükséges framebuffer és textúra erőforrásokat a *frameBuffer* változó tartalmazza. Az árnyékgenerálás opciója a *useShadow* változóban található meg, illetve a mélységtextúrák frissítési rátáját az *updateFrequency* változóval definiálom. A *toRender* opció definiálja, hogy



95. ábra - Spotfényforrás

SpotLight Component

+ toRender : bool
+ color : vec3
+ position : vec3
+ direction : vec3
+ strength : float
+ length : float
+ updateFrequency : int
+ useShadow : bool
+ shadowSize : int
+ farPlane : float
+ viewProj: mat4
+ proxyTransform : mat4
+ visibleEntities : std::vector<char>
+ frameBuffer : std::shared_ptr<FrameBufferGL>

96. ábra - Spotfényforrás komponens struktúra

SpotLightGLSL

+ color : vec4 (color + strength)
+ position : vec4 (position + useShadow)
+ direction : vec4 (direction + farPlane)
+ angles : vec4
+ viewProj : mat4
+ shadowTexture : GLuint64
+ filler : vec2

97. ábra - SpotLightGLSL struktúra

a fényforrás látható-e a kamera számára, illetve a *visibleEntities* vektor tartalmazza azokat az entitásokat, amelyek a spotfényforrás árnyékszimulálásában részt vesznek. A változókat az optimalizációs rendszerek fogják felhasználni, segítségükkel kiküszöbölhető a felesleges renderhívások, fényszimulációk. A kúp alakzat megfelelő irányú és pozíciójú megjelenítéséhez szükséges transzformációs mátrix a *proxyTransform* változóban található meg.

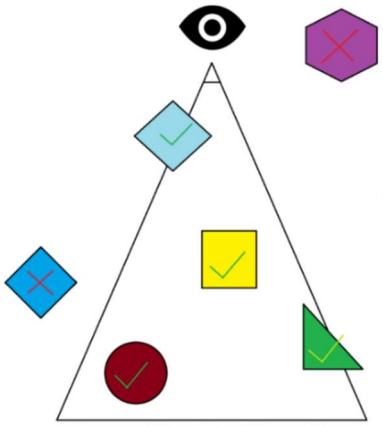
A **SpotLightGLSL** struktúra, aminek az osztálydiagramja a 97. ábrán látható, tartalmazza azokat a komponens változókat, amelyekre az árnyékszimulációs és fényszimulációs shadereknek szüksége lesz. A struktúrát a shader bufferek által definiált memóriaeloszlási szabályokat betartva definiáltam.

A **SpotLight System**ben implementáltam a komponensek frissítését, szimulációs adatok újraszámítását változás esetén. Az osztály *OnUpdate* metódusában végigiterálok a SpotLight komponenssel rendelkező entitásokon, szükség esetén újragenerálom az OpenGL-es erőforrásokat, illetve paraméter változás esetén kiszámolom az árnyékszimulációhoz szükséges transzformációs mátrixokat. Végül, a frissített komponensek adatait feltöltöm a megfelelő GPU bufferbe.

3.4.5. Optimalizációs eljárások

3.4.5.1. Frustum Culling

A **FrustumCulling System**ben implementáltam a ténylegesen megjelenő alakzatok és fényforrások kiszűrését. A Frustum Culling egy optimalizációs technika, amely során egy adott objektumhoz határozzuk meg azokat a többi objektumokat, amelyek térbeli kiterjedése metszésben állnak vele, ez látható a 98. ábrán. A rendszer célja, hogy meghatározza azokat az alakzatokat és fényforrásokat, amelyek az adott frame-ben láthatóak lesznek a kamera számára, így kiküszöbölve a költséges, felesleges renderhívásokat. A rendszer *OnUpdate* metódusa kezdetben kiszámítja a főkamera látógúlájának közeli és távoli vágósíkjaihoz tartozó térbeli koordinátákat, majd ebből egy Convex GJK ütközési alakzatot hoz létre. Ezt követően párhuzamos végrehajtás mellett a *GeometryCulling* metódusban a DefaultCollider komponenssel rendelkező alakzatokon, a

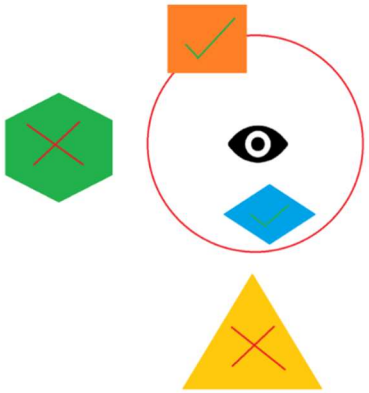


98. ábra - Kamera frustum culling

PointLightCulling metódusban a pontfényforrásokon, a *SpotLightCulling* metódusban a spotfényforrásokon iterálok végig, és egyenként meghatározom a GJK ütközési vizsgálóval, hogy a kamera látógúlájával az adott alakzat metszésben áll-e. Amennyiben metszés történt, a komponens *toRender* változóját igazra állítom, és az alakzat megjelenik, illetve a fényforrás szimulálódik a színtérben. Ellenkező esetben nincs szükség az alakzat/fényforrás megjelenítésére, így nem történik semmilyen renderhívás.

3.4.5.2. Árnyék Culling

A **ShadowCulling System** osztályban található a fényforrásokhoz tartozó, az árnyékszimulációban megjelenő alakzatok kiszűrése. A szűrési algoritmus elméleti lényege és felhasználási haszna teljesen megegyezik a Frustum Cullingnál bemutatottal. Mivel a pontfényforrások és spotfényforrások hatása térben jól definiált alakzattal leírhatóak, így a színtér objektumai és a fényforrások között elvégezhető a metszésvizsgálat, ezt reprezentálja a 99. ábra. A pontfényforrások esetén egy gömb GJK ütközési alakzatot hozok létre, a spotfényforrások esetén egy kúpot leíró convex GJK ütközési alakzatot definiálok. Az osztály *PointLightShadowCulling* metódusában az összes pontfényforráson, a *SpotLightShadowCulling* metódusában a spotfényforrásokon iterálok végig, majd párhuzamos végrehajtással egyenként megvizsgálom az összes DefaultCollider komponenssel rendelkező alakzathoz, hogy metszésben áll-e a fényforrás alakzatával. A metszésvizsgálathoz először a gyorsan elvégezhető AABB ütközési vizsgálatot hajtom végre, és amennyiben metszést detektáltam, akkor a komplexebb, pontosabb eredményt nyújtó GJK metszés vizsgálatot futtatom le.



99. ábra - Pontfényforrás frustum culling

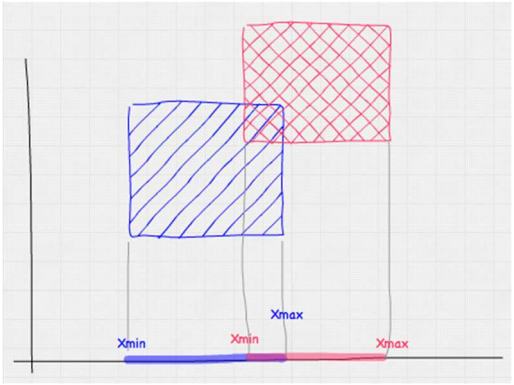
3.4.5.3. Instancing előkészítése

Az **Instance Systemben** az azonos geometriák optimális rendereléséhez implementált Instancing eljárást fogom előkészíteni. A renderelési technikához minden egyszerű és komplex alakzathoz szükségem van az azt felhasználó entitás azonosítójára, transzformációs komponensének indexére, illetve a materiális komponensének az indexére. Ezeket az adatokat vektoros formában fogom összegyűjteni, majd a megfelelő GPU bufferbe feltölteni, ahol az eljárást implementáló shaderek az indexeket felhasználják. Az indexek meghatározásában már figyelembe veszem a nem megjelenő alakzatokat is, és azok indexei természetesen nem kerülnek regisztrálásra a vektorban.

3.4.6. Ütközési alakzatok és algoritmusok

3.4.6.1. AABB ütközés vizsgálat

Az AABB (Axis-Aligned Bounding Box) ütközésvizsgálat egy egyszerű és hatékony módszer a 3D objektumok közötti ütközések detektálására. Az AABB ütközési alakzat egy olyan téglatestet jelent, amely az objektum köré igazodik, és mindig a tengelyekhez igazított élei vannak, azaz nem forog az objektummal együtt. Az AABB ütközésvizsgálat során azt ellenőrizzük, hogy a két téglatest X, Y, Z irányban átfedésben van-e egymással, ezt mutatja be a 100. ábra. Ha mindhárom irányban átfedést találok, akkor a két AABB metszik egymást, vagyis ütközés történt. Ha bármelyik tengely mentén nincs átfedés, akkor a két objektum biztosan nem ütközik.



100. ábra – AABB boxok átfedése

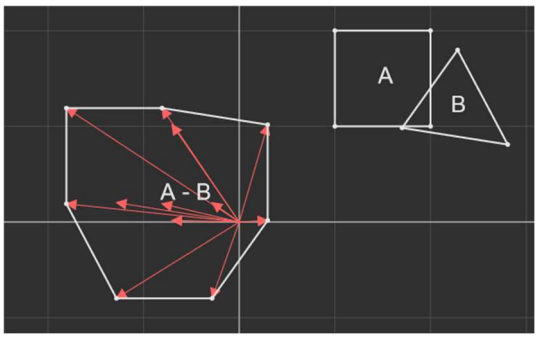
```
return (
    colliderA.min.x <= colliderB.max.x &&
    colliderA.max.x >= colliderB.min.x &&
    colliderA.min.y <= colliderB.max.y &&
    colliderA.max.y >= colliderB.min.y &&
    colliderA.min.z <= colliderB.max.z &&
    colliderA.max.z >= colliderB.min.z
    );
```

101. ábra - Box AABB metszés vizsgálat

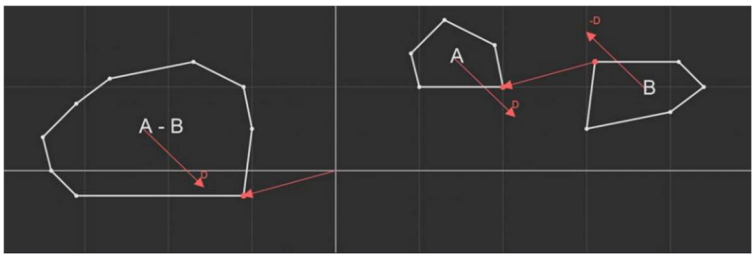
Az AABB alakzatok metszését viszonylag egyszerű és gyors matematikai műveletekkel lehet meghatározni, amit a 101. ábra mutat be. A játékmotorban az AABB alakzat paramétereit, és ütközési algoritmusát az **AABB** osztályban implementáltam.

3.4.6.2. GJK ütközés vizsgálat

A **GJK (Gilbert–Johnson–Keerthi)** ütközési algoritmus egy sokkal fejlettebb, pontosabb eredményt adó algoritmus, amely bármilyen 2D és 3D konvex alakzatok közötti átfedést, metszést ismeri fel. A GJK algoritmus alapja a 102. ábrán is látható **Minkowski-differencia**, amely két alakzat egymásból kivont összes pontjainak távolságából létrejött alakzatot jelenti (*A* alakzat összes pontja – *B* alakzat összes pontja). Az így megkapott geometria, ha tartalmazza az origó (0, 0, 0) pontot, akkor a két alakzat biztosan metszi egymást, hiszen két azonos pontot kell egymásból ahhoz kivonni, hogy az origót kapjuk meg. A Minkowski differencia kiszámításának algoritmusa $O(n^2)$ műveletigényű, ami túlságosan lassú lenne valós környezetben. A GJK feladata tehát az lesz, hogy a Minkowski-differencia geometriájának felületi pontjaiból létrehozzon iteratívan egy olyan háromdimenziós tetraéder alakzatot, amely közrefogja az origót, hiszen ekkor az origó biztosan a geometria része. Az algoritmusban a differencia geometriájának csak a felületi pontjaira van szükség, amelyeket sokkal hatékonyabban is meg lehet határozni, mégpedig úgy, hogy minden iterációs lépésben egy adott irányban és ellenirányban a két alakzat legmesszebbi felületi pontját határozom meg, hiszen ezek különbsége lesz a differencia geometria egy felületi pontja, ezt reprezentálja a 103. ábra. Tehát ha az első alakzat N pontból, a második M pontból áll, akkor N*M műveletigény helyett egy N+M műveletigényű iterációs lépést kapok, amely lépések általában nagyon hamar terminálnak.



102. ábra – Minkowski-differencia

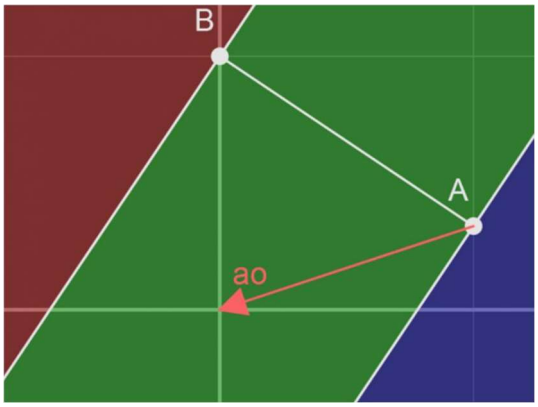


101. ábra - Adott irányban meghatározott legmesszebbi pontok különbsége

Az algoritmus **első lépésében** kiválasztok egy kezdeti irányt, majd mind a két alakzat összes pontján végigiterálva kiszámítom a kezdeti irány és az adott pont skaláris szorzatát és a legnagyobb értékkel bíró pontot elmentem, mivel ez lesz a legtávolabbi pontja az alakzatnak.

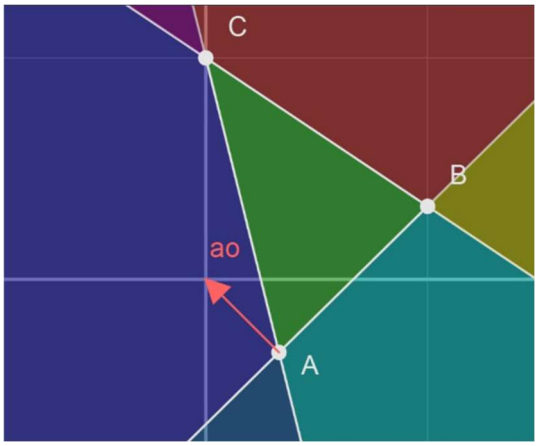
A **második lépésben** a megtalált pontból az origóba mutató irányhoz keresem a legtávolabbi felületi pontot, amit ugyanazzal az eljárással érek el, amit az első pontban már bemutattam. A pontot megtalálva már két felületi pont, azaz egy szakasz áll rendelkezésre, amin

megvizsgálom, hogy milyen irányban érdemes továbblépni, hogy leghamarabb megtaláljam az origót. Ahogyan a 104. ábrán is látható, a *B* pont volt a kezdeti felület pontja és az *A* pont az origó felé lévő legtávolabbi pont. Az origó biztosan nincsen a piros mezőben, hiszen ellentétes irányba találtuk meg az *A* pontot. Az origó vagy a zöld vagy a kék régióban van, amelyet az *AB* és *AO* vektorok skaláris szorzatának értékéből döntök el. Amennyiben a zöld régióban van az origó, akkor továbblépek a következő, a háromszög építésének esetére. Ehhez még meg kell határozni a keresés irányát, ami az AB vektorra merőleges, origó irányba mutató vektor lesz. Ezt az AB és AO vektorok keresztszorzatának eredményét az AB keresztszorzatával vett irányvektor definiálja. Amennyiben a kék régióban van az origó, akkor elhagyom a B pontot, és az A ponttal megismétlem a szakasz létrehozás lépését.



104. ábra - Szakasz esete

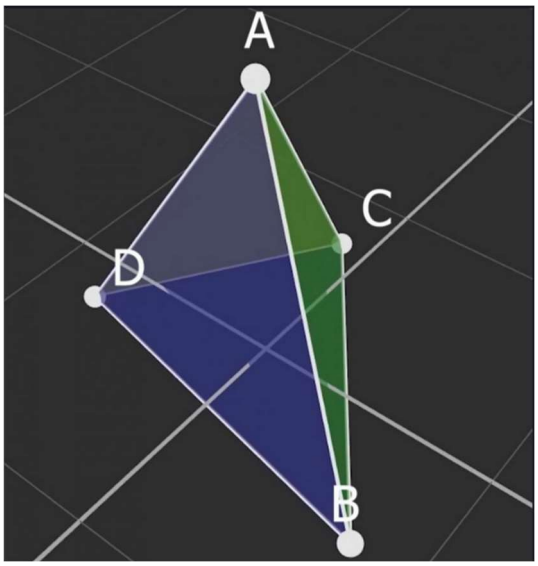
A **harmadik lépésben** meghatározom a harmadik felületi pontot, amellyel már egy térbeli háromszög alakzatot kapok, ez látható a 105. ábrán is. Ebben az esetben a szakaszos esethez hasonlóan lila, piros és sárga régiókat nem kell megvizsgálni, hiszen az origó felé megtalált *A* pont az BC szakasz túloldalán van, azaz az origó biztosan nem ezekben a régiókban lesz. Amennyiben az origó a sötétkék részben található, azaz az AO vektor az AC vektorra merőleges vektorral, illetve magával az AC vektorral vett skaláris szorzata pozitív, akkor elhagyom a B pontot, és az AC szakasszal visszatérek a háromszög építős pontra (3. lépés). Amennyiben az AC vektorral vett skaláris szorzata negatív, akkor a világoskék régióban van az origó, ekkor visszatérek a szakaszos lépésre (2. lépés) az AB szakasszal, és az AO irányvektorral. Ha az origó türkiz kék régióban van, azaz az AO vektor az AB vektorra merőleges vektorral, és magával az AB vektorral vett skaláris szorzata pozitív, ekkor a sötétkék esethez hasonlóan a C pontot eldobom, és az AB szakasszal megyek tovább a háromszög építési lépéssel (3. lépés). Ha az AB vektorral vett skaláris szorzata negatív, akkor a világoskék régióban van az origó,



105. ábra - Háromszög esete

tehát itt is hasonlóan az előbb ismertetettel az AB szakasszal és az AO iránnyal visszatérek a szakasz vizsgálós lépésre (2. lépés). Ha az előbbi feltételek egyikre se teljesül, akkor az AO vektor a zöld régióban van, vagyis az origó vagy a háromszög felett, vagy a háromszög alatt található meg. Kiválasztom a origó felé mutató, háromszögre merőleges irányvektort, majd a továbbmegyek a következő, egyúttal az utolsó lépésre.

A **negyedik lépés** során meghatározok egy újabb felületi pontot a kiválasztott irányba, amely hatására már egy tetraédert fogok kapni, amit a 106. ábra mutat be. A tetraéder egy négyoldalú háromszögekből álló alakzat, tehát ebben a lépésben elég elvégezni a három új oldalra (ABC, ACD, ADB) a 3. lépésben ismertettett háromszögekre történő feltétel vizsgálatot. Amennyiben mind a három oldalra teljesül, hogy az origóba mutató vektor a háromszög vetületén belül van (zöld terület), akkor sikerült közrezárni az origót a Minkowski differencia geometriájának felületi pontjából létrehozott tetraéder alakzattal, azaz a két alakzat biztosan metszi egymást. Ha valamelyik oldalra ez nem teljesül, akkor a 3. lépésben bemutatott további esetekkel folytatódik tovább az algoritmus.

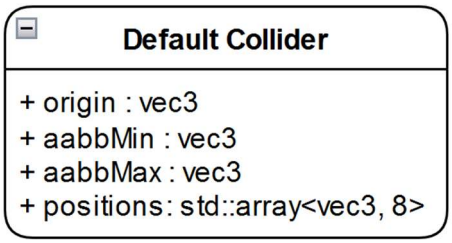


106. ábra - Tetraéder eset

A játékmotorban a GJK algoritmus implementációja a **CollisionTester** osztályban található meg. Implementáltam box, gömb és convex alakzatokhoz GJK ütközési alakzatot. A box üktözési alakzat megfeleltethető a DefaultCollider komponenssel, ami már alapból tartalmazza az objektumok körüli téglatest pontjait. A gömb ütközési alakzat nem tartalmaz felületi pontokat, csak egy origót és egy sugarat, mivel ezeket felhasználva minden irányba meghatározható a GJK számára a legmesszebbi felületi pont, és ez természetesen sokkal gyorsabb eredményt jelent, mintha végig kellene iterálni a gömb összes felületi pontján. A convex ütközési alakzat a geometria összes felületi pontját tartalmazza, és azokon végigiterálva hátározza meg a GJK algoritmusa adott irányba lévő legtávolabbi pontot. Minden optimalizációs rendszerhez adottak a GJK algoritmushoz szükséges ütközési alakzatok, így bennük elvégezhetőek a pontos metszésvizsgálatok.

3.4.6.3. Default ütközési alakzat

A 107. ábrán látható **DefaultCollider Component** tárolja az egyszerű és komplex alakzatok körbefoglaló AABB és OBB ütközési alakzatainak paramétereit. A komponens hatására az optimalizációs rendszerek képesek eldönteni, hogy az alakzat megjelenjen-e a színtérben, illetve a fényforrásokhoz tartozó árnyékszimulációkban. A *positions* 8 hosszúságú tömbben tárolom az OBB alakzat csúcsait, amelyek az alakzathoz tartozó transzformációk változásaira is reagálnak. Ezen csúcsok középpontját pedig az *origin* változó definiálja, amire majd az ütközési alakzatok vizualizációja során lesz szükségem. Az *aabbMin* és *aabbMax* változók a kiszámolt OBB ütközési alakzatra illesztett AABB alakzat minimális és maximális csúcsainak térbeli koordinátáját tartalmazzák.



107. ábra - DefaultCollider komponens struktúra



108. ábra - DefaultCollider vizualizáció

A **DefaultCollider System**ben található a komponens változóinak frissítése és kiszámítása. Az osztály *OnUpdate* metódusában a DefaultCollider komponenssel rendelkező entitásokon párhuzamos végrehajtással végigiterálok. Az entitásokhoz tartozó transzformációs adatokból és az egyszerű vagy komplex alakzatokhoz generált körbefoglaló alakzatok paramétereiből kiszámítom a komponens OBB ütközési alakzatát, majd annak AABB paramétereit.

```
glm::vec3 maxPosition{ std::numeric_limits<float>::lowest() };
glm::vec3 minPosition{ std::numeric_limits<float>::max() };

for (unsigned int i = 0; i < 8; ++i)
{
    glm::vec4 point = transformComponent.fullTransform * glm::vec4(obb[i], 1);
    defaultColliderComponent.positions[i] = glm::vec3(point);

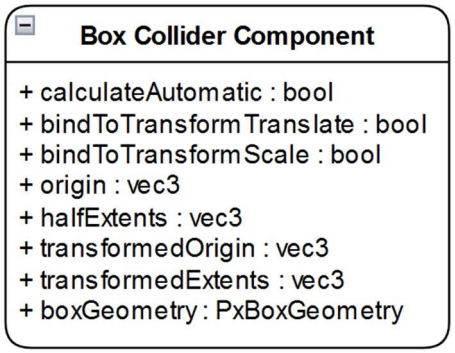
    maxPosition = glm::max(maxPosition, defaultColliderComponent.positions[i]);
    minPosition = glm::min(minPosition, defaultColliderComponent.positions[i]);
}

defaultColliderComponent.aabbMin = minPosition;
defaultColliderComponent.aabbMax = maxPosition;
defaultColliderComponent.origin = transformComponent.fullTransform * glm::vec4(origin, 1);
```

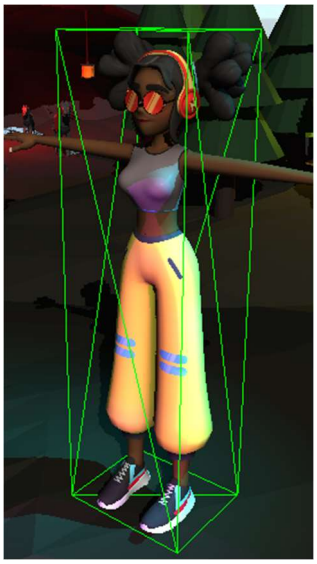
109. ábra - DefaultCollider AABB és OBB paraméterek kiszámítása

3.4.6.4. Téglatest ütközési alakzat

A **BoxCollider Component** egy téglatest alakú ütközési alakzatot definiál, amely a PHYSX könyvtár által definiált Box ütközési alakzatra épül, ennek osztálydiagramja látható a 110. ábrán. Az ütközési alakzat középpontját az *origin* változó, a térbeli kiterjedését a *halfExtents* változó tartalmazza. A *bindToTransformTranslate* opció segítségével hozzákötöm a téglatest csúcsait az eltolás transzformációhoz, illetve a *bindToTransformScale* opció segítségével a téglatest méretét kötöm a skálázáshoz. Így elegendő beállítani az ütközési alakzat paramétereit az objektum körül, majd annak mozgatására/skálázására a paraméterek megfelelően reagálnak. A *calculateAutomatic* opció segítségével az entitáshoz tartozó egyszerű vagy komplex alakzatok által definiált alapvető körbefoglaló alakzat paraméterei használhatóak a komponens ütközési alakzatjának létrehozásához. A *transformedOrigion* és *transformedExtents* a már eltranszformált paramétereket tárolják. A boxGeomerty változóban pedig a PHYSX Box ütközési egy alakzat példányát tárolom.



110. ábra - Téglalap ütközési alakzat komponens struktúra

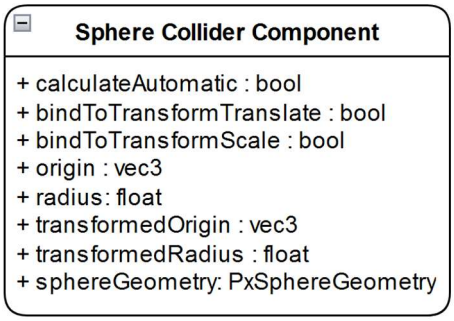


111. ábra - Téglalap ütközési alakzat vizualizációja

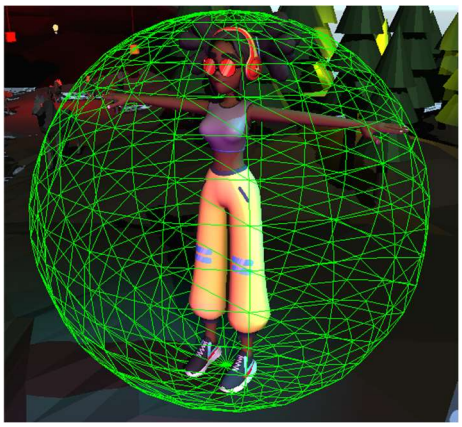
A **BoxCollider System** felelős a komponens paramétereinek megfelelő kiszámításáért, a PHYSX objektumok létrehozásáért, illetve az ütközési alakzat vizuális megjelenítéséhez szükséges adatok GPU-ra való feltöltéséért. Az osztály *OnUpdate* metódusában végigiterálok a BoxCollider komponenssel rendelkező entitásokon, majd változás esetén kiszámolom a szükséges paramétereket, meghatározom a vizualizációs transzformációkat és feltöltöm a megfelelő bufferbe.

3.4.6.5. Gömb ütközési alakzat

A **SphereCollider Component**, aminek az osztálydiagramja a 112. ábrán látható, egy gömb alakú ütközési alakzatot definiál, amely a PHYSX könyvtár által definiált Sphere ütközési alakzatra épül. A gömb középpontját az *origin* változó, a sugarát a *radius* változó tartalmazza. A *sphereGeomerty* változó pedig a PHYSX Sphere ütközési alakzat egy példányát tárolja. A komponens további felépítése teljesen megegyezik a BoxCollider komponensnél bemutatottal.



112. ábra - Gömb ütközési alakzat komponens struktúra

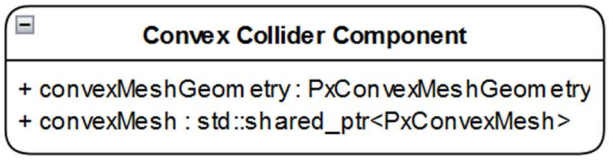


113. ábra - Gömb ütközési alakzat vizualizációja

A **SphereCollider System** felelős a komponens paramétereinek megfelelő kiszámításáért, a PHYSX objektumok létrehozásáért, illetve az ütközési alakzat vizuális megjelenítéséhez szükséges adatok GPU-ra való feltöltéséért. Az osztály *OnUpdate* metódusában végigiterálok a SphereCollider komponenssel rendelkező entitásokon, majd változás esetén kiszámolom a szükséges paramétereket, meghatározom a vizualizációs transzformációkat és feltöltöm a megfelelő bufferbe.

3.4.6.6. Convex ütközési alakzat

A **ConvexCollider Component** egy konvex alakú ütközési alakzatot definiál, amely a PHYSX könyvtár által definiált Convex ütközési alakzatra épül. A komponens osztálydiagramját mutatja be a 114. ábra. Konvex alakzatokra teljesül, hogy bármely két pontját összekötő szakasz mindig az alakzaton belül marad. Konvex alakzat például a kocka, henger, kúp, de a tórusz már konkáv. A *convexMesh* változó a konvex alakzatot térbeli pontjait és azok számát tárolja, a *convexMeshGeometry* pedig a PHYSX Convex ütközési alakzat egy példányát tárolja.



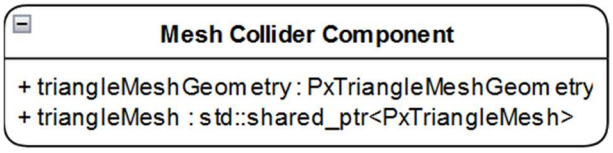
114. ábra - Konvex ütközési alakzat struktúra

A **ConvexCollider System** felelős a komponensekhez tartozó paraméterek frissítéséért, a PHYSX objektumok létrehozásáért. Az osztály *OnUpdate* metódusában végigiterálok a ConvexCollider

komponenssel rendelkező entitásokon, majd lekérdezem a megadott geometriai alakzat térbeli pontjait (Shape/Mesh Component), és azok felhasználásával példányosítom a PHYSX objektumokat.

3.4.6.7. Háromszög ütközési alakzat

A **MeshCollider Component** egy tetszőleges alakzat háromszögein alapuló, tűpontos ütközési alakzatot definiál, amely a PHYSX könyvtár által definiált Triangle ütközési alakzatra épül, ennek az osztálydiagramja tekinthető meg a 115. ábrán. A *triangleMesh* változó a tetszőlegesen komplex alakzat térbeli pontjait és számát tárolja, továbbá a térbeli pontokat háromszöggé összekötő indexeket is tartalmazza. A *conexMeshGeomerty* a PHYSX Triangle ütközési alakzat egy példányát tárolja.



115. ábra - Háromszög ütközési alakzat komponens struktúra

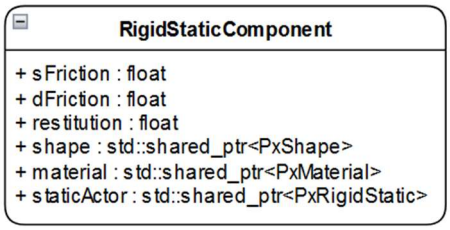
A **MeshCollider System** felelős a komponensekhez tartozó paraméterek frissítésért, a PHYSX objektumok létrehozásáért. Az osztály *OnUpdate* metódusában végigiterálok a MeshCollider komponenssel rendelkező entitásokon, majd lekérdezem a megadott geometriai alakzat térbeli pontjait (Shape/Mesh Component), és azok felhasználásával példányosítom a PHYSX objektumokat.

3.4.7. Fizikai szimuláció

A játékmotor fizikai szimulációját a PHYSX könyvtár funkcionalitásával valósítottam meg. Az objektumok közötti ütközési alakzatokat már részletesen bemutattam fentebb, amelyek segítségével már feloldhatóak az ütközések. A tényleges fizikai szimulációhoz viszont még szükség van az ütközési alakzatok mellett szilárdtestekre is, amelyek az objektumok közötti (illetve gravitáció) erőhatásokat definiálják. A PHYSX inicializálásához létrehoztam a megfelelő *foundation* és *physics* objektumot, amelyek segítségével létrehozható lesz a fizikai szimulációs jelenet, amihez a szilárdtesteket fogom tudni hozzáadni, amelyek közötti erőhatásokat a PHYSX maga fogja leszimulálni. A szimulációs tér létrehozása előtt beállítom a gravitációs erőhatást, illetve implementáltam egy ütközési filtert, amely az alakzatok ütközése kezdetén és végén eseményeket váltanak ki, melyek hatására a felhasználói scriptek *OnCollisionEnter* és *OnCollisionExit* metódusai meghívódnak.

3.4.7.1. Statikus szilárdtest

A **RigidbodyStatic Component**, melynek osztálydiagramja a 116. ábrán látható, tárolja a mozdulatlan, állandó helyzetű, gravitációval nem kölcsönható fizikai objektumokhoz szükséges paramétereit, a komponens hatására az entitás részt vesz a fizikai szimulációban. A komponens *sFriction* változója a szilárdtest önmagára ható erőhatás erősségét tárolja, a *dFriction* a más szilárdtestek ütközéséből keletkező erőhatások erősségét jelenti, illetve a *restitution* a pattogás erősségét definiálja. A *staticActor* változó a fizikai szimulációban résztvevő tényleges objektumot tárolja, amely létrehozásához szükséges ütközési alakzat és materiális információkat a *shape* és *material* PHYSX objektumokban tárolom el.

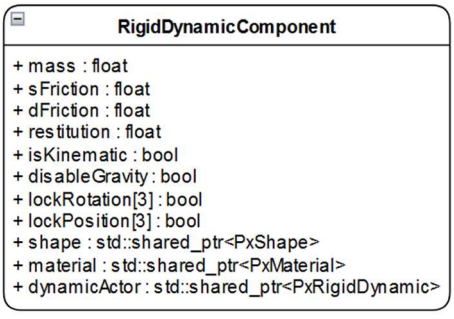


116. ábra - Statikus szilárdtest komponens struktúra

A **RigidbodyStatic Systemben** implementálom a komponensek frissítését, szükséges fizikai objektumok létrehozását. Az osztály *OnUpdate* metódusában a RigidbodyStatic komponenssel rendelkező entitásokon iterálok végig, és amennyiben az entitás rendelkezik valamilyen már fentebb bemutatott ütközési komponenssel, akkor a PHYSX objektumokat megfelelően inicializálom, majd a fizikai szimulációs térbe regisztrálom. Az *UpdateRigidbodyGlobalPose* metódusban frissítem a szilárdtestek fizikai szimulációs pozícióját és forgatását, amennyiben az entitás transzformációs változói változtak. Erre azért van szükség, hogy a fizikai szimuláció közben is szabadon lehessen mozgatni, forgatni az objektumot.

3.4.7.2. Dinamikus szilárdtest

A **RigidbodyDynamic Component**, melynek osztálydiagramja a 117. ábrán látható, tárolja az erőhatásokra reagáló, irányukat és sebességüket változtató fizikai objektumok paramétereit, a komponens hatására az entitás részt vesz a fizikai szimulációs térben. A komponens felépítése nagyon hasonlít a statikus szilárdtesteknél bemutatottra. Az *sFriction*, *dFriction*, *restitution*, *shape*, *material* változók értelme teljesen megegyezik a fenti paramétereknél ismertetettel. A *mass* változóban a szilárdtest tömegét tárolom, a *disableGravity* opcióval pedig a gravitációs kölcsönhatást lehet be/kikapcsolni. A *lockRotation*



117. ábra - Dinamikus szilárdtest komponens struktúra

és *lockPosition* opciókkal az ütközés során fellépő forgatás/pozíció (XYZ) változást lehet be/kikapcsolni. Az isKinematic változóval a szilárdtestet kinematikusra állítható be.

A **RigidbodyDynamic Systemben** implementálom a komponensek frissítését, szükséges fizikai objektumok létrehozását. Az osztály *OnUpdate* metódusában a RigidbodyDynamic komponenssel rendelkező entitásokon iterálok végig, és amennyiben az entitás rendelkezik valamilyen már fentebb bemutatott ütközési komponenssel, akkor a PHYSX objektumokat megfelelően inicializálom, majd a fizikai szimulációs térbe regisztrálom. Az *UpdateRigidbodyGlobalPose* metódusban frissítem a szilárdtestek fizikai szimulációs pozícióját és forgatását, amennyiben az entitás transzformációs változói változtak. A *FetchRigidbodyGlobalPose* metódusban a szilárdtestek fizikai szimulálása utáni pozíció és forgatási paramétereit határozom meg, majd a transzformációs komponensek változóit szinkronizálom velük. Erre azért van szükség, mert a további rendszerek felhasználhatják az entitás Transform komponensének transzformációs értékeit, amelyek már a szimuláció utáni értékeket kell tartalmaznia.

3.4.7.3. Fizikai tér szimulálása

Miután minden entitáshoz tartozó ütközési alakzatokat létrehoztam/frissítettem, majd minden fizikai szilárdtestet inicializáltam/frissítettem és a szimulációs fizikai térbe beregisztráltam, akkor elindíthatom a tényleges fizikai szimulációt. A létrehozott PHYSX fizikai jelenet objektum *simulate* metódusát meghívva, az előző frametől számított eltelt időt paramétert átadva elvégzem a fizika szimulálását, aminek a kódját a 118. ábra mutatja be.

```
gScene->simulate(deltaTime);
gScene->fetchResults(true);
```

118. ábra - Fizikai szimuláció futtatása (PHYSX)

```
gFoundation = PxCreateFoundation(PX_PHYSICS_VERSION, gAllocator, gErrorCallback);
gPhysics = PxCreatePhysics(PX_PHYSICS_VERSION, *gFoundation, PxTolerancesScale());
collisionCallback = new CollisionCallback(this->m_Registry);
PxSceneDesc sceneDesc(gPhysics->getTolerancesScale());
sceneDesc.gravity = PxVec3(PhysicsSystem::gravity.x, PhysicsSystem::gravity.y, PhysicsSystem::gravity.z);
gDispatcher = PxDefaultCpuDispatcherCreate(4);
sceneDesc.cpuDispatcher = gDispatcher;
sceneDesc.filterShader = CollisionCallback::FilterShader;
sceneDesc.simulationEventCallback = collisionCallback;
gScene = gPhysics->createScene(sceneDesc);
```

119. ábra - PHYSX inicializálása

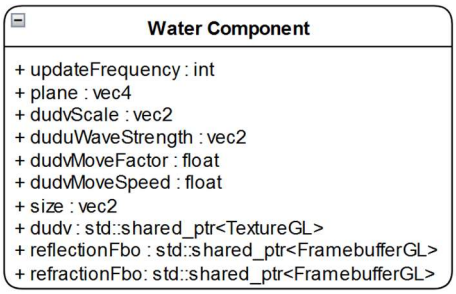
3.4.8. További komponensek és rendszerek

3.4.8.1. Víz

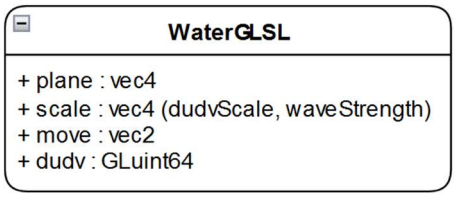
A 121. ábrán definiált **Water Component** tartalmazza a vízszimuláláshoz szükséges paramétereket, a komponens hatására a játéktérben egy vízszintes felületű tükröződő vízfelület jelenik meg. A sík felszínt leíró normálvektort és magasságot a *plane* változó tartalmazza. A víz hullámázásának erősségét és gyorsaságát a *dudvMoveStrength* és *dudvMoveSpeed* változóban tárolom. A *dudvMoveFactor* tartalmazza ezen változók kombinációját, amely paraméterre majd a shadereknek lesz szüksége a tényleges vízhullámzás szimulálásához. A hullámzás irányát a *dudv* textúra értékei definiálja, illetve a *dudvScale* változóval biztosítom, hogy XY koordináták mentén az alakzatokhoz tartozó textúra koordináták skálázódjanak, melynek hatására a hullámzási textúra többször is megjelenik az adott felületen. A víz rendereléséhez szükséges fénytörési és fényvisszatükröződési textúrák gpu erőforrásai tárolom a *reflectionFbo* és *refractionFbo* változókban. A két textúra felbontási méretét a *size* változó tartalmazza. A textúrák frissítési rátáját az *updateFrequency* változó definiálja.



120. ábra - Vízszimuláció



121. ábra - Vízszimuláció komponens struktúra



122. ábra - WaterGLSL struktúra

A **WaterGLSL** struktúra tartalmazza a víz rendereléséhez szükséges adatokat, amelyeket a vízszimuláló shaderekben fogok felhasználni, ennek az osztálydiagramja látható a 122. ábrán.

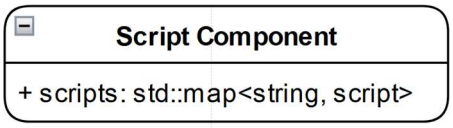
A **Water System** osztályban implementáltam a komponensek frissítését. Az *OnUpdate* metódusban végigiterálok a Water komponenssel rendelkező entitásokon, szükség esetén újragenerálom a fénytörési és fényvisszatükröződési textúrákhoz tartozó OpenGL-es erőforrásokat, illetve az idő függvényében frissítem a hullámzást leíró paramétereket.

3.4.8.2. Hang/Zene

A hangok és zenékhez nem implementáltam külön komponenst a játékmotorban. Hangokat természetesen támogat a játékmotor, a hangok betöltéséért, és hatékony tárolásáért a **SoundManager** osztály felelős. A játékmotor támogatja a 2D és 3D hangokat is. A hangok és zenék lejátszását, paramétereinek kezelését a felhasználói scriptekben lesz lehetőség elindítani és beállítani.

3.4.8.3. Scriptek

A **Script Component** tartalmazza az entitásokhoz csatolt felhasználói scripteket, ezt mutatja be a 123. ábra. A felhasználói scriptekben definiáltam többek között *OnStart*, *OnUpdate*, *OnCollisionEnter*, *OnCollisionExit*, *OnGui* metódusokat, amelyekben a felhasználó az entitás komponenseinek paramétereit változtathatja, ütközésekre reagálhat és tetszőleges ImGui interfészablakokat hozhat létre.



123. ábra - Script komponens struktúra

A **Script System** felelős a felhasználói script projekt betöltéséért, a betöltött scriptek metódusainak végrehajtásáért, illetve a script projekt megfelelő felszabadításáért. Az osztály *LoadScripts* metódusában implementáltam a felhasználói scripteket tartalmazó Visual Studio 2022 projekt lefordítását, majd az így létrejött .dll állomány betöltését a LoadLibrary (windows.h) függvény felhasználásával a programban. Az állomány betöltése után végigiterálok az összes Script komponenssel rendelkező entitáson, és azok scripts listájában lévő névazonosítók alapján meghívom a .dll állományban definiált a megfelelő script objektumot példányosító függvényeket. A létrejött script objektumot végül eltárolom a komponens scripts listájába. Az osztály *OnStart* metódusa felelős a betöltött scriptek *OnStart* metódusának meghívásáért, illetve az *OnUpdate* metóduson belül hívom meg a scriptek *OnUpdate* metódusát, amely minden frameben végrehajtódik. Az osztály *FreeScripts* metódusa felelős a betöltött .dll állomány felszabadításáért, amit FreeLibrary (windows.h) függvény felhasználásával érek el. Az állomány törlése előtt szükséges a példányosított script objektumokat törölni.

3.5. Renderelés

Szakdolgozatom egyik legfontosabb célja a színtér renderelésének hatékony felépítése és végrehajtása volt. A renderelési feladatokhoz az OpenGL API-t választottam, mivel ez áll legközelebb a tudásomhoz, és a három legelterjedtebb render API (OpenGL, DirectX, Vulkan) közül ez bizonyult a legcélszerűbbnek, egyrészt letisztultsága miatt, másrészt funkcionalitását tekintve kielégít szinte minden igényt, ami egy játékmotor felépítéséhez szükséges. Az **OpenGL 4.6**-os verzióját használtam, amely számos fejlett, ámbár sokak által nem ismert funkcióval rendelkezik, melyeknek köszönhetően egy sokkal optimalizáltabb, bizonyos esetekben logikusabban felépíthető renderelés valósítható meg. Én alapvetően a **Direct State Access** (DSA) funkcionalitást használtam fel extraként, amely elsősorban lehetővé teszi, hogy az OpenGL által lefoglalt GPU erőforrások használatához ne kelljen többé előzetesen bindolást alkalmazni (teljesítményben is előnyösebb, nincsenek felesleges GPU függvényhívások), illetve sokkal optimalizáltabb, összetettebb bufferkezelést is biztosít. A másik nagy OpenGL 4.6 konstrukció, amit sok helyen felhasználtam, az a **Bindless Textúra** generálás, amely lehetővé teszi a GPU számára, hogy egy adott textúrát egy egyedi 64 bites szám azonosítson a shaderekben, amelyeket majd textúra sampler objektumokra konvertálok át. A shaderkeben a bindless textúrák funkcionalitásának aktiválásához a *#extension GL_ARB_bindless_texture : require* direktívát kell definiálni. Nagyon komplex, akár több 100 textúrát is felhasználó modellek esetén a shaderek által biztosított 32 textúra bindolási egység túl kevésnek bizonyul. Potenciális megoldás lenne, ha textúra tömbökben tárolnánk a betöltött textúrákat, de ezek kezelése túlságosan korlátolt, körülményes. Ilyen esetekben nagyszerű megoldás a bindless textúrák felhasználása.

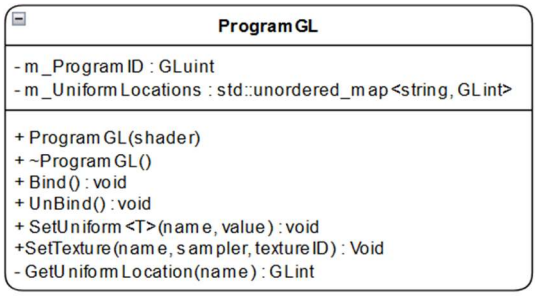
3.5.1. OpenGL absztrakciók

3.5.1.1. Shaderek

A **ShaderGL** osztályban implementáltam a különböző típusú GLSL nyelven íródott shaderkódok betöltését OpenGL-es shader objektumokba. Az osztályban támogatom a vertex, geometry, fragment, compute és tessellation shaderek betöltését. Minden betöltött shaderhez az OpenGL egy azonosítót rendel, amit az *m_ShaderID* változóban tárolok el. Hibás shaderkód esetén a beépített hibafigyelő jelez, és a console felületre kiírja a hiba pontos okát, helyét.

3.5.1.2. Programok

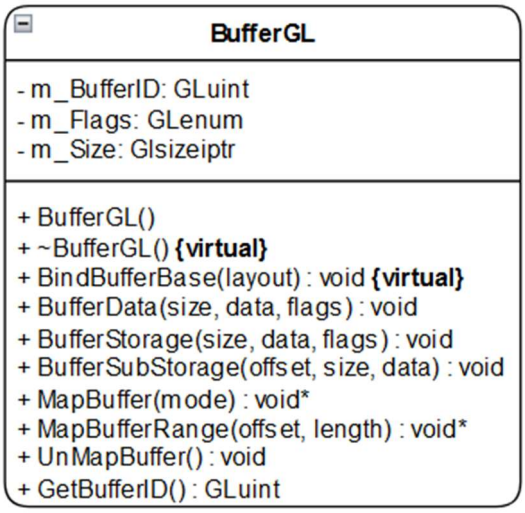
A 124. ábrán látható **ProgramGL** osztállyal kezelem az egymáshoz tartozó shaderek megfelelő összekötését, illetve az OpenGL-es programobjektumok létrehozását. Az osztály generikus *SetUniform<T>* metódusában kezelem az uniform változók feltöltését. A *SetTexture* metódusban pedig a textúrák hagyományos feltöltését és aktiválását kezelem.



124. ábra - Program absztrakció

3.5.1.3. Bufferek

A **BufferGL** ősosztály tartalmazza az OpenGL-es buffer objektumok közös alapvető funkcionalitását és paramétereit, ennek az osztálydiagramját mutatja be 125. ábra. Ezzel az osztállyal elegendő funkcionalitást biztosítok a bufferek létrehozására, adatok átmásolására, olvasására és az erőforrások megfelelő megsemmisítésére is. A *BufferData*, *BufferStorage* metódusokon keresztül változtatható méretű, illetve inicializálás után fix méretű tárhelyet foglalhatok le a bufferek számára a GPU memóriájában. A *BufferData* és *BufferSubStorage* metódusokon keresztül adatokkal tölthetem fel a buffereket. A *MapBuffer* és *MapBufferRange* metódusok segítségével a bufferekhez memóriapointereket tudok generálni, amelyeken keresztül módosíthatom a bufferek tartalmát.



125. ábra - Buffer absztrakció

A **VertexBufferGL** osztályban az alakzatokhoz tartozó pozíció, normál vektor, textúra koordináta adatokat tárolom. Az ősosztály által biztosított funkcionalitás teljes mértékben lefedi a geometriai adatok tárolásához szükséges funkcionalitást, így a VertexBufferGL osztály egy aliasa a BufferGL osztálynak. (using VertexBufferGL = BufferGL;)

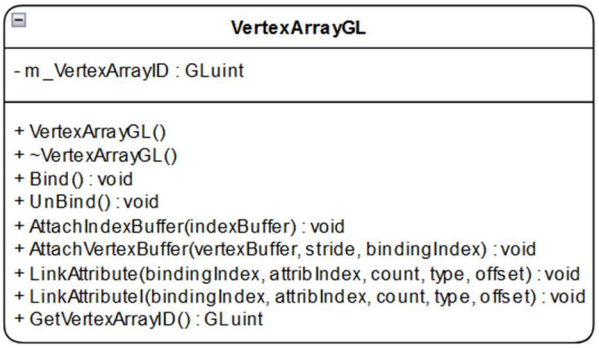
Az **IndexBufferGL** osztályban tárolom az alakzatokhoz tartozó vertexeket háromszögekké összekötő index adatokat. Hasonlóan a fent említettekhez, az IndexBufferGL semmilyen különleges funkcionalitást nem igényel, így az osztály egy aliasa lesz a BufferGL osztálynak.

A **UniformBufferGL** osztályban a rendereléshez szükséges uniform adatokat tárolhatom bufferes formában. A hagyományos uniform változókhoz képest az uniform bufferek használata jelentősen hatékonyabb, mivel elegendő az uniform adatot egyszer feltölteni a bufferbe, majd csak akkor frissíteni, ha a CPU oldalon változás történt. (Az uniform változós megoldás minden render hívás előtt, minden shaderbe framenként újra feltöltötte az adatokat). Az UniformBufferGL a BufferGL ősosztályból származik, és felüldefiniálja a *BindBufferBase* virtuális metódust. Ez a metódus biztosítja, hogy a buffer megfelelően azonosítható legyen a shader számára, így annak tartalma közvetlenül elérhetővé válik. Az uniform bufferek az std140 memória layoutot használják, és általában maximum 64 kB adatot képesek tárolni, ami sok esetben kevés lehet.

A **ShaderStorageBufferGL** osztály hasonló szerepet tölt be, mint az UniformBufferGL osztály, annyi különbséggel, hogy egy sokkal modernebb megoldást kínál az uniform változók tárolására. A shader buffer objektumok jelentős előnyökkel bírnak az uniform buffer objektumokhoz képest, egyrészt tartalma felhasználható, és akár írható is compute shaderek számára, másrészt 128MB adatot képes eltárolni. A buffer az std430 memória layoutot használja, amely rugalmasabb és hatékonyabb adatkezelést biztosít, különösen nagyobb adatmennyiségek esetén.

3.5.1.4. VertexArray és Vertex struktúra

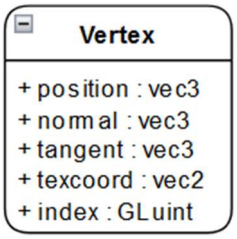
A **VertexArrayGL** osztályban implementáltam a vertexarray objektumok létrehozását és funkcionalitását, ennek az osztálydiagramja látható a 126. ábrán. Az osztály az alakzatokhoz tartozó vertex és index bufferek tartalmát köti össze, illetve a vertex struktúra memória felépítését definiálja a GPU számára. Az *AttachVertexBuffer* és *AttachIndexBuffer* a fent említett VertexBufferGL és IndexBufferGL objektumokat vár paraméterül, amelyeket megfelelően összecsatol. A



126. ábra - VertexArray absztrakció osztálydiagram

LinkAttribute és *LinkAttributeI* metódusok pedig a vertexek (bármilyen tetszőleges struktúra is lehet) belső szerkezetét definiálja. Annak ellenére, hogy DSA funkcionalitást használom, a Vertex Array használata előtt szükség lesz az erőforrás bindolásához.

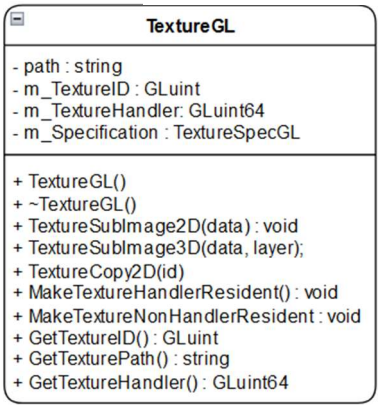
A 127. ábrán látható **Vertex** struktúra az objektumok geometriai adatait fogja tárolni, beleértve a pozíció, normál, tangent vektorokat, textúra koordinátákat és egy egyedi textúrát azonosító ID-t. Ezeket az adatokat fogják a shaderek megkapni bemeneti paraméterként.



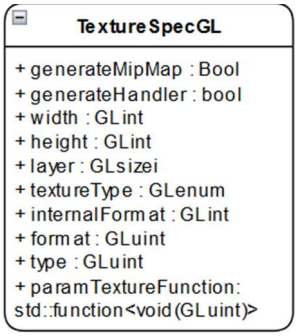
127. ábra - Vertex struktúra

3.5.1.5. Textúrák

A 128. ábrán megtekinthető **TextureGL** osztályban absztraktáltam az OpenGL-es textúra objektumok funkcionalitásait. A textúra létrehozásához szükséges paramétereket a **TextureSpecGL** struktúrában tárolom (129. ábra), amelyre a textúra inicializálásánál lesz főleg szükség. A struktúra *paramTextureFunction* változója tartalmazza a textúra létrehozásánál meghívandó OpenGL textúraparaméterek beállítását. A kívülről testre szabható függvénnyel érem el, hogy akár kétdimenziós, akár háromdimenziós textúrák paramétere is beállítható legyen. Az osztály *TextureSubImage2D* és *TextureSubImage3D* metódusa felel a betöltött textúra adatainak feltöltéséért, mely metódusban lehetőség van továbbá mipmap generálásra és bindless textúra azonosítók létrehozására is.



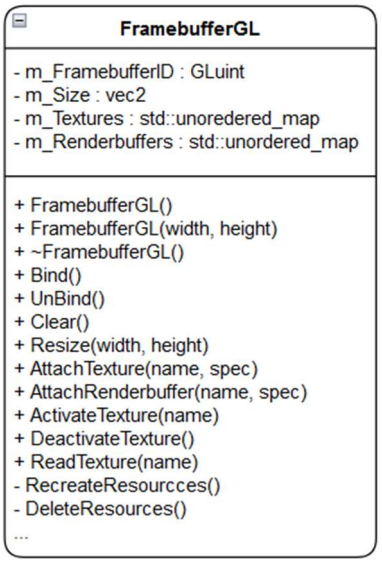
128. ábra - Textúra absztrakció osztálydiagram



129. ábra - Textura specifikáció

3.5.1.6. Framebufferek

A **FramebufferGL** osztállyal, melynek osztálydiagramja a 130. ábrán látható, egy teljesen leáltalánosított konstrukciót biztosítok az OpenGL framebuffer objektumaihoz, amelyeket a háttérben történő rendereléshez fogok felhasználni, azaz a játék színterét (és bármi mást) a grafikus ablak helyett OpenGL-es textúrákba fogom kirenderelni, a textúrákat tetszés szerint módosítani, majd a végleges színtértextúrát a grafikus ablakban megjeleníteni. A játékmotor deferred rendering technikát fogja alkalmazni, ami erősen épít a framebufferek által nyújtott lehetőségekre.



130. ábra - Framebuffer absztrakció osztálydiagramja

A framebufferek két fő komponensből tevődnek össze: a textúrákból és a renderbufferekből. A textúrák általánosításához a TextureGL osztálynál bemutatott megoldást fogom alkalmazni, annyi különbséggel, hogy ezentúl rendelkezésre áll egy *clearTextureFunction* függvényváltozó a speciális textúra törlési mechanizmus definiálásra, illetve egy *readTextureFunction* függvényváltozó a textúra egy adott pixeléhez tartozó adatainak visszaolvasására. A textúrák létrehozását és a framebufferhez való csatolását az *AttachTexture* függvényben valósítottam meg. A renderbufferek a rendereléshez szükséges mélységértékek és stencil értékek tárolásában töltenek be szerepet. Ennek a generálását és framebufferhez való csatolását az *AttachRenderbuffer* függvény végzi el. A létrehozott textúrákat és renderbuffereket névazonosítóval ellátott asszociációs listában tárolom (std::unordered_map), mivel a textúrák lekérdezésére a renderelési rendszereknek, és a grafikus interfésznek is szüksége lesz. Renderelés előtt a felhasználni kívánt textúrákat outputként aktiválni kell, amelyet az *ActivateTexture(s)* metódusban implementáltam.

3.5.2. Forward Shading vs Deferred Shading

A forward rendering a számítógépes grafika egyik legalapvetőbb technikája, amelynek alapelve, hogy a különféle fényforrások mindegyike rendelkezésre áll a fragment shader számára. A renderelési folyamat során minden objektum esetében létrejönnek a fragmentek, amelyek színét az összes fényforrásra vonatkozóan kiszámoljuk. Az algoritmus minden egyes fényforráson végigiterál, és meghatározza az adott fényforrás által kifejtett hatást az adott fragment anyagmodelljére, ezáltal módosítva a színét. Az összes fényforrás feldolgozása után kapunk egy színt, amely végül megjelenik a képernyőn. A módszer legnagyobb hátránya, hogy ahogy a fényforrások száma növekszik, úgy esik vissza jelentősen a teljesítmény. A renderelés során minden egyes pixelhez tartozó mélység- és színérték felülíródik, ha egy közelebbi objektumhoz tartozó fragment jön létre. Ez azt eredményezi, hogy a korábban kiszámolt színérték, azaz a fényszimuláció gyakorlatilag elvész, és felesleges munkává válik. Nagyobb jelenetekben gyakoriak ezek a felülírások, ami azt jelenti, hogy a GPU rengeteg erőforrást pazarol el azáltal, hogy akár több száz fényforrás hatását is kiszámolja egy pixelre, csak azért, hogy aztán egy másik renderelési lépés során ezek felülíródjanak.

A deferred shading megoldást kínál a forward rendering egyik legnagyobb problémájára. Ez a technika két fázisra osztható: az első a geometriai adatgyűjtés, a második pedig a fényszámítások elvégzése. Az első fázisban minden objektumot kirenderelünk hagyományos módon, azonban a fényszámítás helyett a fragment shader különböző framebuffer textúrákba menti el a fragmentekhez tartozó adatokat. Ezek közé tartoznak például az objektum világkoordinátái, színe, normál vektora, valamint minden egyéb információ, amely a későbbi fényszámításokhoz szükséges lehet.

A második fázisban ezekből a textúrákból olvassuk ki az adatokat, és ezeket felhasználva végezzük el a fényszámításokat. Ezzel a megközelítéssel megszabadulunk a forward rendering egyik legnagyobb hátrányától: a fragment adatok felülírását és a vele járó felesleges fényszámításokat. Az első fázis végeztével már csak azok az adatok maradnak meg a textúrákban, amelyek valóban a megjelenő pixelekhez tartoznak. Ennek köszönhetően a második fázisban minden számítást csak egyszer kell elvégezni. A deferred rendering további előnye, hogy az egyes fényforrásokhoz egyszerűen meghatározható egy körbefoglaló alakzat, amelyen belül a fényhatás érvényesül. Például egy pontfényforrás hatása egy gömb alakban érzékelhető, így elegendő csak azokat a pixeleket vizsgálni, amelyek ebbe az alakzatba esnek.

Ez azt jelenti, hogy a második fázisban nem szükséges minden fényforrást minden egyes pixelre kiszámolni, csak azokra a pixelekre, amelyekre a fényhatás valóban vonatkozik. Ez jelentős teljesítményjavulást eredményez. A deferred shading egyik legnagyobb hátránya azonban a magas memóriaigény. Egy 1920x1080 felbontású framebuffer esetén rengeteg adatot kell eltárolni a GPU-n, és a shader programok textúraolvasási műveletei is lassabbak. Mégis, a modern GPU-k teljesítménye elegendő ahhoz, hogy ez ne jelentsen jelentős problémát. A deferred rendering lehetővé teszi, hogy több ezer fényforrást szimuláljunk viszonylag magas teljesítmény mellett, amit a forward rendering módszerrel aligha érhetnénk el.

3.5.3. Instance renderelés

Az Instance renderelés egy hatékony renderelési technika, amit akkor alkalmazunk, amikor sok azonos geometriából álló objektumot kell megjeleníteni, de ezek mind különböző pozícióban, méretben, vagy orientációban helyezkednek el. A hagyományos renderelési technika során minden objektumhoz külön rajzolási parancsot hívok, ami rengeteg redundáns adatátvitelt igényel a CPU és GPU között, mivel minden egyes példányhoz újra és újra elküldésre kerülnek az objektumot leíró geometriai adatok. Az Instance renderelési technikával ezt a felmerülő problémát fogom orvosolni. Az objektumok megjelenítéséhez elegendő egy rajzolási függvényhívás, amely során egyszer küldöm el a geometriai adatokat és az összes példányhoz szükséges egyedi transzformációs mátrixokat a GPU-nak. A vertex shaderben ilyenkor a gl_InstanceID értéket használom fel az objektumok azonosítására, illetve az ahhoz tartozó transzformációs és egyéb shader buffer adatok lekérdezésére. Az Instance rendereléshez szükséges háttérkódok felépítése jóval összetettebb, nehezebben kivitelezhető a hagyományos megoldással szemben, viszont teljesítmény optimalizálása szempontjából egy elengedhetetlen része a játékmotornak.

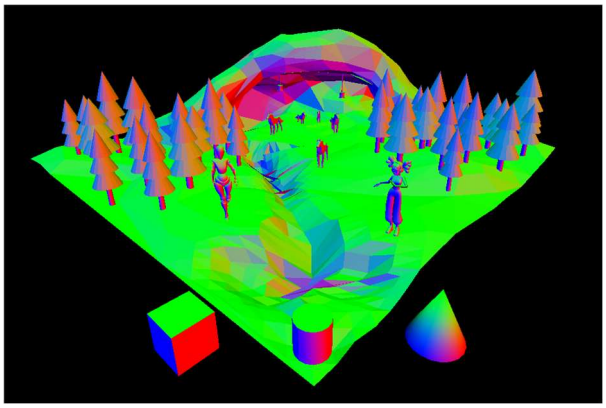
3.5.4. Játékmotor Renderelési Fázisai

3.5.4.1. GeometryRenderer

A Geometry Renderer osztállyal implementálom a deferred shading technika első lépését, amelyben az alakzatokhoz tartozó geometriai és egyéb, a további rendereléshez szükséges adatokat mentem el OpenGL textúrákba. Az osztály 5 fő metódusból áll, melyek a különböző típusú alakzatokhoz tartozó más-más renderelési eljárásokat implementálnak. Az alakzatok rendereléséhez együttesen a *DeferredPre* shaderprogramot használom, amely vertex shader állománya összefogja az öt eljárás szükséges lépéseit. A fragment shader pedig a már raszterizált alakzat fragmentjeihez fogja elmenteni a hozzá tartozó adatokat, ez tekinthető meg a 131. és 132. ábrákon.



131. ábra - Szín textúra



132. ábra - Normál vektor textúra

A *RenderShapes* metódusnan az Engine által generált egyszerű alakzatokat renderelem hagyományos módon. Először a kamerához tartozó adatok, majd az entitásokhoz tartozó transzformáció és materiális adatokhoz tartozó GPU buffereket bindolom a megfelelő shader index pozíciókra, így a shaderben az entitások komponenseihez tartozó indexek alapján hozzáférek a renderelendő alakzatok adataihoz. A bufferek bindolása után végigiterálok minden Shape komponenssel rendelkező entitáson, és ha a *FrustumCulling* optimalizációs rendszer alapján megjelenítendőnek bizonyosul az adott entitás, akkor a komponensekhez tartozó indexek shaderbe való feltöltése után elindítom az alakzat renderelését, ezt mutatja be a 133. ábra.

```
program->SetUniform("u_shapeModelIndex", shapeIndex);
program->SetUniform("u_transformIndex", transformIndex);
program->SetUniform("u_materialIndex", materialIndex);
program->SetUniform("u_entityIndex", entityIndex);
shapeComponent.shape->Bind();
glDrawElements(GL_TRIANGLES, shapeComponent.shape->GetIndexCount(), GL_UNSIGNED_INT, nullptr);
shapeComponent.shape->UnBind();
```

133. ábra - Alakzatok renderelése

A *RenderModel* metódus algoritmusa nagyon hasonló a *RenderShapes* algoritmusához, annyi különbséggel, hogy a komplex alakzatokat tartalmazó Model komponenssel rendelkező entitásokon iterálok végig. Az adott model betöltésénél elmentett anyaginformációkat bindolom a shaderhez, és a vertexeihez tartozó egyedi indexek alapján fogom a fragment shaderben azonosítani a materiális adatokat.

A *RenderShapesInstanced* és *RenderModelInstanced* metódusok az egyszerű és komplex alakzatok Instance renderelési technikával történő megjelenítését végzik. Ebben az esetben az entitásokat nem egyenként renderelem ki, hanem az azonos alakzattal rendelkező entitásokat egy rendehívással egyszerre rajzolom ki. Az Instance rendszer renderelés előtt meghatározta minden egyes alakzathoz és modelhez azokat a komponens indexeket, amelyek a shaderben szükségesek lesznek. A renderhíváshoz a *glDrawElementsInstanced* OpenGL függvényt használom, amely során szükséges megadni az Instance alakzatok számát paraméterként, ez látható a 134. ábrán. A shaderbe beépített egyedi InstanceID paramétert fogom használni a különböző entitások azonosítására, illetve ezt az azonosítót használom a indexeket tartalmazó shader tömb indexelésre is, amely indexek kiolvasása után képes leszek a tényleges komponens bufferekből meghatározni az entitáshoz tartozó adatokat.

```
for (auto& data : resourceManager->GetGeometryList())
{
    auto geometry = data.second;

    if (geometry->GetInstances().size() > 0)
    {
        geometry->GetInstanceSsbo()->BindBufferBase(2);
        geometry->Bind();
        glDrawElementsInstanced(GL_TRIANGLES, geometry->GetIndexCount(), GL_UNSIGNED_INT, nullptr, geometry->GetInstances().size());
        geometry->UnBind();
    }
}
```

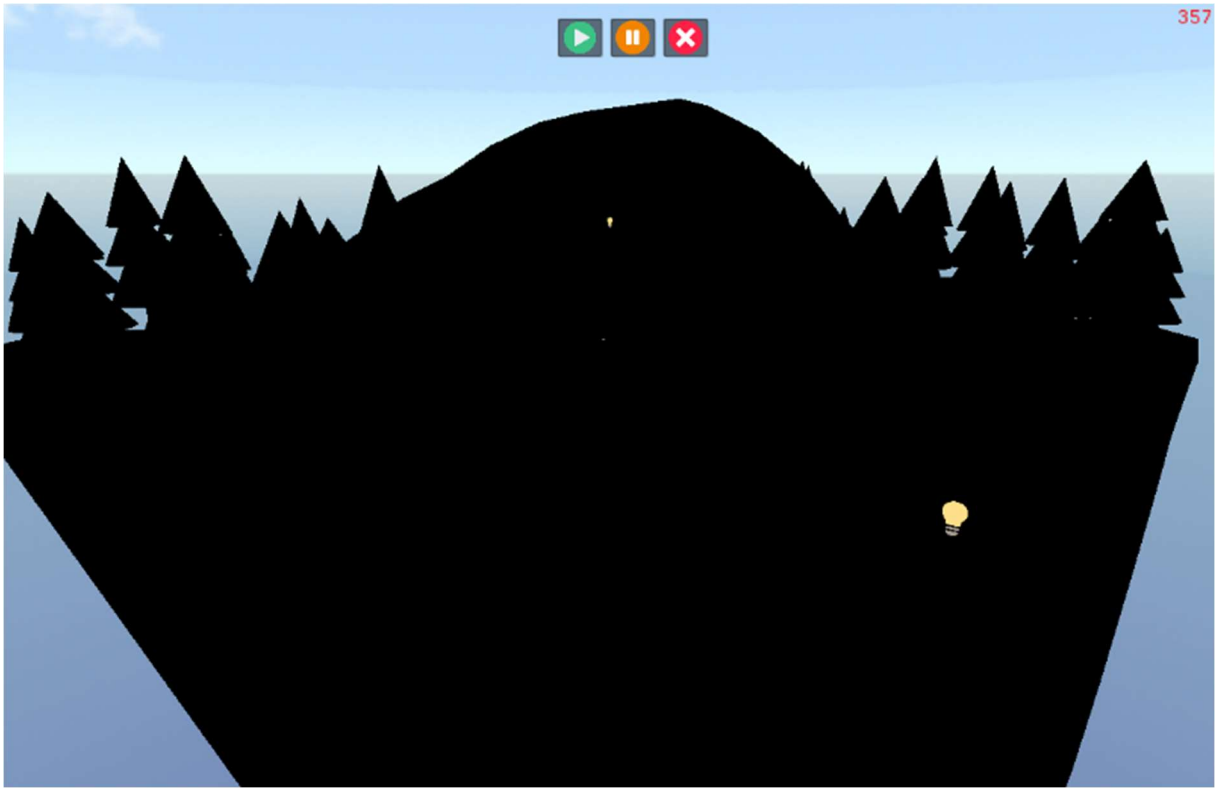
134. ábra - Alakzatok Instance renderelése

Végül a *RenderModelAnimated* metódusban az animációval rendelkező entitásokat renderelem ki. Ebben az esetben nem statikus alakzatot jelenítek meg, hanem folyamatosan változó, dinamikus alakzatokat. Ehhez a shader kódban az animációhoz szükséges csontadatok és transzformációk feltöltése is szükséges, és minden egyes vertexre ki kell számítani az adott időpillanathoz tartozó térbeli pozícióját, figyelembe véve a csontok hatását. Az animáció renderelése lényegesen összetettebb és erőforrásigényesebb folyamat. Az Instance renderelés elméletileg itt is alkalmazható lenne, de csak akkor, ha minden Instance ugyanazokat a csont és transzformáció adatokat használná, vagyis az összes modell azonos ütemben mozogna. Ennek a megvalósítását azonban nem látom túl praktikusnak, így ezt nem implementáltam.

3.5.4.2. Deferred Renderer

3.5.4.2.1 Általános technikák a renderelés során.

A **Deferred Renderer** osztállyal implementálom a deferred rendering technika második lépését, azaz az előzetesen létrehozott játékteret leíró textúrákat felhasználva szimulálom a fényforrások hatását. A színtér összeállításához előzetesen szükségem van egy teljesen fekete textúrára, ami a ténylegesen megjelenő játék textúrája lesz, ez látható a 135. ábrán. Minden egyes fényszimuláció során a pixelekhez kiszámolt szín adatokat fogom a fekete textúra megfelelő pixeléhez hozzáadni, és a szimulációk végén a tényleges játékteret megkapni.



135. ábra - Teljesen fekete kezdeti textúra (a skybox helyén is fekete)

A fényszimuláció során a fényekhez tartozó alakzatokat fogom kirenderelni és a létrejött fragmenteken a fényt szimulálni. Renderelés során nincsen szükség a mélységtextúrára, így a mélység tesztelését és a mélységértékek felülírását letiltom. A fragment shaderek adott fragmentjéhez tartozó output színértékét fogom a „fekete” textúra azonos fragmenthez tartozó színértékéhez hozzáadni. Ezt a mechanizmust érem el a 136. ábrán is látható OpenGL blendelési funkciók megfelelő beállításával.

```
glDisable(GL_DEPTH_TEST);
glDepthMask(GL_FALSE);
glEnable(GL_BLEND);
glBlendEquation(GL_FUNC_ADD);
glBlendFunc(GL_ONE, GL_ONE);
```

136. ábra - Blendelési opciók

Minden deferred renderinghez tartozó fragment shaderbe előzetesen feltöltöm a szín (color), normál vektorokat tartalmazó (normal), térbeli pozíció koordinátákat tartalmazó (positions) mélység értékeket tartalmazó (depth) és minden további mellékes adatot tartalmazó (additional) textúrákat. A fragment shaderekben ezután az adott fragmenthez tartozó textúra koordinátákat meghatározva kiolvasom az értékeket, amiket a fény szimulálásánál felhasználok.

3.5.4.2.2 Ambiens fény szimulálása

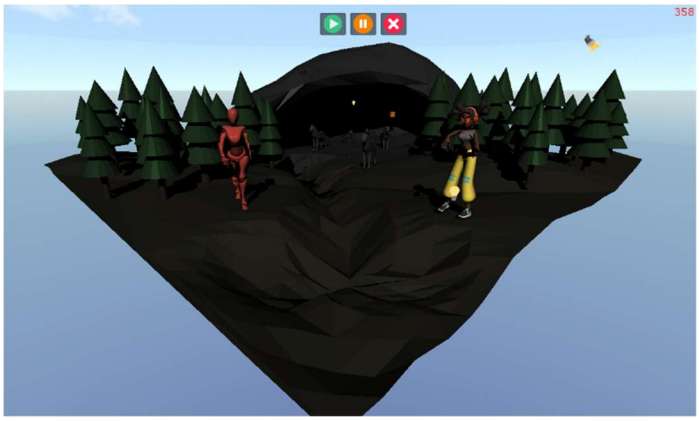
A *RenderAmbientColors* metódus felelős a színtér ambiens fényének szimulálásáért. Az ambiens fény egy konstans enyhe megvilágítást jelent a színtérben, amelynek köszönhetően a megjelenő alakzatok geometriája és anyagtulajdonsága fényforrás nélkül is látható. A shaderben a fragmentekhez tartozó színadatokat kiolvasom, és outputként azokat egy kicsi számmal, például 0.1 konstanst értékkel beszorzom. A shader output színértéke pedig automatikusan hozzáadódik a fekete textúrához, így egy még mindig sötét, de mégis az alakzatokat és színeket enyhén kivehető textúra jön létre, ezt reprezentálja a 137. ábra. Az ambiens fény valójában indirekt környezeti fényt szimulálja, amely a különböző felületekről egyenletesen szóródva éri el a tárgyakat.



137. ábra - Ambiens fényszimuláció (Enyhén láthatóak az objektumok)

3.5.4.2.3 Irányfényforrás szimulálása

A *RenderDirectionLights* metódusban implementálom az irányfényforrások szimulálását. Az irányfényforrás hatása a teljes játéktérre kihat, azaz az összes megjelenő objektumra, ezért renderelés során az összes fragment aktiválására szükségem lesz. Ennek eléréséhez, egy a kamera közeli vágósíkjára illesztett téglalap alakzatot fogok felhasználni renderelésnél, amely tehát teljesen lefedi a színtért. Az téglalap pontjait elegendő a vertex shaderben normalizált eszközkoordinátarendszerben definiálni, majd kimeneti paraméterként megadni, ezt mutatja be a 139. ábra. Előzetesen kamera adatait tartalmazó *CameraData* buffert, az irányfényforrások szimulálásához szükséges adatokat tartalmazó *DirLightData* buffert bindolom a shader megfelelő index pozíciójára. Mivel egy játékjelenetben több irányfényforrás is megtalálható, ezért azok hatékony szimulálásához az Instance rendering eljárást fogom alkalmazni. A vertex shaderben lekérdezhető InstanceID, ami alapján azonosítani tudom a fragment shaderben a fragmenthez tartozó irányfényforrás adatait. Az irányfényforrás szimulálásához a Phong árnyalást alkalmaztam, amely algoritmusa a 140. ábrán tekinthető meg.



138. ábra - Irányfényforrás szimuláció

```
vec4 positions[4] = vec4[4](
    vec4(-1,-1, 0, 1),
    vec4( 1,-1, 0, 1),
    vec4(-1, 1, 0, 1),
    vec4( 1, 1, 0, 1)
);

vec2 texCoords[4] = vec2[4](
    vec2(0,0),
    vec2(1,0),
    vec2(0,1),
    vec2(1,1)
);
```

139. ábra - Közeli vágósíkra illesztett NDC téglalap pozíciók

```
vec3 to_light = normalize(-light.direction);
vec3 to_eye   = normalize(camera.eye - fragment.position);

//Diffuse
vec3 diffuse = intensity * light.color * fragment.color * clamp(dot(normal, to_light), 0, 1);

//Specular
vec3 reflection = normalize(reflect(-to_light, normal));
vec3 specular = intensity * light.color * fragment.color * pow(clamp(dot(to_eye, reflection), 0, 1),  256);
```

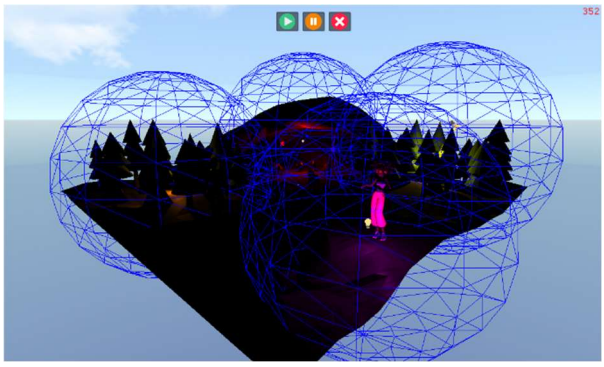
140. ábra - Phong irányfényforrást szimuláló algoritmus

3.5.4.2.4 Pontfényforrás szimulálása

A pontfényforrások szimulálását a *RenderPointLights* metódusban implementálom. Első lépésben aktiválom a színtér framebufferének a „fekete” textúráját outputként, majd letiltom a shaderek mélységtesztelését, illetve beállítom a megfelelő blendelési opciókat, hogy a fragment shader output színei megfelelően hozzáadódjanak a fekete textúra értékeihez. A kamera adatait tartalmazó *CameraData* buffert, a pontfényforrások gömb alakzataihoz tartozó transzformációkat tartalmazó *PointLightTransform* buffert, illetve a fényforrások szimulálásához szükséges adatokat tartalmazó *PointLightData* buffert bindolom a shader megfelelő index pozíciójára. A shaderbe továbbá feltöltöm a color (szín), normal (normál vektor), depth (mélység adat) és additional (minden más szükséges adat) textúrákat uniform változóként, amikből ki tudom majd olvasni a fragmentekhez tartozó textúra adatokat. A pontfényforrások által kifejtett hatás tökéletesen modellezhető egy gömb alakzattal, így az előbbiekben bemutatott közeli vágósíkra illesztett téglalap helyett egy kisebb felbontású gömb alakzatot fogok felhasználni renderelésnél, amely a kamera által látható tényleges 3D térben helyezkedik el. A fényforrásokhoz tartozó gömb alakzatok vizualizációja látható a 142. ábrán. Az alakzat renderelésénél a raszterizációs folyamat során csak azok a pixelek fognak aktiválódni, amelyek ténylegesen a gömb 2D vetületéhez tartoznak, így nagyságrenddel kevesebb számítást fog végezni a videókártya, hiszen ennek köszönhetően eliminálтam azokat a pixeleket, amelyekre biztosan nem ért volna el a fény hatása. Ennek a technikának a vizuális reprezentációja látható a 141. ábrán, ahol az aktiválódott pixelek színeződtek be.



141. ábra - Pontfényforrások által szimulált pixelek



142. ábra - Pontfényforrásokhoz tartozó hatását térben reprezentáló gömb alakzat

A színtérben bármennyi pontfényforrás megjelenhet, ezért a gömbök egyenkénti renderelésből adódó potenciális teljesítmény csökkenés elkerülése végett az alakzatokat Instance rendering technikával renderelem ki. A vertex shaderben az InstanceID azonosító felhasználásával tudom azonosítani egyenként a fényforrásokat, mely azonosítót felhasználva tudom lekérdezni az adott fényforráshoz tartozó adatokat a shader bufferekből. Egy adott pixel többször is aktiválódhat, hiszen több gömb is lefedheti ugyanazt a pixelt. Ilyen esetben több fragment jön létre, de a fragmentekhez tartozó azonosítók különböznek, így mindig a megfelelő fényforrást tudom szimulálni a shaderben. A pontfényforrás szimulálásához a Phong árnyalást alkalmaztam, amely algoritmusa a 143. ábrán tekinthető meg.

```
//Pontfényforrás felé mutató vektor
vec3 light_vec  = light.position - fragment.position;

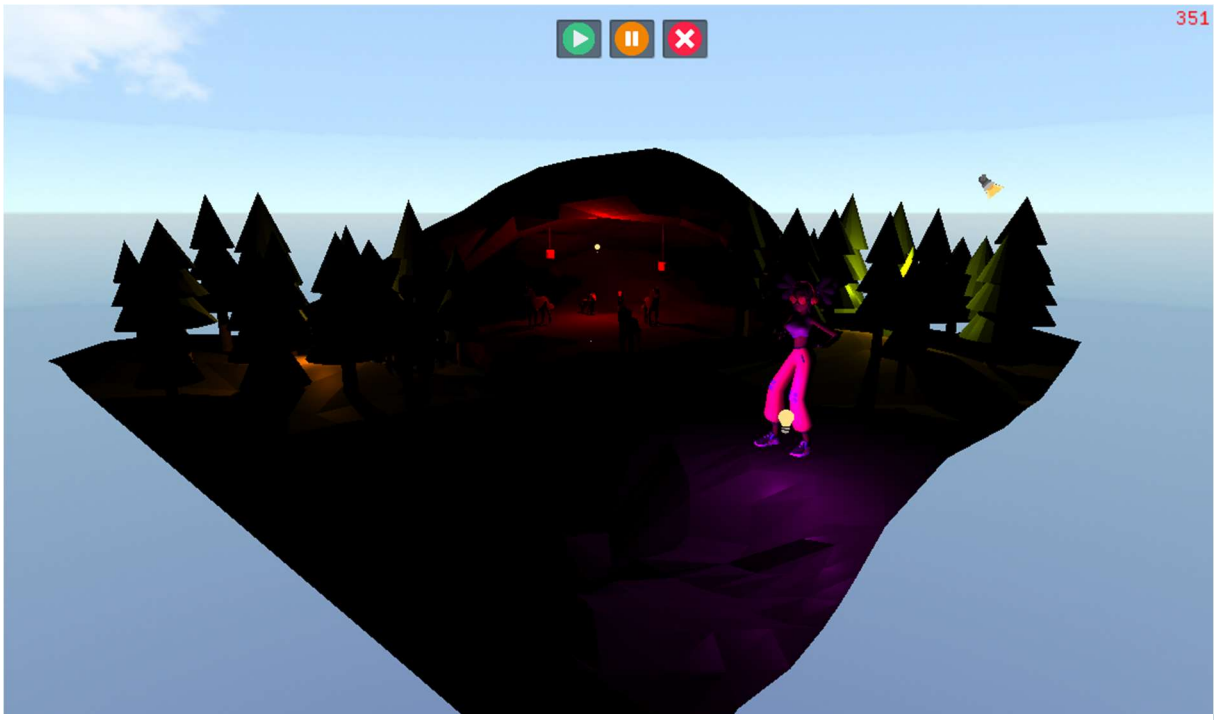
//Intenzitás számolás fényforrástól való távolsággal arányosan
float dist = length(light_vec);
float constant = 1.0;
float linear = 0.09;
float quadratic = 0.032;
float attenuation = 1.0 / (constant + linear * dist + quadratic * (dist * dist));
float intensity = attenuation;

vec3 to_light   = normalize(light_vec);
vec3 to_eye     = normalize(eye.xyz - position);

//Diffuse fényszimuláció
vec3 diffuse = intensity * light.color * fragment.color * clamp(dot(fragment.normal, to_light), 0, 1);

//Specular fényszimuláció
vec3 reflection = normalize(reflect(-to_light, fragment.normal));
vec3 specular = intensity * light.color * fragment.color * pow(clamp(dot(to_eye, reflection), 0, 1), 256);
```

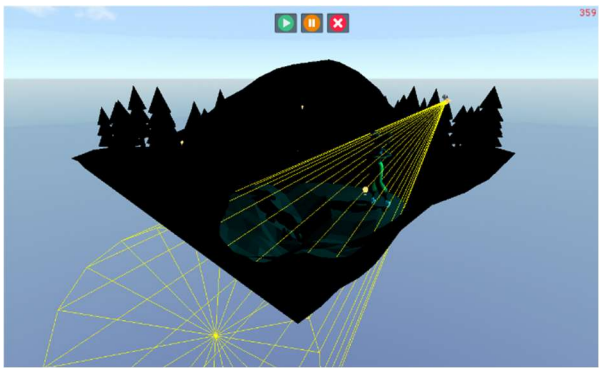
143. ábra – Phong pontfényforrás szimuláció



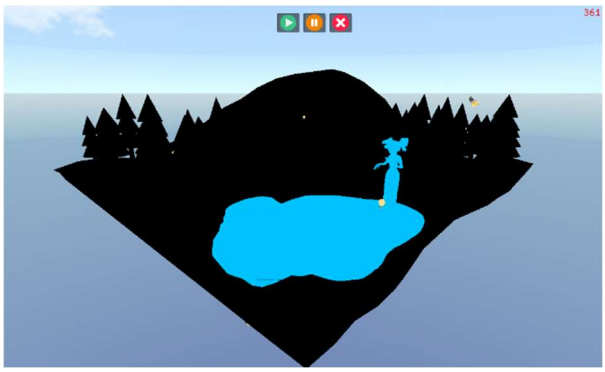
144. ábra - Pontfényforrások szimulálása utáni színtér állapot

3.5.4.2.5 Spotfényforrás szimulálása

A spotfényforrások szimulálását a *RenderSpotLights* metódusban implementálom. A metódus felépítése, gyakorlati megvalósítása nagyban megegyezik a pontfényforrásnál részletesen bemutatottal. A renderelés előtt feltöltöm a színtér textúráit és bindolom a spotfényforrásokhoz tartozó GPU buffereket. Ahogyan a 145. ábrán is látható, a spotfényforrás hatását egy kúp alakzat definiálja, így nem gömböket, hanem kúpokat fogok kirenderelni. Mivel több spotfényforrás is létezhet a játéktérben, ezért hasonló módon ebben az esetben is a hatékonyság miatt az Instance renderelést fogom használni. A renderelés során létrejövő fragmentek csupán a kúp alakzatok 2D vetületét definiálják, azonban előfordulhat, hogy a fragmentekhez tartozó pozíció adatok a kúp 3D alakzatán kívül esnek, ami azt jelenti, hogy a fényforrás semmilyen hatást nem fejt ki az adott fragmentre, azaz teljesen feleslegen számoltam a shaderben. Az ilyen fragmentek eldobásához implementáltam az előzetes metszés vizsgálatot, amely során a fényforrás irányát hasonlítom össze a fényforrás pozíciójából a fragment pozíciójába mutató vektorral. Az így megkapott bezárt szöget pedig felhasználom a kúp által definiált bezárt szöggel való összehasonlításra, amely segítségével eldönthető a metszés. Az algoritmus részletes implementációja a 147. ábrán tekinthető meg. A tényleges szimulációban résztvevő fragmenteket a 146. ábra mutatja be kékre színezve.



145. ábra – Spotfényforráshoz tartozó hatását térben reprezentáló kúp alakzat



146. ábra – Spotfényforrás által szimulált pixelek

```
//Check if fragment position is inside the proxy cone
vec3 fromSpotToFrag = position - spotLightData[fs_in_id].position.xyz;
vec3 fromSpotToFragNorm = normalize(fromSpotToFrag);
vec3 toSpotDirNorm = normalize(spotLightData[fs_in_id].direction.xyz);
float alpha = dot(toSpotDirNorm, fromSpotToFragNorm); // Angle tells if we are inside the cone
float delta = dot(toSpotDirNorm, fromSpotToFrag); //The projected length to check if inside the far plane
if(alpha < spotLightData[fs_in_id].angles.w || delta > spotLightData[fs_in_id].direction.w)
    discard;
```

147. ábra – Kúppal történő metszés algoritmus a shaderben

A spotfényforrás szimulálásához a Phong árnyalást alkalmaztam, amely algoritmus elméleti lépései a 148. ábrán tekinthetőek meg. A fényforrás szimulálása során a kúp belső szögén belül az intenzitás maximális, majd a belső szögtől a külső szög felé haladva lineárisan csökken a fényforrás által kifejtett hatás.

```
//Spotfényforrás felé mutató vektor
vec3 light_vec  = light.position - fragment.position;
vec3 to_light   = normalize(light_vec);

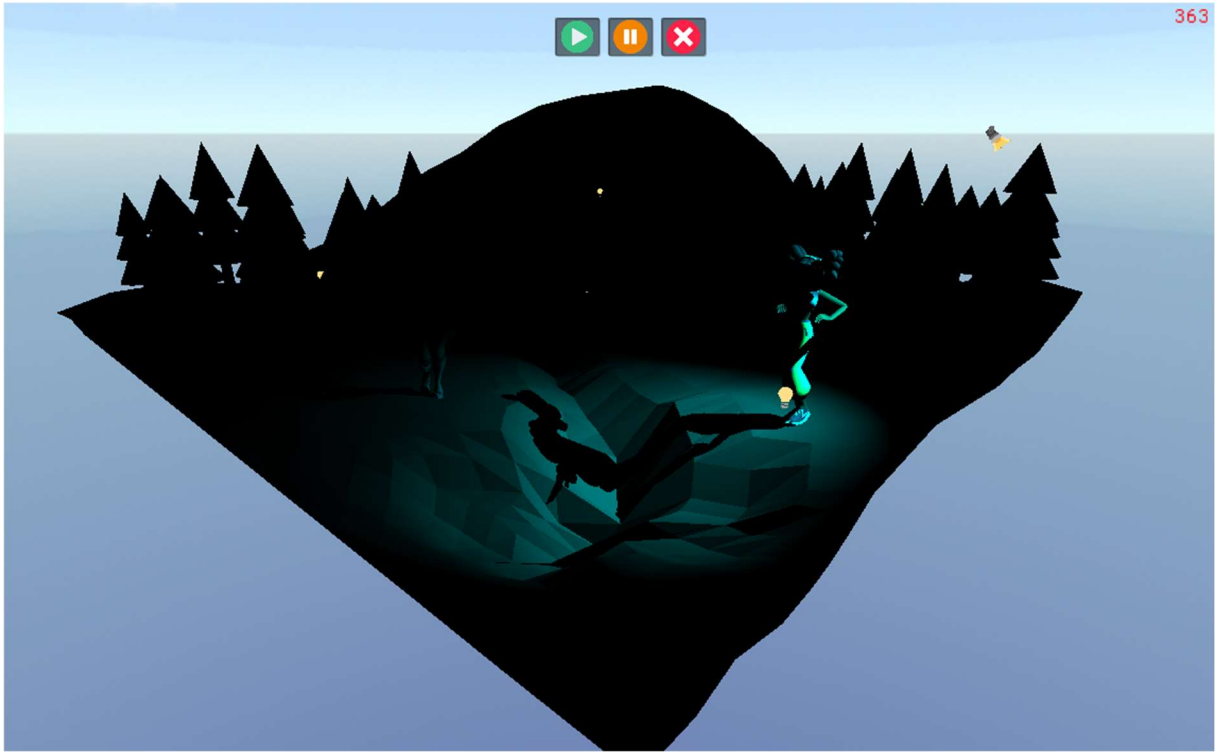
//Kamera felé mutató vektor
vec3 to_eye     = normalize(eye.xyz - position);

//Intenzitás számolás
float intensity = 1;
float cosTheta = dot(normalize(light.direction), -to_light);
if(cosTheta < light.cosInnerAngle)
{
    float theta = degrees(acos(cosTheta));
    intensity = clamp((light.outerAngle - theta) / (light.outerAngle - light.innerAngle), 0.0, 1.0);
}

//Diffuse fényszimulció
vec3 diffuse = intensity * light.color * fragment.color * clamp(dot(normal, to_light), 0, 1);

//Specular fényzimuláció
vec3 reflection = normalize(reflect(-to_light, normal));
vec3 specular = intensity * light.color * fragment.color * pow(clamp(dot(to_eye, reflection), 0, 1), 256);
```

148. ábra - Phong spotfényforrás szimuláció



149. ábra - Spotfényforrások szimulálása utáni színtér állapot

A spotfényforrások szimulálása után a kezdeti fekete textúra immáron tartalmazni fogja az ambient, irányfényforrás, pontfényforrás és spotfényforrás szimulációk végeredményét, amely színeket összegezve megkapom a játékjelenet színterét, ezt látható a 150. ábrán. A deferred renderelési lépések végeztével még van lehetőségem a színtérbe renderelni, viszont ekkor már csak a hagyományos forward rendering technikát alkalmazhatom. Többek között az ikonok, skybox, körbefogó alakzatok vizualizációja is a deferred renderelési lépéseket követően fognak megjelenni.

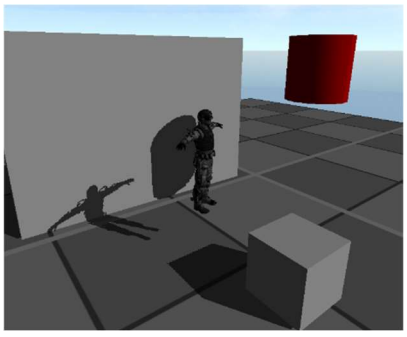


150. ábra - Fényszimulációk utáni színtér állapot

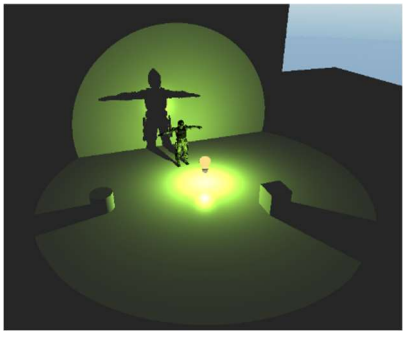
3.5.4.3. Shadow Renderer

Az irány, pont és spotfényforrások árnyékszimulálásának grafikai előkészítését implementáltam a **Shadow Renderer** osztályban. Az árnyékszimuláció során a *Shadow Mapping* technikát alkalmaztam. A technika lényege, hogy a fényforrások szempontjából kirenderelem a színteret, majd a megjelenő objektumok mélységértékeit elmentem a fényforrásokhoz tartozó mélység textúrába. A deferred renderelés fényszimulációja során a fragmentek világkoordinátarendszerbeli pozícióit a szimulált fényforráshoz tartozó kamera koordinátarendszerébe konvertálom. Az így megkapott pozíció vektor mélységértékét (z koordinátát) összehasonlítom a fényforrás mélységtextúrájába tárolt mélységértékkel, és amennyiben a fragment mélységértéke a textúra mélységértéke mögött helyezkedik el, akkor arra árnyék vetül, tehát a fényszimuláció során kiszámolt fényhatást megszüntetem, így elérve a sötét árnyékok vizuális megjelenését. Fontos megjegyezni, hogy a pont, és spot fényforrásoknál nem a teljes játékteret renderelem ki újra és újra. A Shadow Culling System meghatározta már előzetesen az összes fényforráshoz azokat az objektumokat, amelyek a fényhatásukat leíró alakzatokkal metszésben vannak, azaz amely objektumok az árnyékszimulációban potenciálisan részt vesznek, így elkerülve a teljesen felesleges, a fényforrások mélységtextúráiban soha meg nem jelenő objektumok renderhívásait.

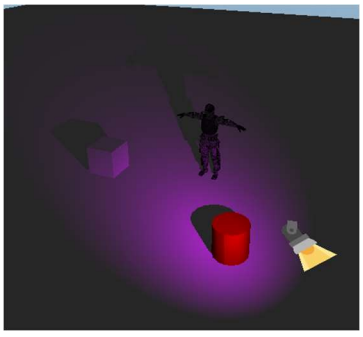
Az osztály *RenderDirLightShadows* metódusa felelős az irányfényforráshoz tartozó mélységtextúrák létrehozásáért, a *RenderPointLightShadows* metódus a pontfényforrások, míg a *RenderSpotLightShadows* a spotfényforrásokért felelősek. A szimulált fényforrások árnyékhatásra a 151-153. ábrákon tekinthetőek meg.



151. ábra - Irányfényforrás árnyékszimuláció



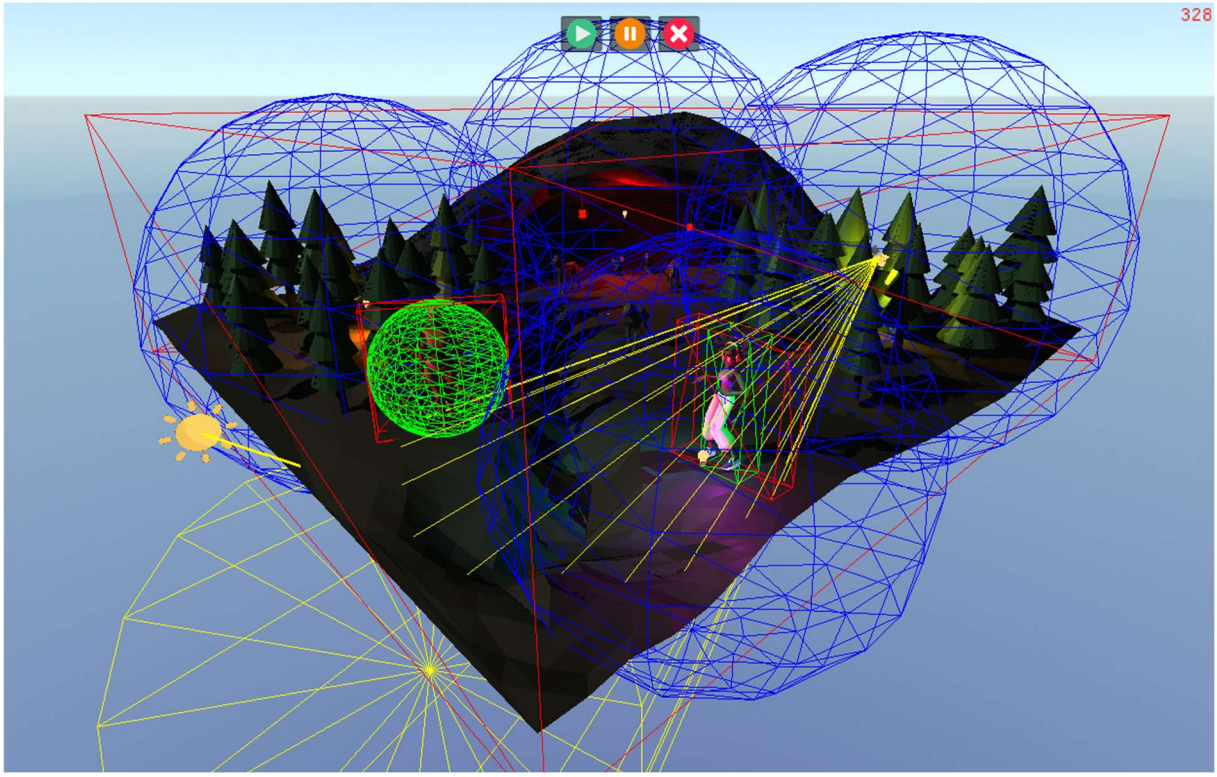
152. ábra - Pontfényforrás árnyékszimuláció



153. ábra - Spotfényforrás árnyékszimuláció

3.5.4.4. Wireframe Renderer

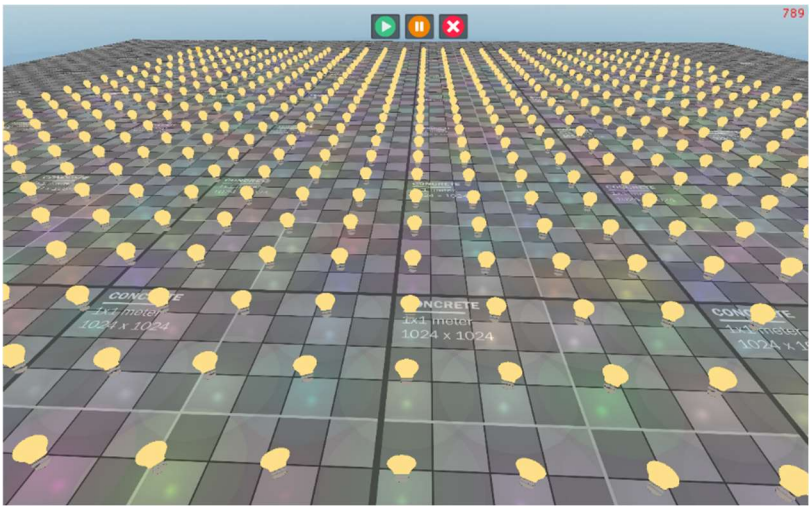
A **Wireframe Renderer** osztály felelős a fényforrások hatását reprezentáló geometriák és ütközési alakzatok hálószerű megjelenítéséért, ez látható a 154. ábrán. A *RenderDirLightsLine* metódusban implementálom az irányfényforrások ikonjaira illesztett, a fény irányát mutató segédvonalakat. A *RenderPointLightsVolume* és *RenderSpotLightsVolume* metódusok felelősek a pont- és spotfényforrások hatását reprezentáló gömb és kúp alakzatok megjelenítéséért. A default, box és sphere colliderrel rendelkező entitásokat körülvevő alakzatok megjelenítését a *RenderDefaultCollider*, *RenderBoxCollider* és *RenderSphereCollider* metódusokban valósítottam meg. Az alakzatok rendereléséhez a forward renderelési technikát alkalmaztam, illetve a hatékony megjelenítésért Instance technikát is alkalmaztam, a geometriák hálószerű megjelenítését pedig az **glPolygonMode(GL_FRONT_AND_BACK, GL_LINE)** opció beállításával értem el. Az alakzatok megjelenítését a grafikus interfészen be- és kikapcsolhatjuk, így tehát elegendő az adott játék fejlesztése során az alakzatokat megjeleníteni, a tényleges játékba pedig a megjelenítést letiltani, így a játék futása során jobb teljesítményt érhetünk el.



154. ábra - Wireframe alakzatok megjelenítése fényforrásokhoz és ütközési alakzatokhoz

3.5.4.5. Billboard Renderer

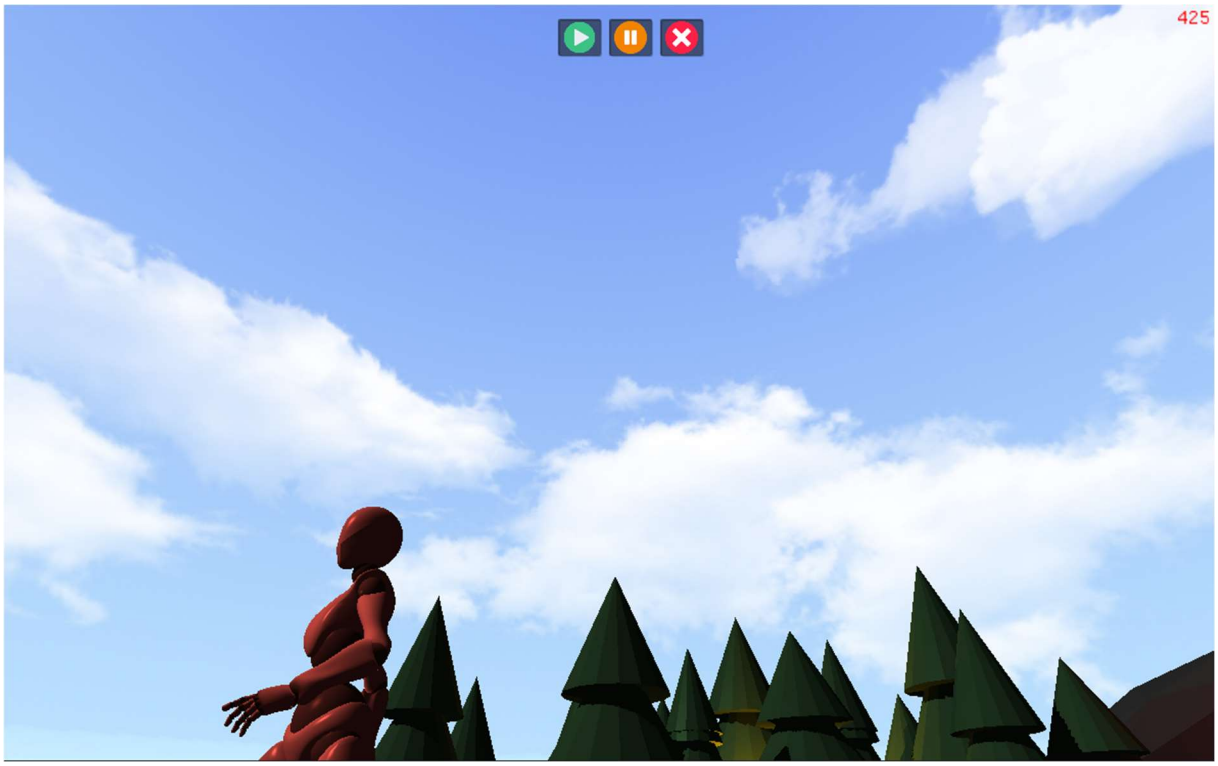
A billboarding egy olyan renderelési technika, amely során 2D felületeket szeretnénk megjeleníteni a 3D térben úgy, hogy a felület minden esetben a kamera felé nézzen. Esetemben ezt a technikát használtam fel a fényforrások és kamerák térbeli ikonjainak megjelenítésére (155. ábra), amelyhez szükséges metódusokat a **Billboard Renderer** osztályban implementáltam. A renderelés gyorsasága és egyszerűsége érdekében a megjelenő felületeket „on-the-fly" generálom a geometry shaderben, így kiküszöbölve a felesleges komplex forgatási mátrixok, transzformációk framenkénti újraszámolását. Az osztály tehát 4 metódust tartalmaz, a *RenderDirLightBillboard* az irányfényforrásokhoz tartozó ikonok, a *RenderPointLightBillboard* a pontfényforrásokhoz tartozó ikonok, a *RenderSpotLightBillboard* a pontfényforráshoz tartozó ikonok és a *RenderAudioBillboard* a hangokhoz tartozó ikonok renderelésért felelősek. A renderelés során csupán a felületek középpontjára és a megjelenítendő textúrára van szükségem. A felület középpontját a fényforrásokhoz/hangokhoz tartozó Transform Component eltolás vektora definiálja, amiket előzetesen már a megfelelő Systemek feltöltöttek tömbös formában a gpu bufferekbe. A vertex shader feladata meghatározni az adott gl_VertexID-hoz tartozó, felület közepét reprezentáló pozíció vektort, amit ezután a geometry shader fogok felhasználni. A geometry shaderben a kamera pozícióját felhasználva meghatározom azokat a *to_up* és *to_right* irányvektorokat, amelyek a kamera nézeti irányára merőlegesek, majd ezeket a vektorokat felhasználva létrehozom a középpont körül a felület négy csúcsát. A létrehozott csúcsokhoz definiálok textúra koordinátákat is, majd a fragment shaderben ráillesztem a megjeleníteni kívánt textúrát a felületre, és az átlátszó fragmenteket eldobom.



155. ábra - Billboard ikonok pontfényforrásokhoz

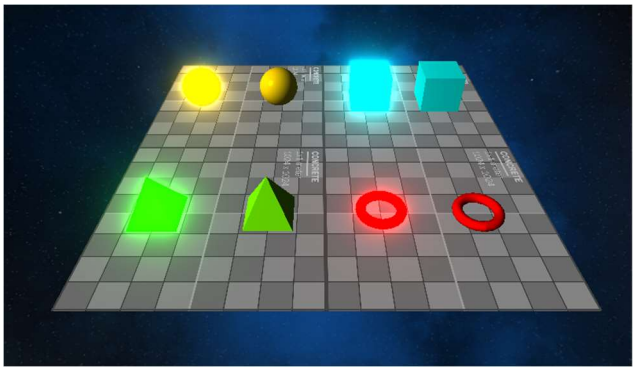
3.5.4.6. Skybox Renderer

A **Skybox Renderer** osztállyal implementálom a kocka alakú skybox textúrák megjelenítését, ami a 156. ábrán látható. A skybox renderelése az egyik utolsó renderelési folyamat, mivel ez rengeteg textúra olvasási és adatmódosítási műveletet jelenthet, ezért érdemesebb megvárni, hogy az összes objektum kirajzolódjon, mélységértéke a depth textúrában megfelelően eltárolódjon, így renderelés előtt beállítható, hogy csak azok a fragmentek aktiválódjanak, amikhez előzetesen nem nyúltam hozzá, azaz amelyek mélységértéke az alapértelmezett 1. Ennek a megoldásnak köszönhetően jelentős plusz teljesítmény érhető el. A skybox megjelenítéséhez tehát szükségem van egy GL_TEXTURE_CUBE_MAP típusú hat oldalból álló OpenGL textúrára, illetve a skybox pozícionálásáért és forgatásáért felelős transzformációs mátrixokra. A skybox alakzatot minden esetben a kamera pozíciójára kell eltolni, így tudom majd biztosítani, hogy bárhova megyek a térben, vizuálisan mégis mindig a skybox közepén maradok. A vertex shaderben a skyboxhoz tartozó mélységértéket trükkösen 1-re állítom (legmesszebbi mélységérték), így minden esetben, akár további objektum renderelése esetén is a skybox az objektumok mögött jelenik meg. A fragment shaderben a skyboxhoz tartozó interpolált pozíció vektorokat fogom felhasználni a 6 oldalú kockaszerű textúra mintavételezésre.



156. ábra - Skybox megjelenítése

3.5.4.7. Bloom Renderer

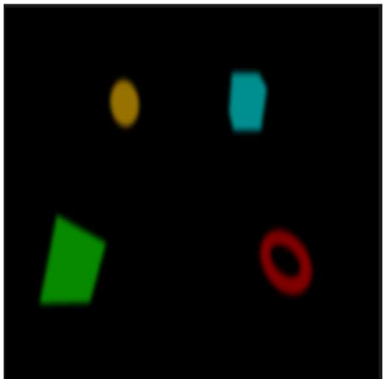


157. ábra - Világító bloom effektus alakzatokra

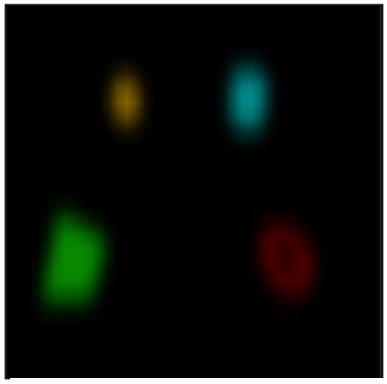
Az objektumok önvilágító effektusának szimulálásáért felelős a **Bloom Renderer** osztály. A bloom effektus egy alapvetőnek minősülő, rengeteg játékmotor által szimulált effektus, amely megvalósításához a Call of Duty játékszériában is felhasznált, egy eseményén bemutatott algoritmust fogom használni. Az algoritmus lépései a következők: Először szükségem van egy kiinduló textúrára, amely tartalmazza azon geometriák színadatait, amelyekre az effektust szimulálni fogom. Ezt a deferred renderelés első részénél könnyedén elérem, hiszen a szín, normál, mélység textúrák mellett egy újabb színtextúrába mentem azokat a színeket, amelyekre a bloom effektus opció aktiválva van. A kezdeti textúra mellett szükségem van további, esetemben 6 egymás után lépcsőzetesen felére csökkentett, kisebb felbontású textúrára is. Az effektus szimulálásához a kezdeti textúrát folyamatosan kisebb felbontású textúrába letömörítem, majd visszafele eljátszva feltömörítem, összegezve a színeket, így elérve végül a világító hatást. A letömörítési eljárás során a kisebb felbontású textúrában a nála eggyel nagyobb textúra adott pixele körül vett környezetének színadatait fogom súlyozottan átlagolni és eltárolni. Ezt a folyamatot fogom lépcsőzetesen ismételni, addig amíg a legkisebb felbontású textúrát el nem érem. A letömörítési eljárás végén a textúrákban egy egyre halványodó, elmosódó színhatás figyelhető meg, amit a 158-160. ábrák mutatnak be. A letömörítési eljáráshoz szükséges lépéseket implementáltam a *RenderBloomDown* metódusban, illetve a *BloomDown* shaderben találhatóak a textúra letömörítési algoritmusa.



158. ábra – Az 1. színtextúra letömörített tartalma

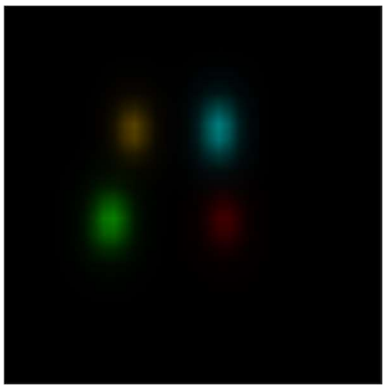


159. ábra – A 3. színtextúra letömörített tartalma (kicsit elmosott)

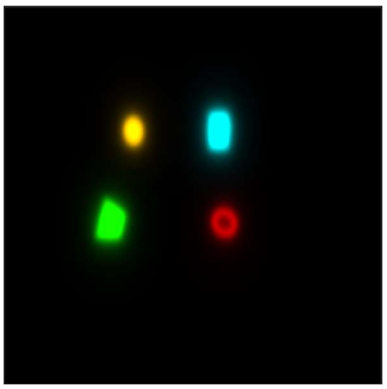


160. ábra – Az 5. színtextúra letömörített tartalma (nagyon elmosott és halvány)

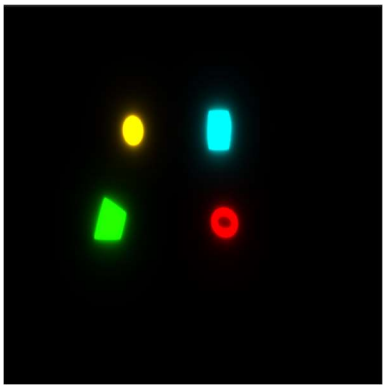
A feltömörítési eljárás során a legkisebb felbontású textúrától a legnagyobbig lépcsőzetesen haladva a textúrákat a megfelelő méretre feltömörítem egy hasonló, a pixelek környezetén alapuló színadatok súlyozott kiátlagolásával. A feltömörített textúra színadatait pedig a soron következő, azonos méretű textúrához fogom hozzáadni. A színtextúrák lépcsőzetes összegzése során egy egyre fényesebb, élesebben világító színhatást érek el a textúrákban, amik a 161-163. ábrákon is láthatóak. A feltömörítési eljárást az osztály *RenderBloomUp* metódusában implementáltam, a tényleges textúra színeinek kiátlagolási és felskálázási algoritmusát pedig a *BloomUp* shaderben implementáltam.



161. ábra - Az 5. színtextúra feltömörített tartalma

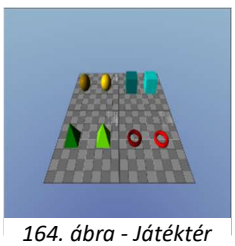


162. ábra - A 3. színtextúra feltömörített tartalma (Enyhén elmosottan világít)

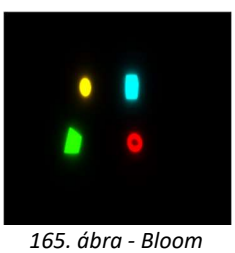


163. ábra - Az 1. színtextúra feltömörített tartalma (Élesen világít)

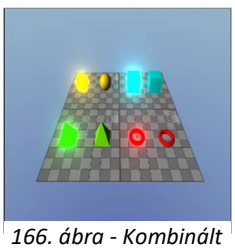
A feltömörítési eljárás végén a kezdeti színtextúrában, a már valósághűen világító hatást keltő színértékek lesznek megtalálhatóak. Mivel a végső textúra mérete megegyezik a játéktér színtextúrájának méretével, így utolsó lépésként a színtér színtextúrájához (deferred rendering outputja) hozzáadom a végső bloom színtextúra tartalmát, ezt mutatja be a 164-166. ábra. A bloom textúra fekete háttérszínének köszönhetően az eljárás biztosítja, hogy a bloom effektus szimulálásában nem résztvevő objektumok színei változatlanul maradjanak, míg a szimulációban résztvevő objektumok és közvetlen környezetük világítsanak. Ezt a lépést implementáltam az osztály *RenderBloom* metódusában. A játéktér színtextúrájának és a végső bloom textúra összemosása található meg a *Bloom* shaderben, ahol még további lehetőségek vannak a világítás erejének beállítására a tényleges megjelenítése előtt.



164. ábra - Játéktér színtextúra



165. ábra - Bloom textúra



166. ábra - Kombinált színtextúra

3.5.4.8. Water Renderer

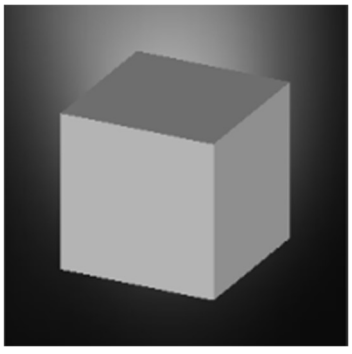


167. ábra – Vízszimuláció

A **Water Renderer** osztály feladata kétdimenziós vízszintes sík vízfelület szimulálása. A vízszimuláció egy nagyon költséges eljárás, amely során a teljes színteret kétszer újra ki kell renderelni, hogy a szimulációhoz szükséges textúrákat létrehozzam. A renderelés során a forward rendering módszert alkalmaztam. A fényszimuláció/árnyékszimuláció ezesetben túlságosan komplikált és költséges lett volna, ezért a vízfelszínen megjelenő objektumokra csak egy beégetett állású irányfényforrás hat, tehát sem a pontfényforrások, spotfényforrások, bloom effekt hatása nem tükröződik a vízfelszínen. A víz szimulálásához két textúrára lesz szükségem. Az egyik textúra a vízfelszínen való fénytörést fogja reprezentálni, ami a vízfelszín síkja alatt lévő játékteret fogja magába foglalni. A textúra létrehozásához kirenderelem a teljes játékteret (objektumok + skybox), majd a vertex shaderben a *gl_ClipDistance* beépített változót megfelelően beállítva eldobom a felszín felett elhelyezkedő vertexeket. A másik textúra a vízfelszínről történő fényvisszaverődést fogja reprezentálni, ami tehát a vízfelszín síkja felett lévő játékteret fogja magába foglalni. A textúra létrehozásához először a kamerát a vízfelszín alá kell pozícionálni (kamera pozíciójának és a vízfelszín síkjának magasságkülönbségével arányosan), hiszen így lesz látható a tükröződő, vízfelszín felett lévő térrészt, továbbá a kamera 'pitch' értékét meg kell fordítani (negálni), ami hatására a kamera által látott játéktér megfordul a textúrában Y irányban. Erre azért van szükség, mert ezeket a textúrákat a víz felszínére fogom majd vetíteni, és a tükröződő alakzatok az eredeti alakzatok irányával ellentétesen jelennek meg, ahogy a 167. ábrán látható. A víz szimulációjához szükséges textúrák létrehozását, az objektumok renderelését az osztály *RenderPreWater* metódusában implementáltam. A textúrák létrehozása után a RenderWater metódusban kirenderelem a tényleges játéktérbe a vízfelszín síkját reprezentáló téglalap alakzatot, amely alakzatra a *Water* fragment shaderben rávetítem a visszaverődési, és fénytörési textúrák összemixelt változatát. A vízfelszín hullámainak méretét és sebességét, illetve a hullámok mozgásának irányát a DuDv textúra felhasználásával számolom ki, amik a textúra mintavételezési koordinátákat módosítják olyan módon, hogy végül egy hullámzást imitáló effektust érjek el.

3.5.4.9. Preview Renderer

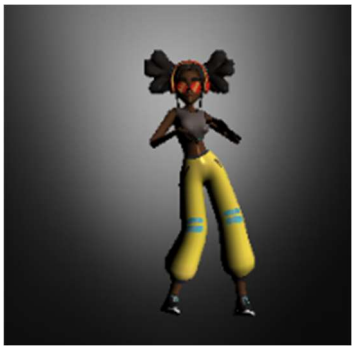
A **Preview Renderer** osztályban implementáltam a generált/betöltött geometriák és animációk alacsony felbontású előnézeti textúráinak generálását, amelyek a GUI komponens és filesystem ablakaiban fognak megjelenni, a felhasználóbarát játék fejlesztést elősegítve. A textúrákba a geometriákat és a textúra háttérszínét forward renderelési technikával fogom kirajzolni. Mivel a betöltött geometriák alakja és térbeli kiterjedése egymástól eltérhetnek, ezért a renderelés előtt úgy kell meghatározni a nézeti és ortografikus kamera mátrixokat, hogy végeredményében az alakzatok minden része megfelelően láthatóan, a teljes geometria mindig a textúra közepén jelenjen meg. Ehhez a geometriákat körülvevő téglatest alakú AABB alakzatok paramétereit fogom felhasználni, amely adatokat felhasználva az ortografikus mátrix mindig a geometria teljes egészét fogja a textúrákra projektálni. Ezesetben tehát nem a megszokott perspektív transzformációt fogok alkalmazni renderelésnél, mivel hatására a nagyobb geometriák túlságosan eltorzulnának a kamerától távolabb eső pontokon. Az ortografikus projekció által nyújtott vizuális megjelenés teljesen megfelelő az előnézeti textúrák generálásához. Az alakzatok/modellekhez tartozó előnézeti textúrát elegendő egyszer létrehozni (1 renderhívás) és eltárolni, az animációkhoz tartozó dinamikus textúrák létrehozásához pedig folyamatos időközönkénti renderelése van szükség, hiszen az animáció folyamatosan változtatja a betöltött model részeinek pozícióit, és így tudom elérni a mozgó hatást keltő előnézeti textúrát. A Preview Renderer osztály *RenderShapePreviews* metódusa felelős a generált alakzatok, a RenderModelPreviews metódusa a betöltött modellek és a *RenderAnimationPreviews metódus pedig az animációkhoz tartozó* előnézeti textúra legenerálásért, amely textúrák láthatóak a 168-170. ábrákon.



168. ábra - Egyszerű kocka alakzat előnézeti textúrája



169. ábra - Betöltött komplex model előnézeti textúrája

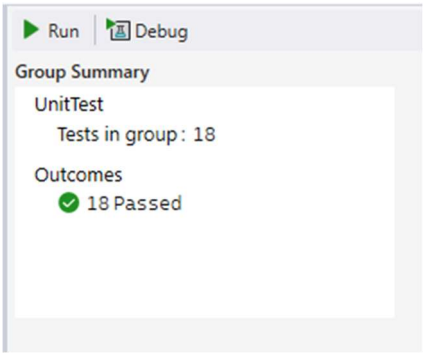


170. ábra - Betöltött animáció mozgó előnézeti textúrája

3.6. Játékmotor tesztelése és mintajátékok

3.6.1. Pool és Registry osztályok unit tesztelése

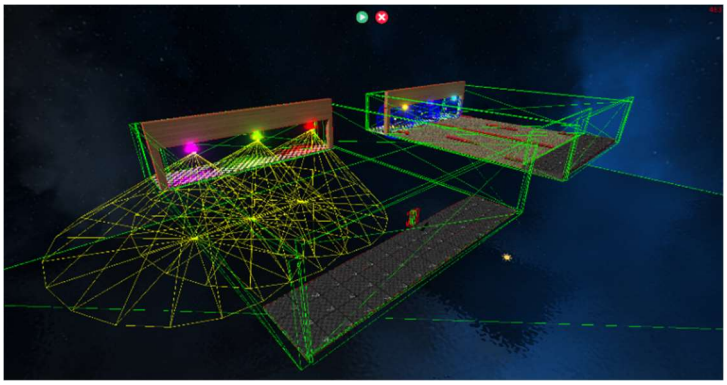
A játékmotor hátterét képző komponenseket és entitásokat menedzselő **pool** és **registry** osztályok tesztelési kódjai a *UnitTest* projektben találhatóak meg, amely tesztek sikerességét a 171. ábra mutatja be. A tesztelési kódokban az entitásokhoz történő komponens regisztrálástól kezdve, az entitások közötti gyerek-szülő kapcsolat kiépítésén és törlésén át, az adatorientált *Pool* osztályban implementált Sparse Set adatstruktúra részletes tesztelése is megtalálható.



171. ábra - Unit tesztek sikeressége

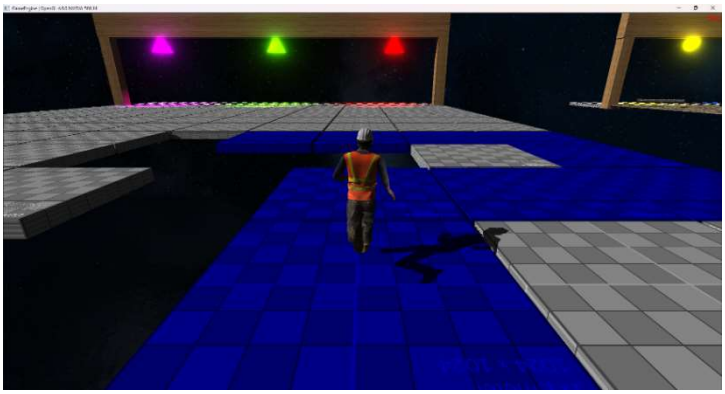
3.6.2. Ügyességi tesztjáték (End-to-end)

A játékmotorban elkészítettem egy három pályából álló ügyességi játékot. A játék tartalmazza szinte az összes játékmotorban implementált jelenséget, és ezek együttes hatása, eredménye látható a 172. ábrán.



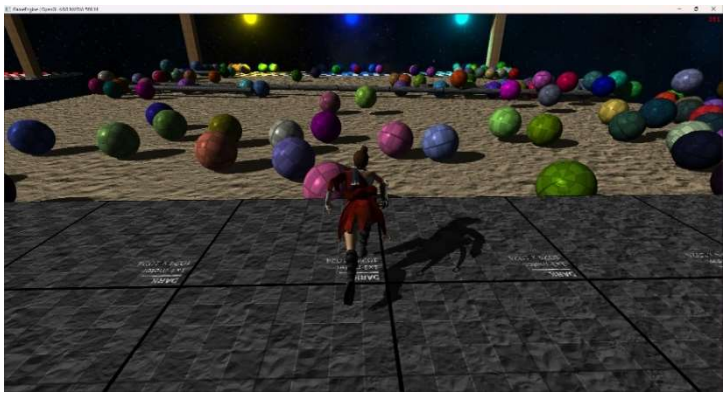
172. ábra - Ügyességi játék elemei az editorban

A 173. ábrán látható első pálya kezdetén egy 10x10 játékmező generálódik le. A játékos célja, hogy megtalálja azt az útvonalat a mezőelemeken haladva, amelyek biztonságosak, így elérve a kijelölt célt. A biztonságos elemekre lépve azok kékre változnak. Veszélyes platformra lépve a játékos lezuhan, majd vízbe érkezése után a pálya elejére érkezik vissza, ahonnan újra elkezdheti a biztonságos útvonal felderítését. Célba érve a játékos karaktere 10 másodpercig táncolni fog, majd a második játékpályára teleportál át.



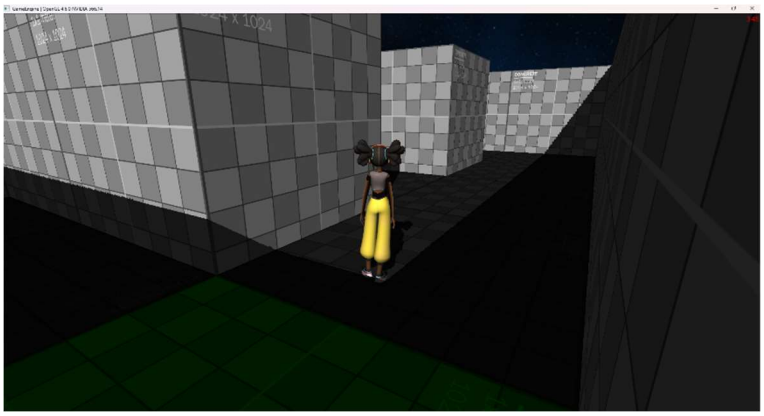
173. ábra - Első ügyességi játékpálya (Biztonságos útkeresés)

A 174. ábrán látható második pályán a játékos feladata úgy elfutni a célba, hogy semmilyen mozgó fizikai objektummal nem ütközik. Amikor a játékos először a játékmezőre lép, 225 darab fizikai gömb generálódik le, amelyek a magasból zuhannak a játékosra. A pályán elhelyeztem 7db vízszintesen forgó elemet is, amelyek a legenerált gömbökkel is ütköznek, azok útvonalát eltérítve, így megnehezítve a játékosnak az átfutását a pályán. Amikor a játékos célba ért, a karaktere elkezd táncolni, majd a harmadik pályára teleportál át.



174. ábra - Második pálya (Végigfutás ütközés nélkül)

A 175. ábrán látható harmadik játékpálya egy 20x30-as teljesen random generált labirintus, ahol a játékos célja megtalálni a labirintus végét, amit egy piros mező jelez. A piros mezőre lépve a játékos karaktere elkezd táncolni, majd visszaérkezik az 1. pályára, ahonnan újra el tudja kezdeni a játékot.

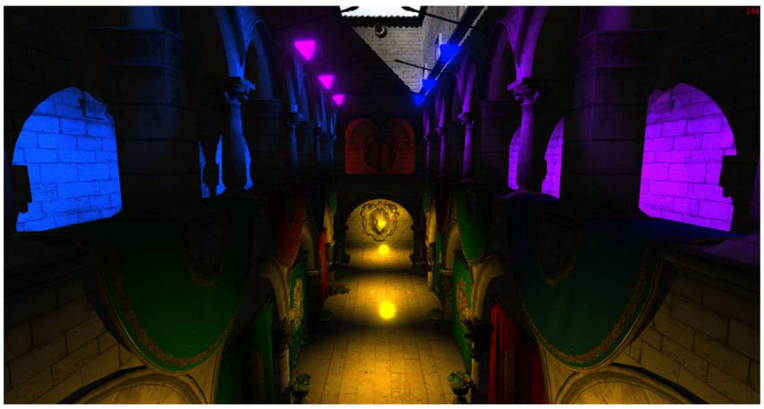


175. ábra - Harmadik pálya (Random Labirintus)

A játék bemutatja, hogy komplex karakter modelleket és animációkat is működőképesen betölthetünk a játékmotorba. A fizikai szimulációt is nagyszerűen reprezentálja mindhárom játékpálya, illetve a második pálya az ütközésekből kiváltódó eseményekhez kötött felhasználói kódok végrehajtását is jól bemutatja. A különböző fényforrások, árnyékok, és más grafikai jelenségek is megtalálhatóak a játékban. Az elkészített játék nagyban épít a felhasználói scriptkódokra, hiszen az első és harmadik pálya mezőelemei, illetve a második pálya fizikai gömbjei teljesen scriptből generálódnak le. A játékos karakterének és kamerájának mozgását, a mozgáshoz köthető különböző animációk cseréjét, a pályákhoz köthető hangokat is scriptből töltöttem be, így a felhasználó által írt külső scriptkódok működését is remekül bemutatja az elkészített tesztjáték.

3.6.3. Grafikai tesztelés

A játékmotor grafikai teszteléséhez betöltöttem a **Sponza** modelt, ez látható a 176. ábrán. Ez a model egy 200.000 vertexből és 45 különböző textúrából álló geometria. A teszt célja bemutatni, hogy nagyon komplex alakzatokat és rengeteg textúrából álló modelleket is képes a játékmotor betölteni, és megjeleníteni. A jelenetben még elhelyeztem 1db irányfényforrást, 10 darab pontfényforrást, és 10 darab spotfényforrást, amelyekkel együttesen árnyékot is szimulálok. A fényforrásokhoz elhelyeztem még az őket reprezentáló gömb és kúp alakzatokat is, amelyek a bloom effektusnak köszönhető a fényforráshoz tartozó színnel világítanak. A grafikai teljesítmény tesztelése a 177. ábrán tekinthető meg.



176. ábra - Sponza model

FULL HD Felbontásban	GPU: RTX 4060 CPU: I5-13600K	GPU: RTX 4050 (Laptop) CPU: I5-12450HX (Laptop)	GPU: GTX 1050TI CPU: I5-7400
Mindennel együtt	300 fps	240 fps	95 fps
Árnyékszimuláció nélkül	400 fps	310 fps	135 fps
Alakzatok és bloom effektus nélkül	450 fps	350 fps	155 fps
Pont és Spot Fényforrások nélkül	600 fps	460 fps	200 fps

177. ábra – Sponza Benchmark (Bal oldali oszlopban az elemek egymásra épülnek, tehát az „alakzatok és bloom effektus nélküli" sorban már az árnyékszimuláció sincs benne)

4. Összefoglalás és további fejlesztési lehetőségek

Szakdolgozatomban sikerült megvalósítanom egy viszonylag komplex, sok elemből álló játékmotort, amellyel egyszerűbb háromdimenziós játékokat lehet elkészíteni. A szakdolgozatomban bemutattam részletesen egy olyan adatorientált ECS megvalósítást, amely a komponensek optimális frissítését és tárolását biztosítja. Rengeteg komponenst és rendszert implementáltan a játékmotorban, melyek együttes hatásával szimulálom az elkészített játéktér elemeit. A színtér életre keltéséhez szükséges grafikai eljárásokat is implementáltam. Ahol lehetőségem volt, optimalizációs eljárásokat is létrehoztam a játékmotorban, a maximális teljesítmény elérése érdekében. Összességében egy komplex, letisztult, a felhasználó által is könnyen kezelhető játékmotort sikerült megvalósítanom szakdolgozatként.

Szakdolgozatom rengeteg potenciális fejlesztési lehetőséget foglal magába:

Vulkan renderelési api: A manapság már elavultnak számító OpenGL renderelési api helyett használhatnám a modernebbnek minősülő platformfüggetlen Vulkan api-t, amivel sokkal hatékonyabb renderelés lenne megvalósítható. A Vulkan támogatja a többszálról történő GPU függvényhívásokat, amelynek köszönhetően a jelenleg egy főszálon futó játékmotor szétbontható lenne több (render + update) szálra, amivel sokkal jobb teljesítmény lenne elérhető.

Fizikán alapuló renderelés: A mai játékmotorok, modellező szoftverek már valós fizikán alapuló anyagmodelleket, renderelési eljárásokat implementálnak a valósághűbb megjelenés érdekében. A szakdolgozatomban egy egyszerű diffúz színből álló, PHONG árnyalási modellt használó renderelést implementáltam, amely helyett a fizikán alapuló renderelés sokkal szebb végeredményt biztosítana.

Saját fizikai szimuláció: A játékmotorban felhasznált PHYSX könyvtár helyett a beépített GJK ütközésvizsgáló algoritmussal, illetve az arra épülő EPA algoritmussal implementálható lenne egy saját fizikai szimulációs rendszer. Természetesen a PHYSX könyvtár által nyújtott lehetőségektől a saját fizikai rendszer jóval elmaradna, de a szilárdtestek közötti erőhatások és fizikai jelenségek megértése érdekében hasznos továbbfejlesztési lehetőség lenne.

5. Köszönet nyilvánítás

Elsősorban köszönettel tartozom témavezetőmnek, **Pataki Norbertnek**, aki a témaválasztás kezdetétől fogva támogatott a szakdolgozatom elkészítésében. Rengeteg segítséget nyújtott a dokumentáció színvonalas elkészítésében, a szakdolgozatomhoz mindig érdeklődően, ötletelve állt hozzá. Bármilyen kérdés felmerülése esetén emailben nagyon hamar és szakértelemmel válaszolt.

Köszönetet szeretnék nyilvánítani az **ELTE grafikai kurzusokért** és az ezért felelős oktatóknak, mivel nekik köszönhetően ismertem meg a grafikus programozás alapjait. A kurzusokban bemutatott grafikai programozás annyira megtetszett, hogy azóta is szabadidőmben rendszeresen fejlesztek játékmotorhoz köthető elemeket.

Végül köszönettel tartozom a **learnopengl.com** weboldalért felelős fejlesztőnek, aki az oldalon a grafikai programozáshoz szükséges magas színvonalú segédanyagokat publikált. A haladóbb színtű grafikus jelenségek megértésében rengeteget köszönhetek az oldalnak.

6. Irodalomjegyzék

C++17 referencia:
https://en.cppreference.com/w/cpp/17 (2024.11.24)

Grafikai jelenségek:
https://learnopengl.com/ (2024.11.24)

Adatorientált ECS elmélete:
https://skypjack.github.io/ (2024.11.24)

GJK ütközésvizsgálat és algoritmusa:

https://winter.dev/articles/gjk-algorithm (2024.11.24)

Vízszimuláció elmélete: (2024.11.24)
https://www.youtube.com/playlist?list=PLRIWtICgwaX23jiqVByUs0bqhnalNTNZh

Játékmotor fejlesztésének ötlete: (2024.11.24)
https://www.youtube.com/playlist?list=PLlrATfBNZ98dC-V-N3m0Go4deliWHPFwT